

Range Query



Range Query



Range Query

**Range Query
using
Hash Table**

Range Query vs Point Query

Range Query

Point Query

Range Query vs Point Query

Range Query

Tuples where a
column is in a
Range of Values

Point Query

Range Query vs Point Query

Range Query

Tuples where a column is in a Range of Values

Point Query

Tuples where a column is equal to a Single Value

Range Query vs Point Query

Range Query

Tuples where a
column is in a
Range of Values

Products priced
between
\$10 and \$20

Point Query

Tuples where a
column is equal to
a Single Value

Range Query vs Point Query

Range Query

Tuples where a
column is in a
Range of Values

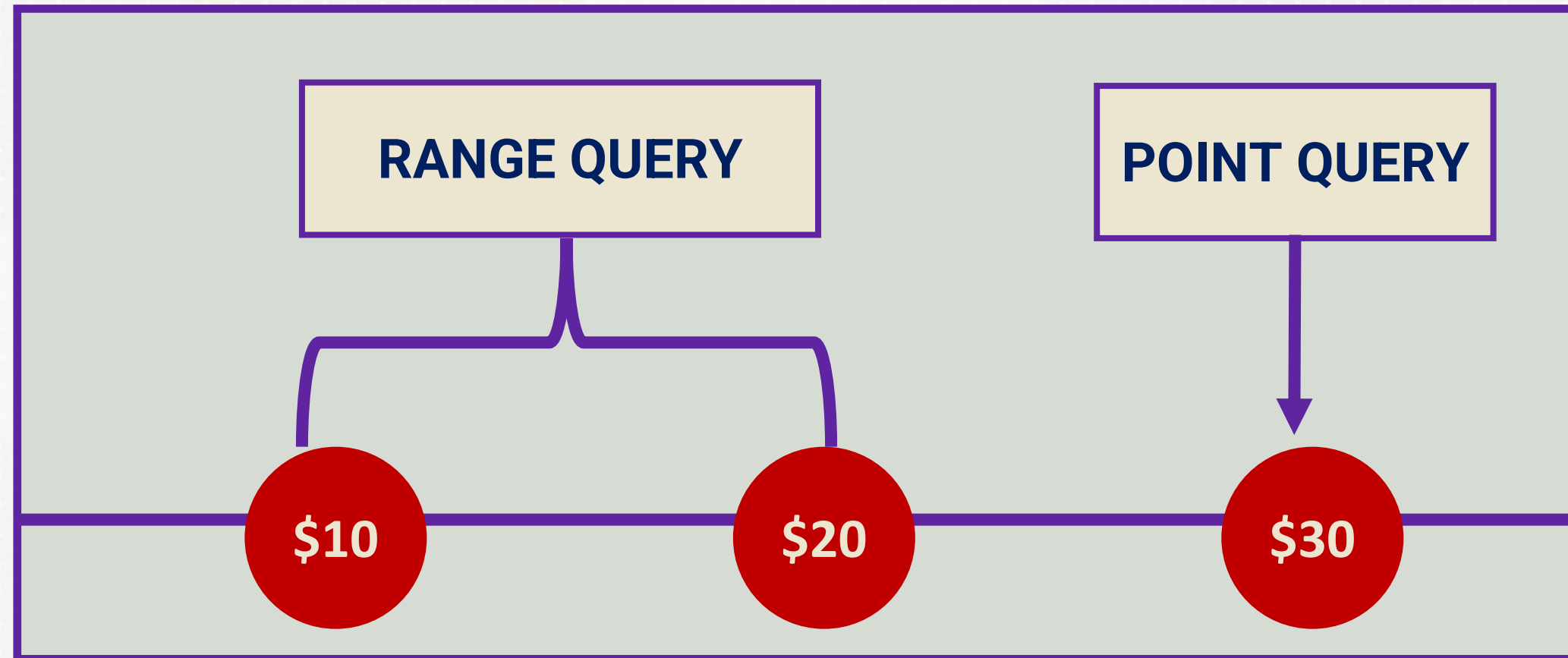
Products priced
between
\$10 and \$20

Point Query

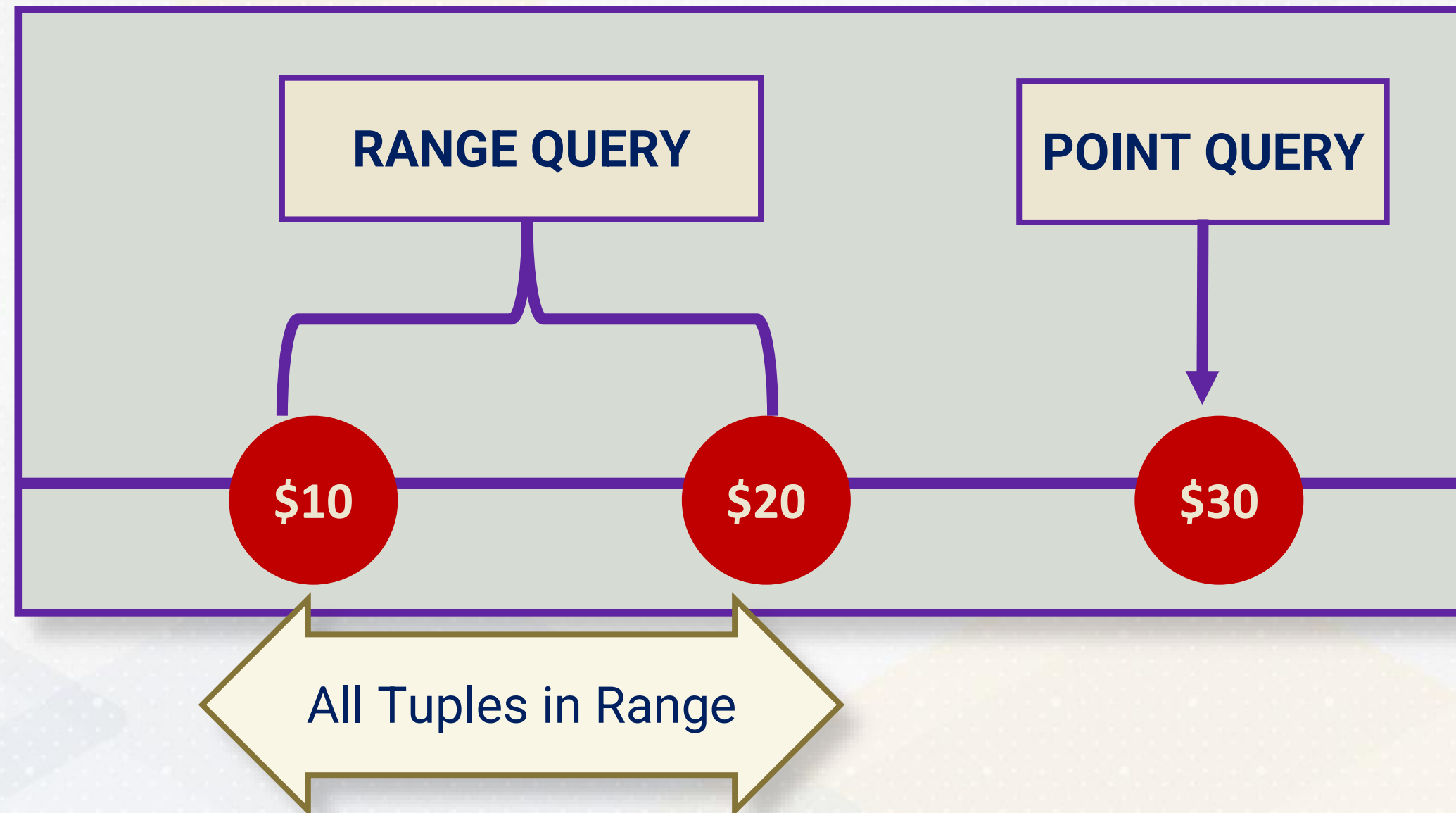
Tuples where a
column is equal to
a Single Value

Products priced
\$30

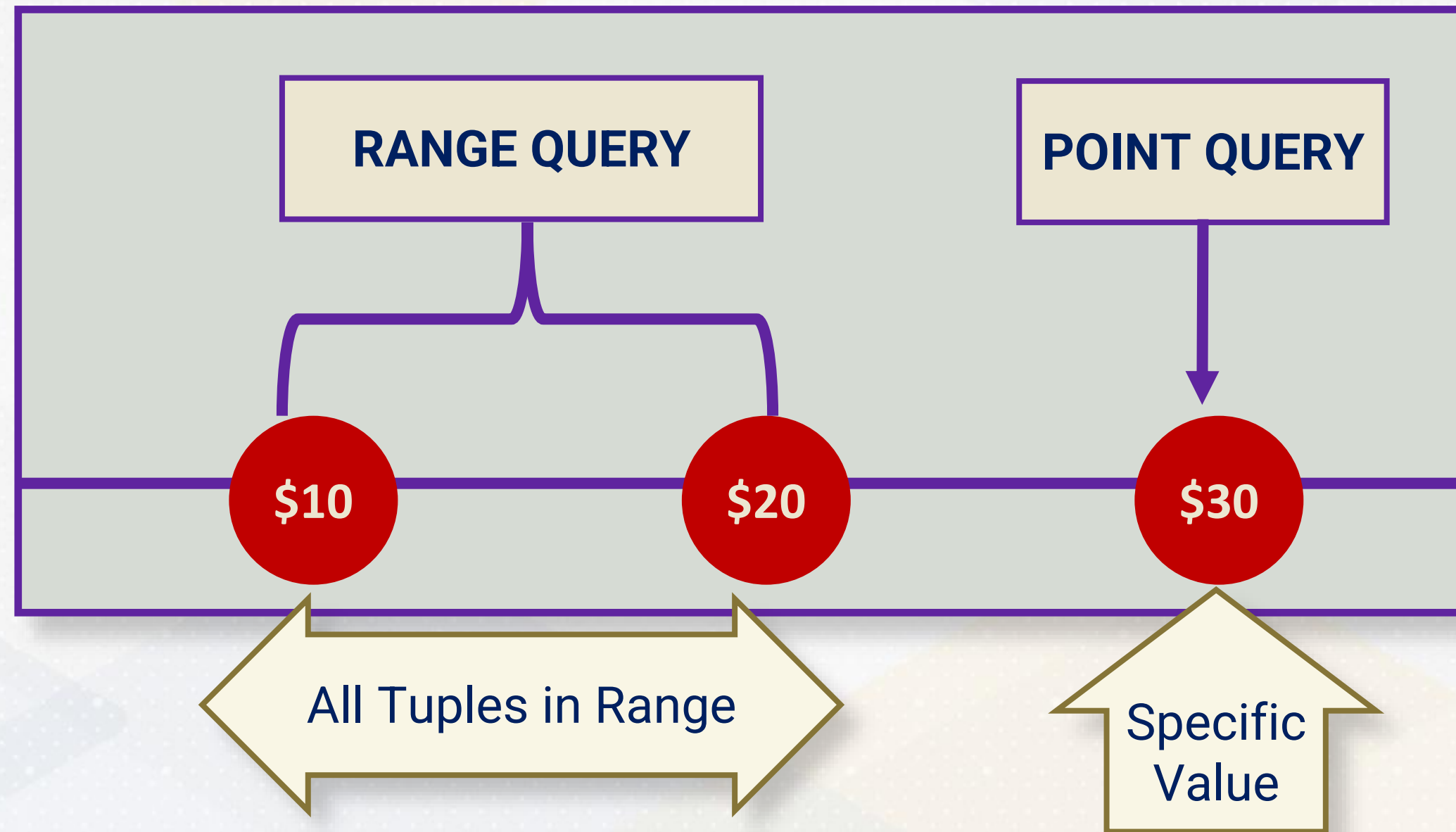
Range Query vs Point Query



Range Query vs Point Query



Range Query vs Point Query



Point Query Using Hash Table Index

Look up a key in the hash table with $O(1)$ average complexity.

```
std::vector<int> tuple_ids = productIndex.getValue(30);
```

POINT QUERY



0	1	2	3	4	5	6	7	8	9
\$30		\$12			\$25		\$17		
[103, 105]		[104]			[101]		[106, 102]		

Point Query Using Hash Table Index

Hash Table

Not ordered based on key

Iterate through entire table

RANGE QUERY

0	1	2	3	4	5	6	7	8	9
\$30		\$12			\$25		\$17		
[103, 105]		[104]			[101]		[106, 102]		

Point Query Using Hash Table Index

```
std::vector<int> rangeQuery(int lowerBound, int upperBound) const {  
    std::vector<int> results;  
    for (size_t i = 0; i < capacity; ++i) {  
        if (hashTable[i].exists && hashTable[i].key >= lowerBound &&  
            hashTable[i].key <= upperBound) {  
            results.push_back(hashTable[i].value);  
        }  
    }  
    return results;  
}
```

Inefficient Range Query in BuzzDB



buzzDB

```
void performRangeQueryWithHashIndex(int lowerBound, int upperBound) {  
    auto results = index.rangeQuery(lowerBound, upperBound);  
    std::cout << "Found " << results.size() << " records in the range.\n";  
}
```



Range Query



Range Query

**Range Query
using
Hash Table**



Ordered Index







Ordered Index



**Ordered
Index**

**B+Tree
Data
Structure**

Ordered Index

RANGE QUERY **[\$10, \$20]**

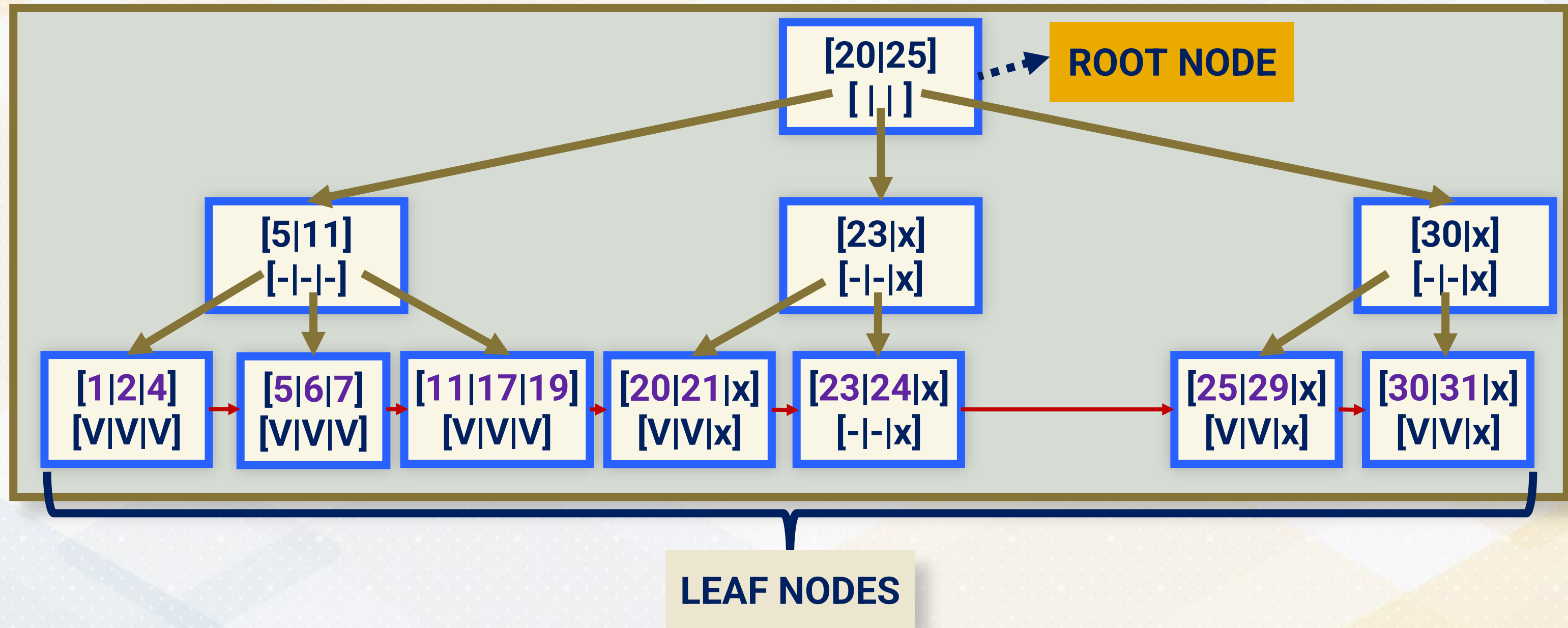
0	1	2	3	4	5	6	7	8	9
			\$12		\$17			\$25	\$30
			[104]		[106, 102]			[101]	[103, 105]

Unordered Index

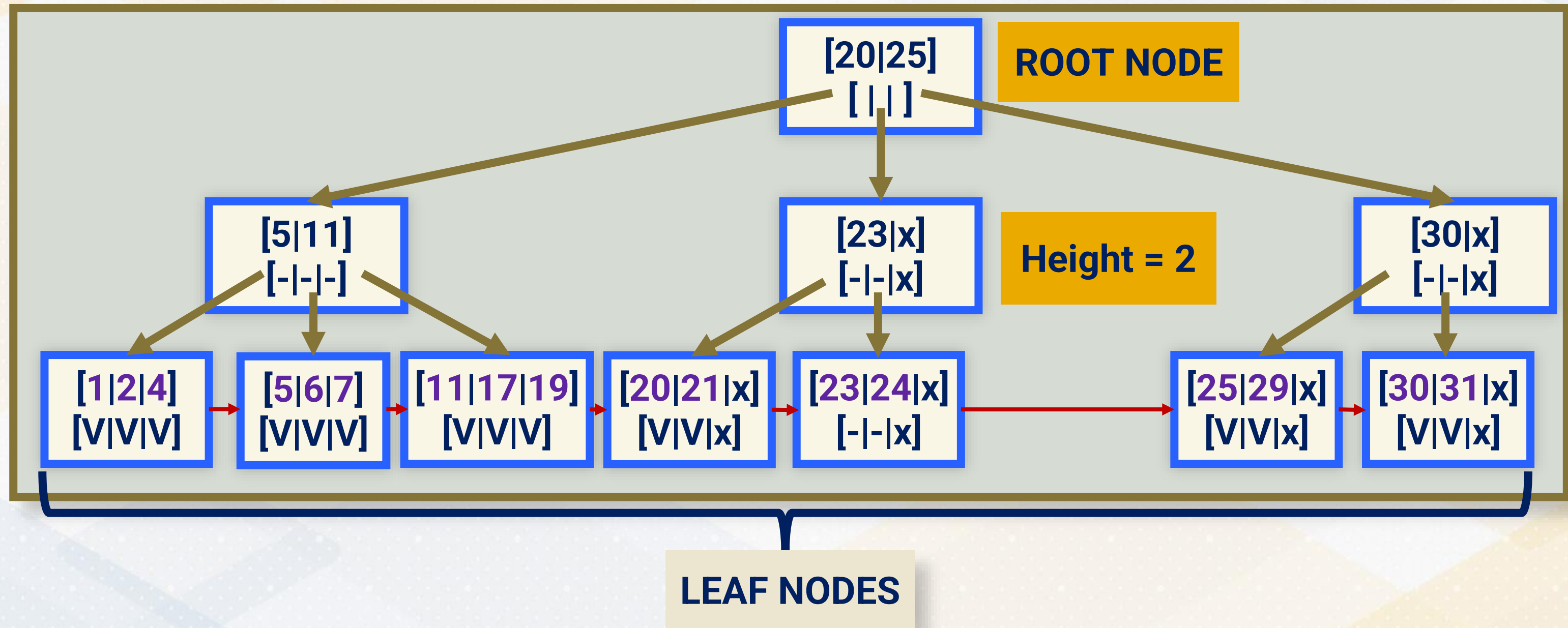
RANGE QUERY **[\$10, \$20]**

0	1	2	3	4	5	6	7	8	9
\$30		\$12			\$25		\$17		
[103, 105]		[104]			[101]		[106, 102]		

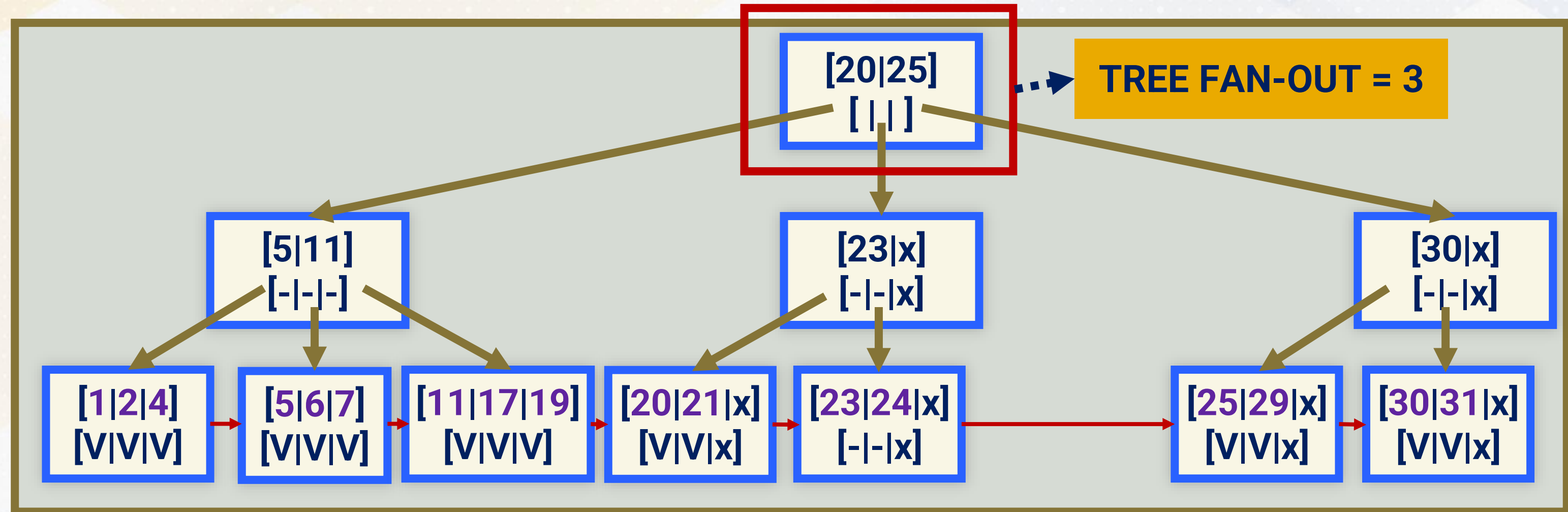
B+Tree Data Structure



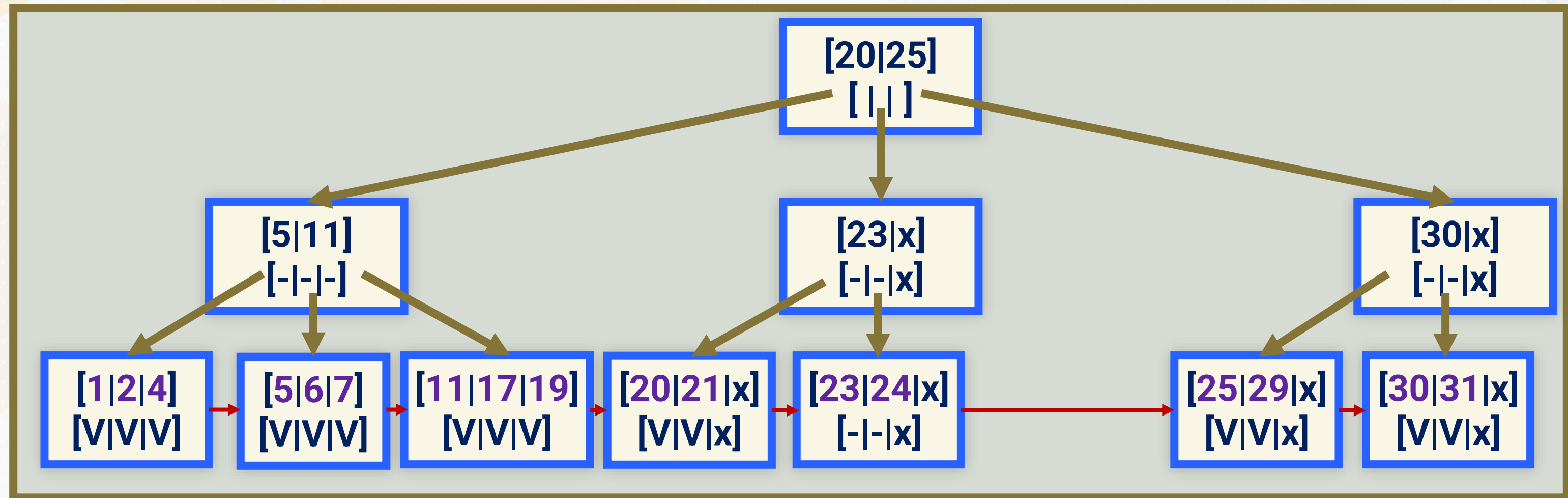
Height of the Tree



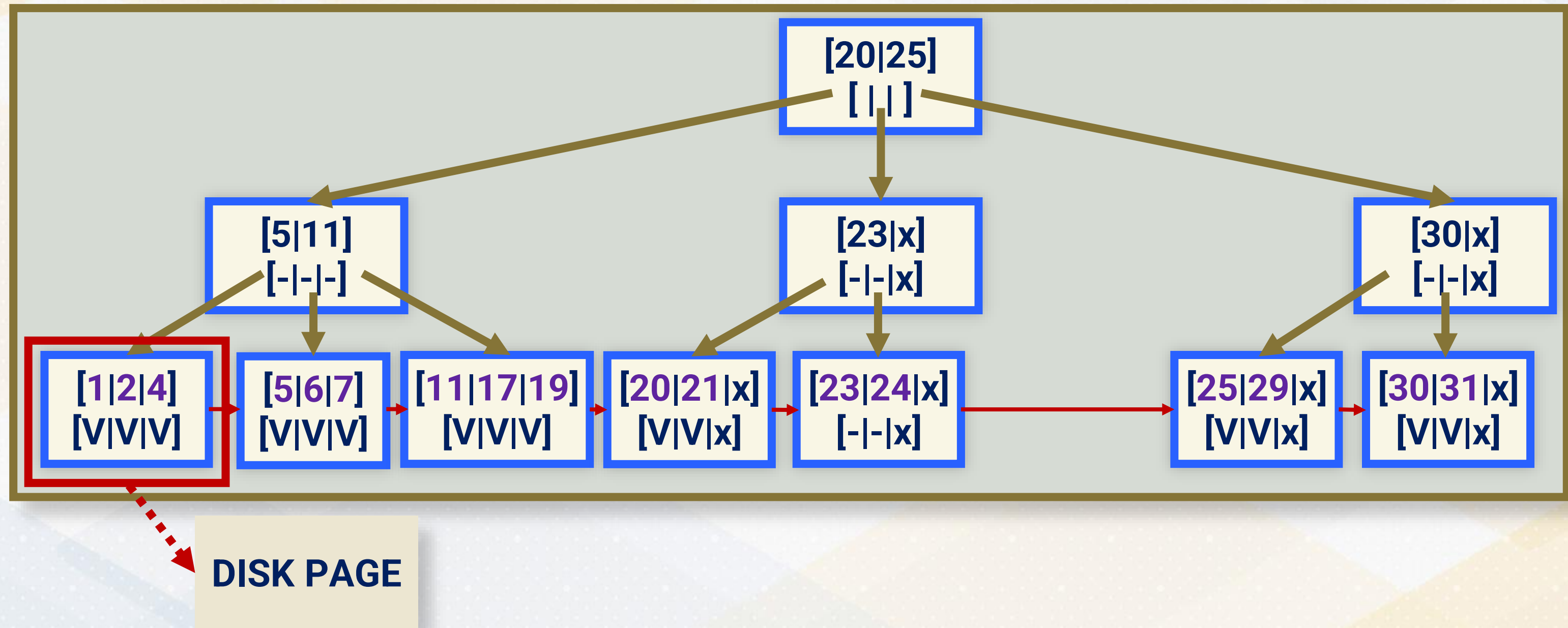
Fan-Out of the Tree



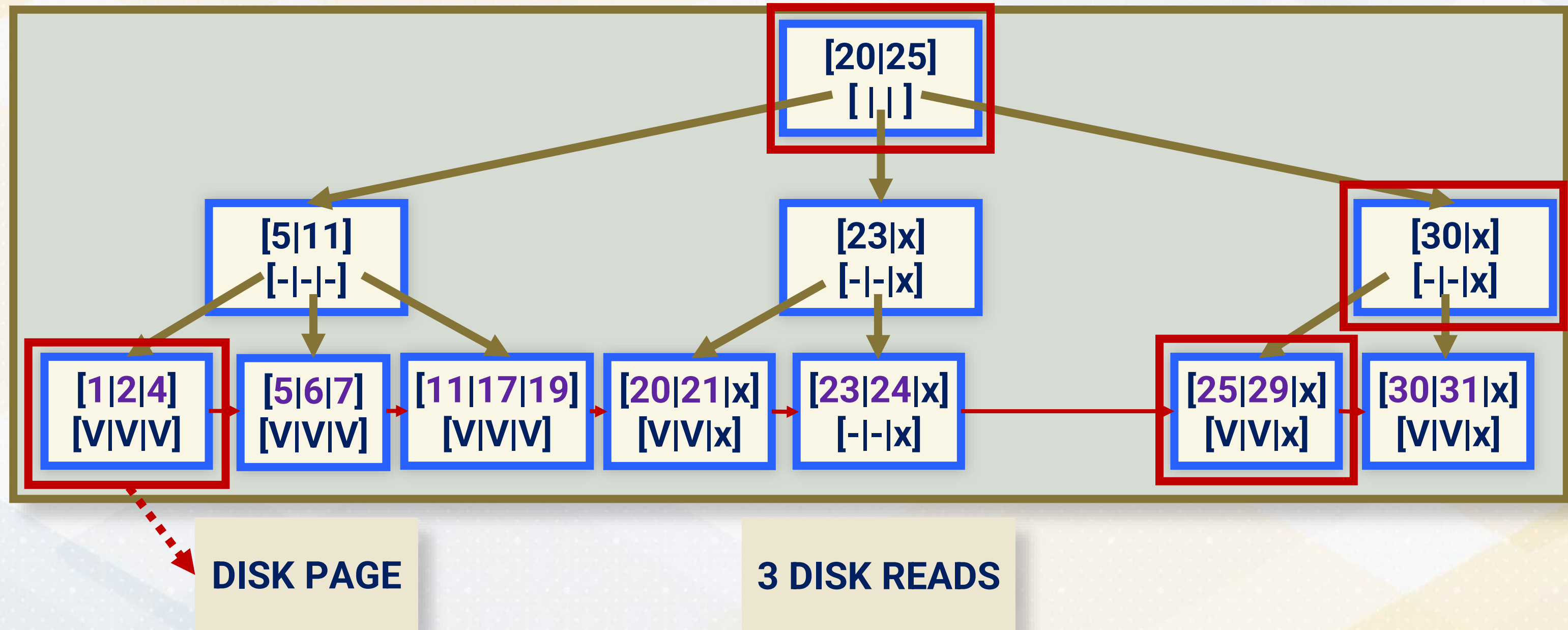
Disk Accesses



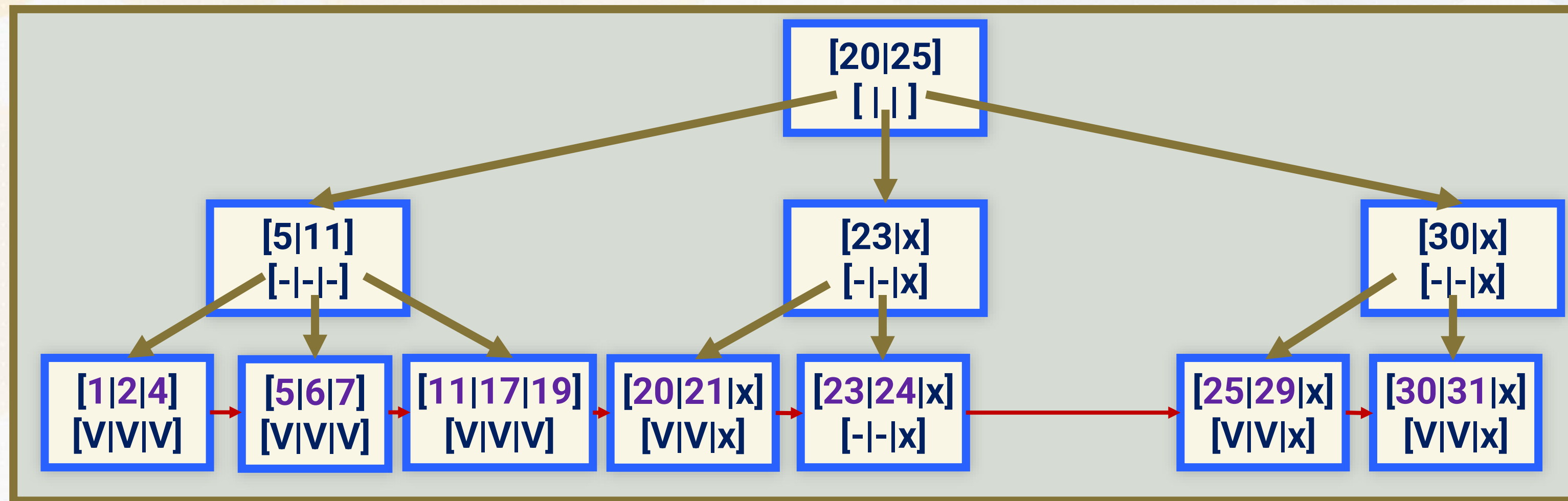
Disk Accesses



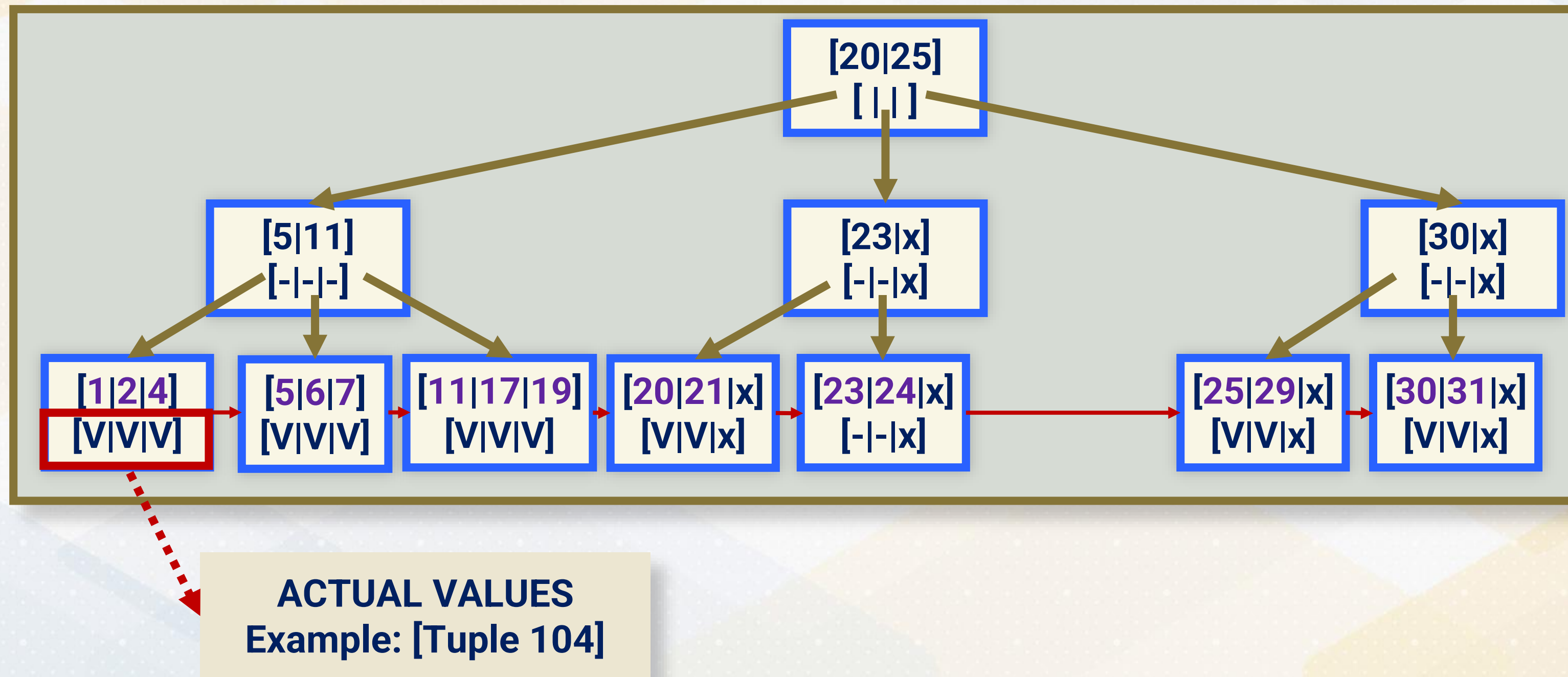
Disk Accesses



Values in Leaf and Inner Nodes

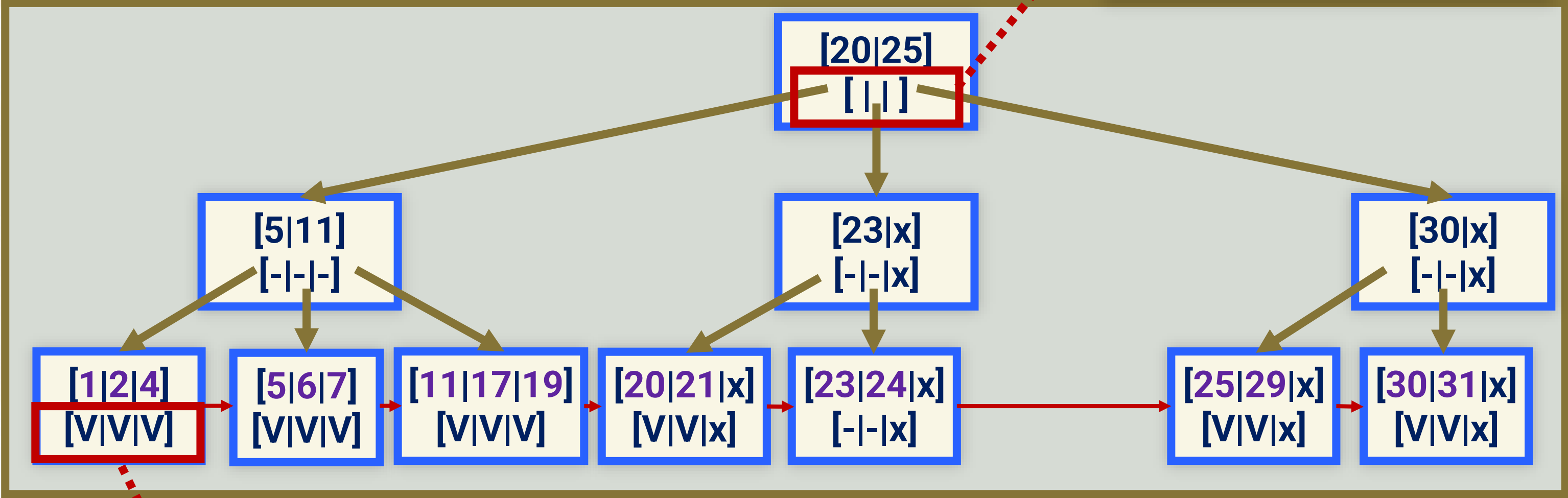


Values in Leaf and Inner Nodes



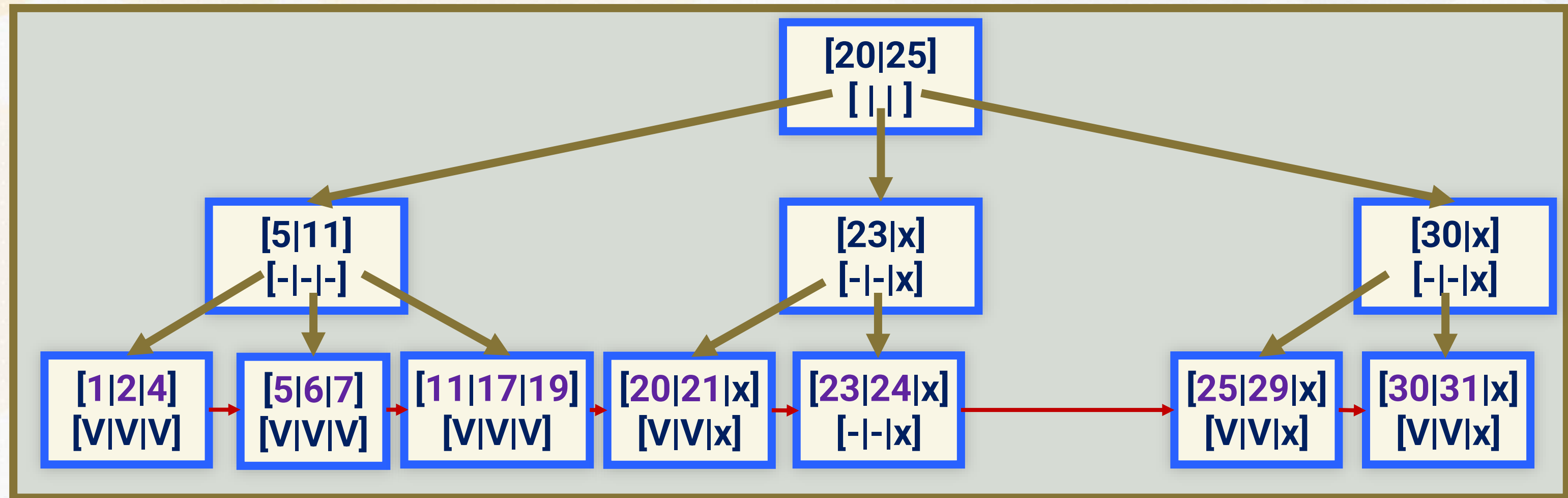
Values in Leaf and Inner Nodes

NODE POINTERS
Example: [Page 10]



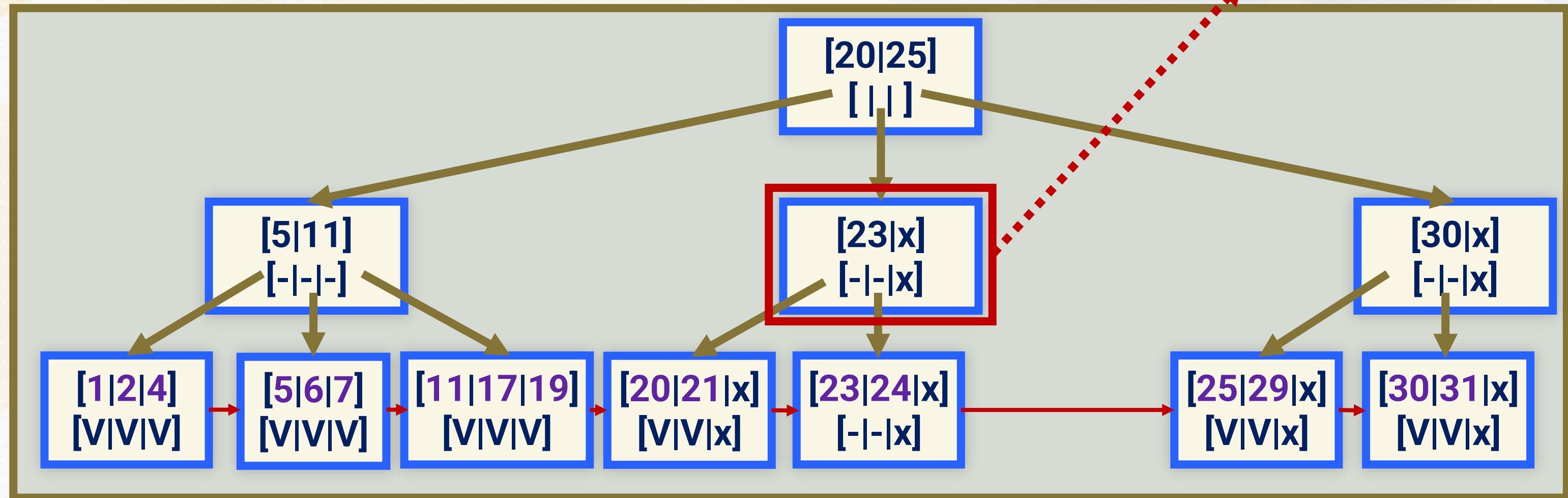
ACTUAL VALUES
Example: [Tuple 104]

Keys in Leaf and Inner Nodes

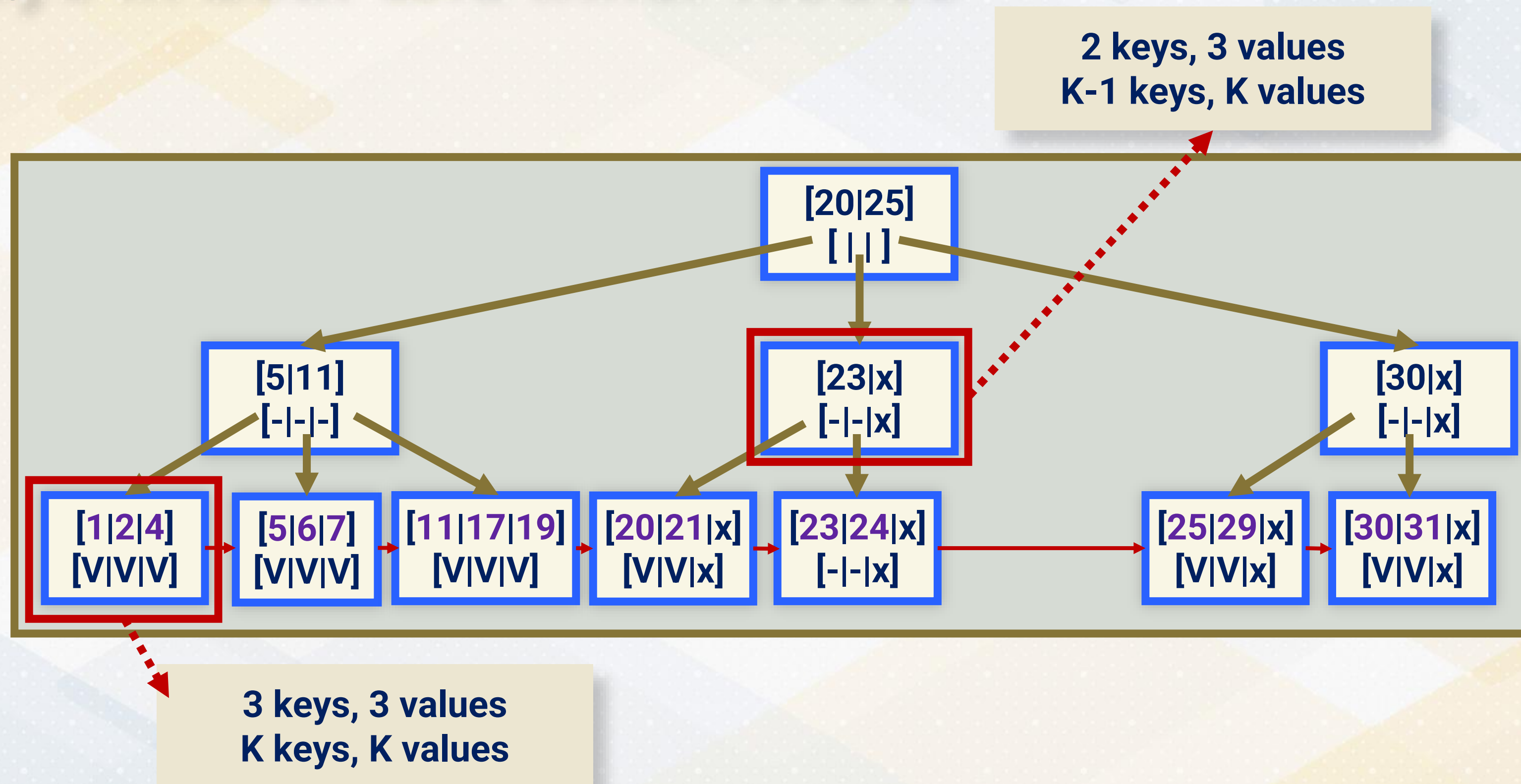


Keys in Leaf and Inner Nodes

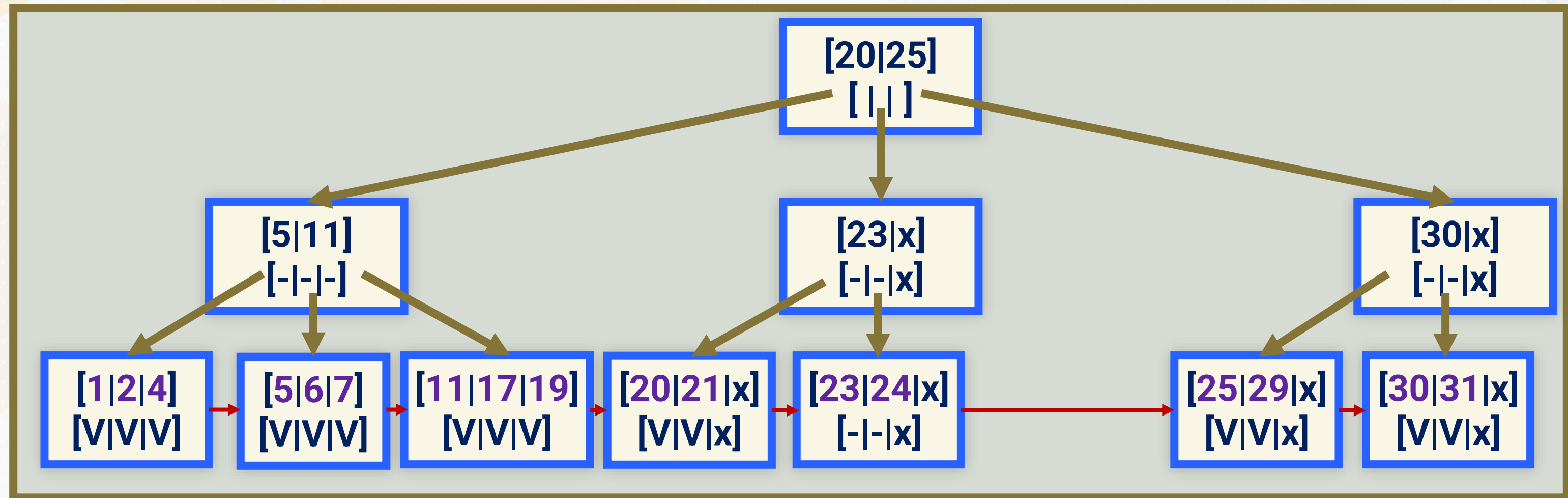
2 keys, 3 values
K-1 keys, K values



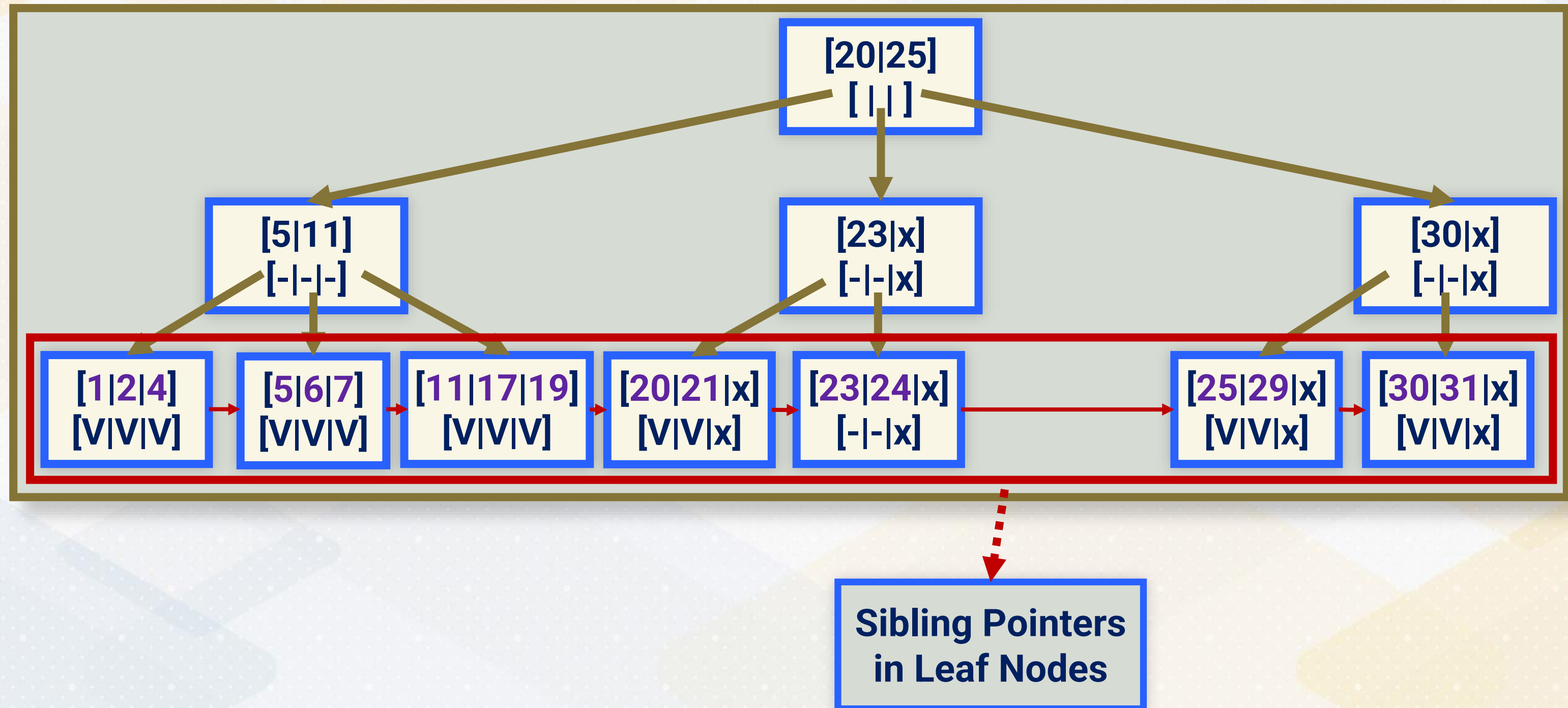
Keys in Leaf and Inner Nodes



Leaf Sibling Pointers for Range Queries



Leaf Sibling Pointers for Range Queries

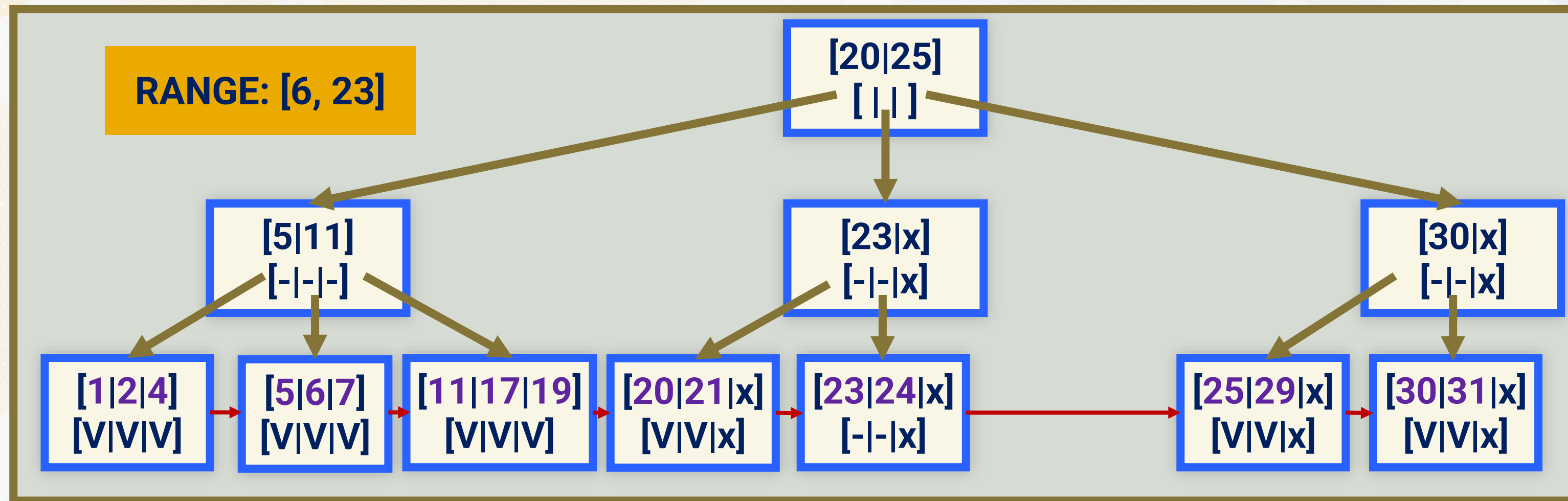


Fast Range Queries Using B+ Trees

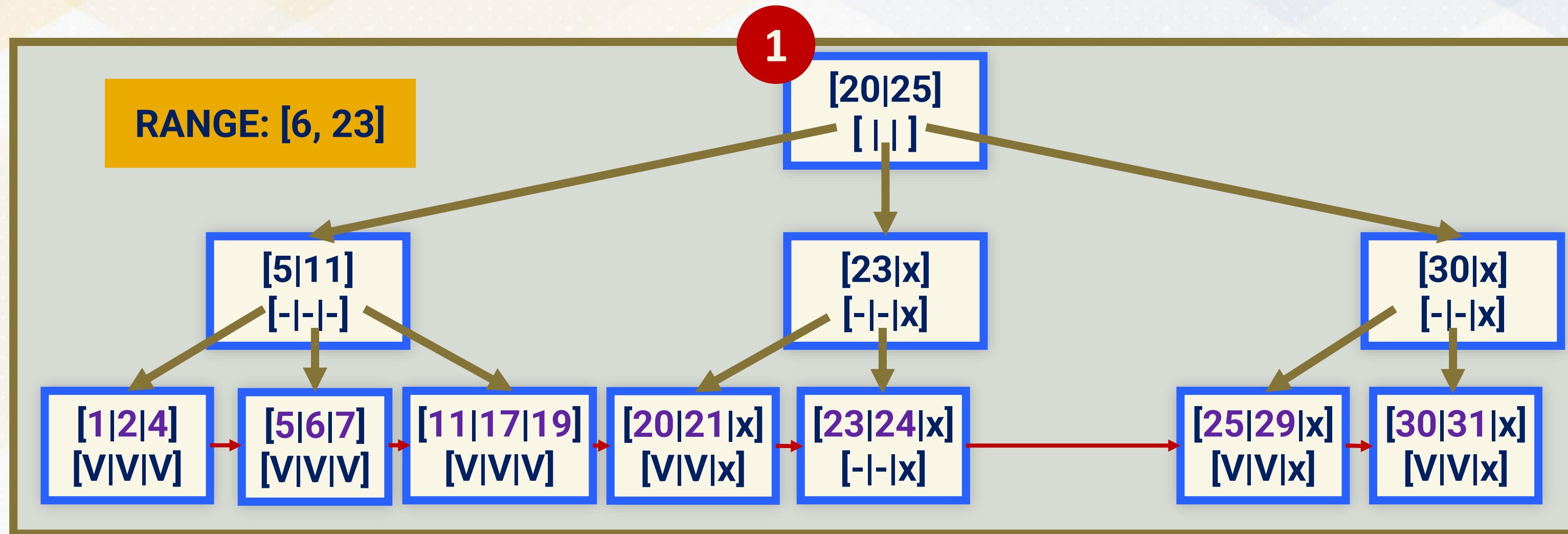
The sorted nature and linked leaf nodes allow B+ trees to execute range queries by simply traversing the leaf nodes that fall within the range

```
std::vector<Value> rangeQuery(const Key &lowerBound, const Key &upperBound) const {  
    // Traverse to the correct leaf node and then follow the linked leaf nodes.  
    // While traversing the leaf nodes, collect all the key-value pairs in the range  
}
```

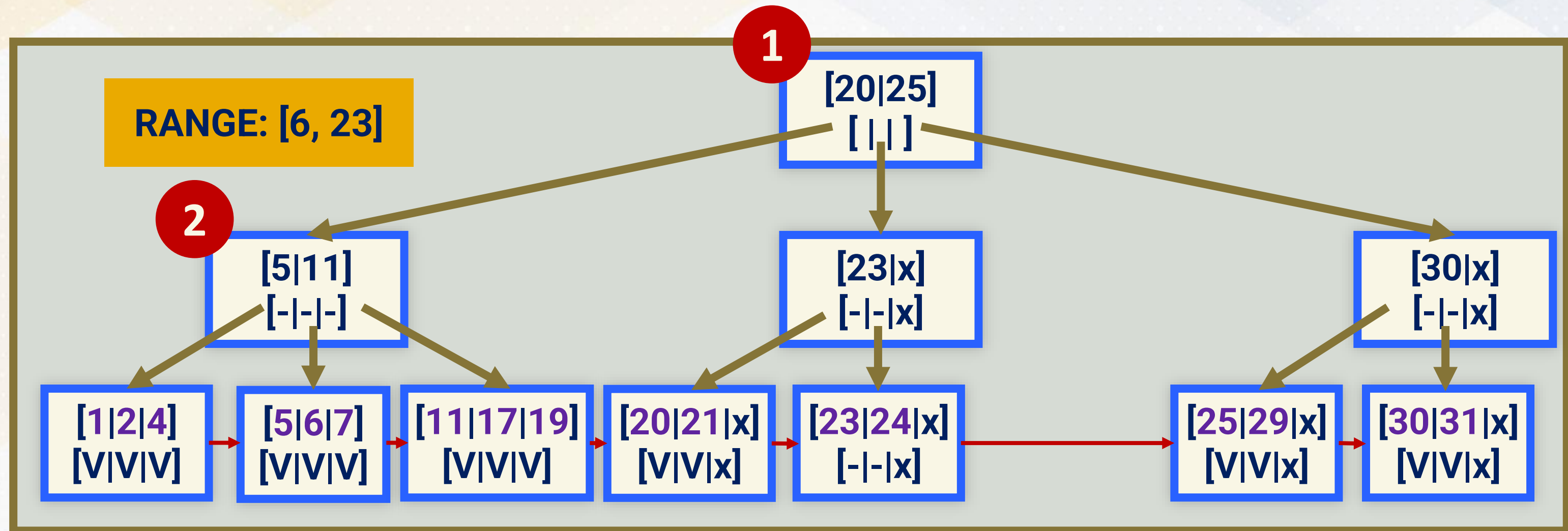
Fast Range Queries Using B+ Trees



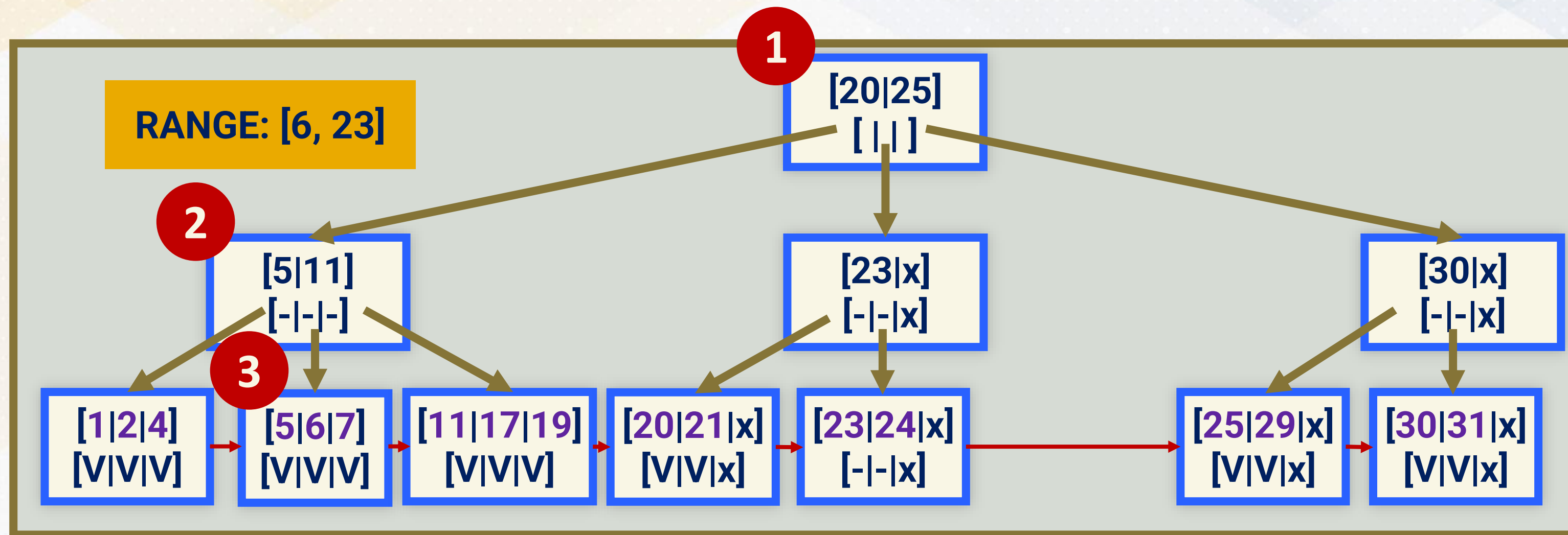
Fast Range Queries Using B+ Trees



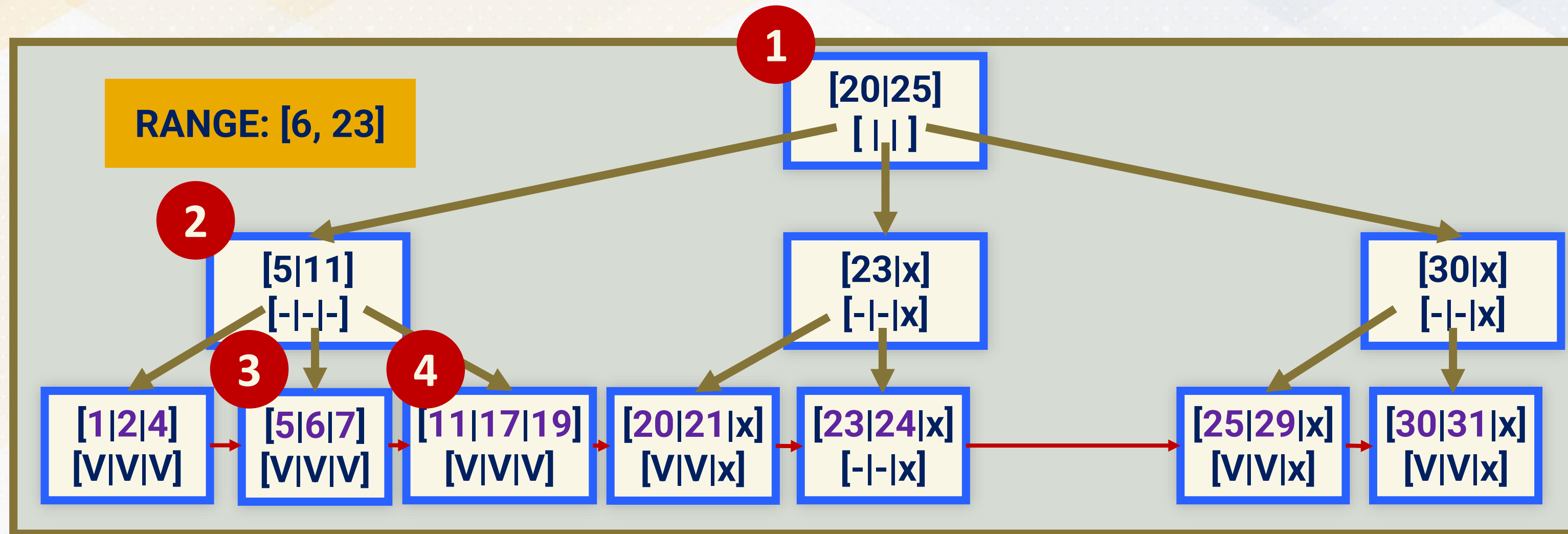
Fast Range Queries Using B+ Trees



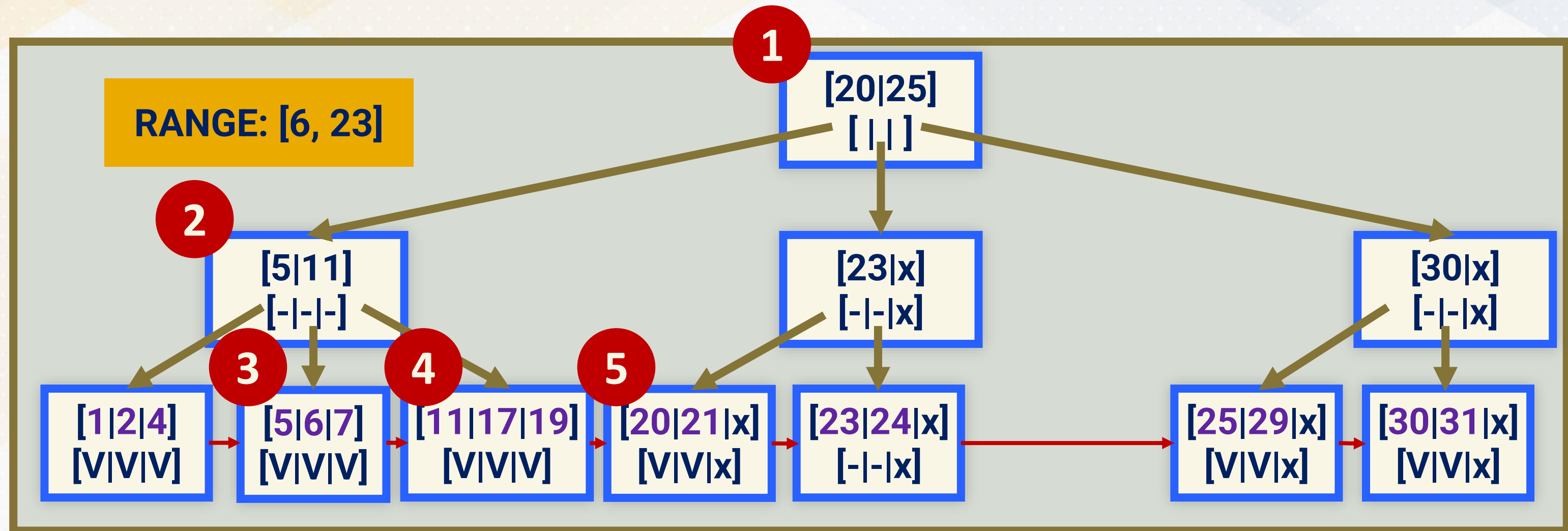
Fast Range Queries Using B+ Trees



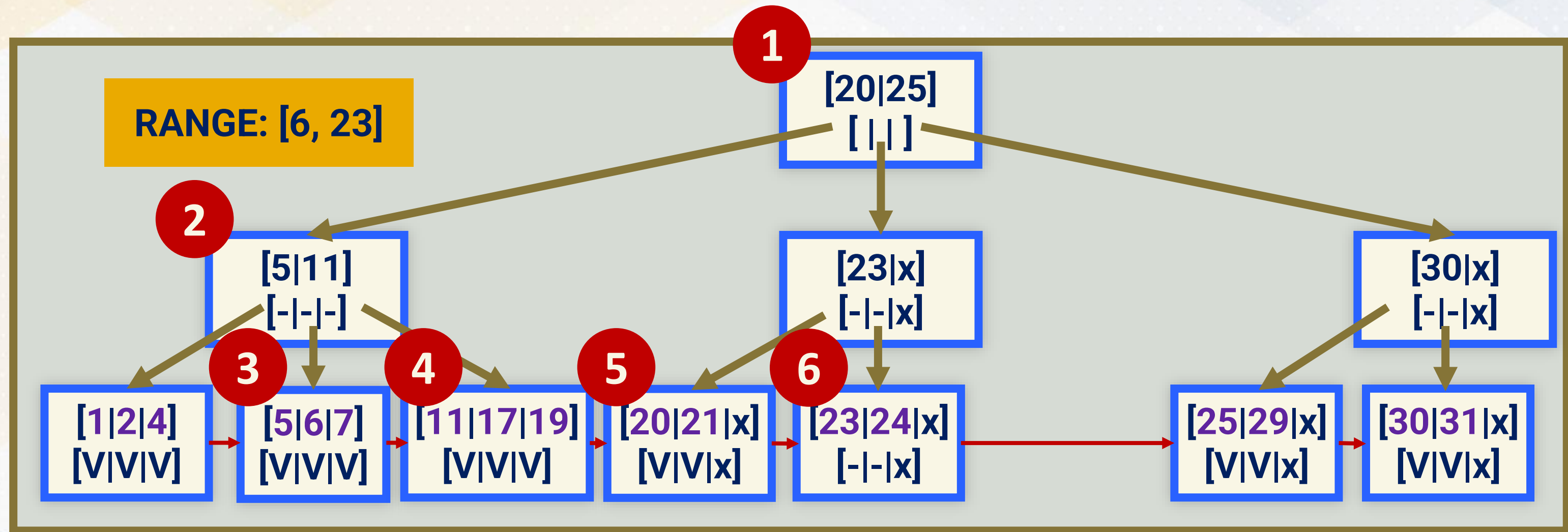
Fast Range Queries Using B+ Trees



Fast Range Queries Using B+ Trees



Fast Range Queries Using B+ Trees







Ordered Index



**Ordered
Index**

**B+Tree
Data
Structure**

B+Tree Template





Implementing B+Tree in C++



**Implementing
B+Tree in C++**

**Templates
in C++**

Templates in C++

```
template <typename T> T add(T a, T b) {  
    return a + b;  
}
```

Templates in C++

Templates

**Enable Generic
Programming**

**Functions & Classes
Operate with
Generic Types**

```
template <typename T> T add(T a, T b) {  
    return a + b;  
}
```

Type Safety Ensured at Compile Time

Template Instantiation

Add Function Instantiated with Different Types

```
int result1 = add<int>(5, 3);
```

```
// returns 8
```

```
std::string result3 = add<std::string>("Hello, ", "world!");
```

```
// returns "Hello, world!"
```

B+Tree Template in C++

```
template <typename Key, typename Value> class BPlusTree {  
    struct Node; // Node definition  
    using NodePtr = std::unique_ptr<Node>;  
    // Node and Entry structures omitted for brevity  
};
```


B+Tree Template in C++

B+ Trees
Use C++
Templates

```
template <typename Key, typename Value> class BPlusTree {  
    struct Node; // Node definition  
    using NodePtr = std::unique_ptr<Node>;  
    // Node and Entry structures omitted for brevity  
};
```


B+Tree Template in C++

B+ Trees
Use C++
Templates

Support Key &
Value Types

```
template <typename Key, typename Value> class BPlusTree {  
    struct Node; // Node definition  
    using NodePtr = std::unique_ptr<Node>;  
    // Node and Entry structures omitted for brevity  
};
```

B+Tree Template in C++

B+ Trees
Use C++
Templates

Support Key &
Value Types

Placeholders
Replaced at
Compile Time

```
template <typename Key, typename Value> class BPlusTree {  
    struct Node; // Node definition  
    using NodePtr = std::unique_ptr<Node>;  
    // Node and Entry structures omitted for brevity  
};
```


B+Tree Template in C++

B+ Trees
Use C++
Templates

Support Key &
Value Types

Placeholders
Replaced at
Compile Time

B+ Tree
Handles Any
Key and Value
Types

```
template <typename Key, typename Value> class BPlusTree {  
    struct Node; // Node definition  
    using NodePtr = std::unique_ptr<Node>;  
    // Node and Entry structures omitted for brevity  
};
```


B+Tree Template in C++

B+ Trees
Use C++
Templates

Support Key &
Value Types

Placeholders
Replaced at
Compile Time

B+ Tree
Handles Any
Key and Value
Types

Enhance Code
Reusability &
Maintainability

```
template <typename Key, typename Value> class BPlusTree {  
    struct Node; // Node definition  
    using NodePtr = std::unique_ptr<Node>;  
    // Node and Entry structures omitted for brevity  
};
```

B+Tree Template Instantiation #1

```
BPlusTree<int, int> idToSalary;
```

```
// Employee ID and salary are inserted into the tree.
```

```
idToSalary.insertOrUpdate(1001, 50000);
```

```
// Retrieve the salary of an employee by their ID quickly.
```

```
int salary = idToSalary.getValue(1001);
```


B+Tree Template Instantiation #1

Example: Employee ID (int) → Salary (int)

```
BPlusTree<int, int> idToSalary;  
  
// Employee ID and salary are inserted into the tree.  
idToSalary.insertOrUpdate(1001, 50000);  
  
// Retrieve the salary of an employee by their ID quickly.  
int salary = idToSalary.getValue(1001);
```


B+Tree Template Instantiation #2

```
BPlusTree<std::string, int> productToPrice;  
  
// Prices of products are stored in the B+ tree  
productToPrice.insertOrUpdate("Apple iPhone 12", 999);  
  
// Quickly find the price of a specific product.  
int price = productToPrice.getValue("Apple iPhone 12");
```

B+Tree Template Instantiation #2

Example: Product Name (string) → Price (int)

```
BPlusTree<std::string, int> productToPrice;  
  
// Prices of products are stored in the B+ tree  
productToPrice.insertOrUpdate("Apple iPhone 12", 999);  
  
// Quickly find the price of a specific product.  
int price = productToPrice.getValue("Apple iPhone 12");
```

B+Tree Template Instantiation #3

```
struct TicketKey {  
    std::string eventDate;  
    int seatNumber;  
    // Overload the comparison operators for ordering in the B+ tree  
    bool operator<(const TicketKey &other) const {  
        if (eventDate == other.eventDate) {  
            return seatNumber < other.seatNumber;  
        }  
        return eventDate < other.eventDate;  
    }  
};
```


B+Tree Template Instantiation #3

Example: TicketKey (user-defined type) $\rightarrow$ Name (int)

```
struct TicketKey {  
    std::string eventDate;  
    int seatNumber;  
    // Overload the comparison operators for ordering in the B+ tree  
    bool operator<(const TicketKey &other) const {  
        if (eventDate == other.eventDate) {  
            return seatNumber < other.seatNumber;  
        }  
        return eventDate < other.eventDate;  
    }  
};
```

B+Tree Template Instantiation #3

Example: TicketKey (user-defined type) → Name (int)

```
BPlusTree<TicketKey, std::string> concertTickets;  
// When a ticket is booked, it is entered into B+ tree using its composite key.  
concertTickets.insertOrUpdate(TicketKey("2030-12-25", 101), "John Doe");  
concertTickets.insertOrUpdate(TicketKey("2030-12-25", 102), "Jane Smith");  
  
// Retrieve the booking information for a specific seat on a particular date.  
std::string ticketHolder =  
    concertTickets.getValue(TicketKey("2030-12-25", 101));  
std::cout << "Seat 101 is booked by: " << ticketHolder << std::endl;
```

Ordered Index Class in BuzzDB

We implement OrderedIndex class by encapsulating a BPlusTree

```
class OrderedIndex {  
    BPlusTree<int, int> bptree;  
  
public:  
    void insertOrUpdate(int key, int value) {  
        bptree.insertOrUpdate(key, value);  
    }  
  
    std::vector<int> rangeQuery(int lowerBound, int upperBound) const {  
        return bptree.rangeQuery(lowerBound, upperBound);  
    }  
};
```




Implementing B+Tree in C++



**Implementing
B+Tree in C++**

**Templates
in C++**

B+Tree Structure





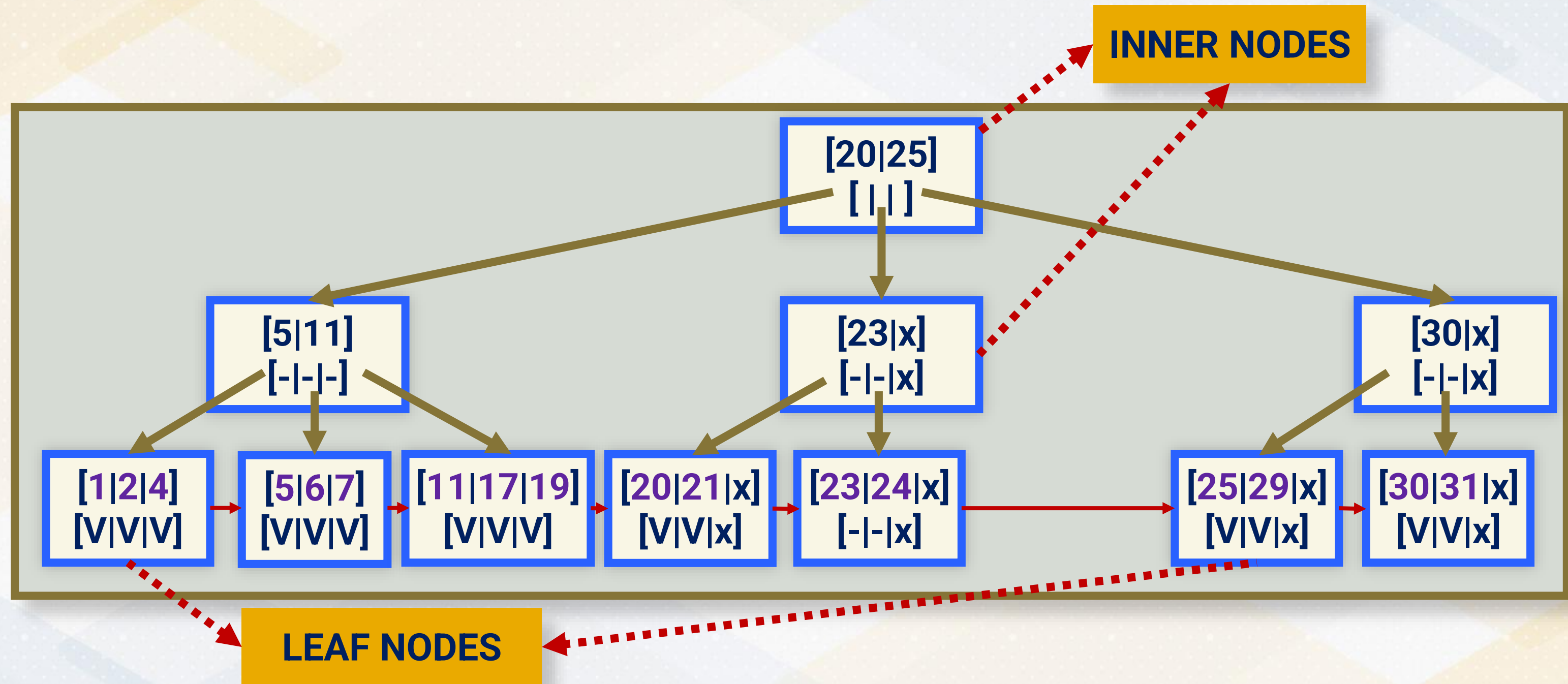
Node Structure



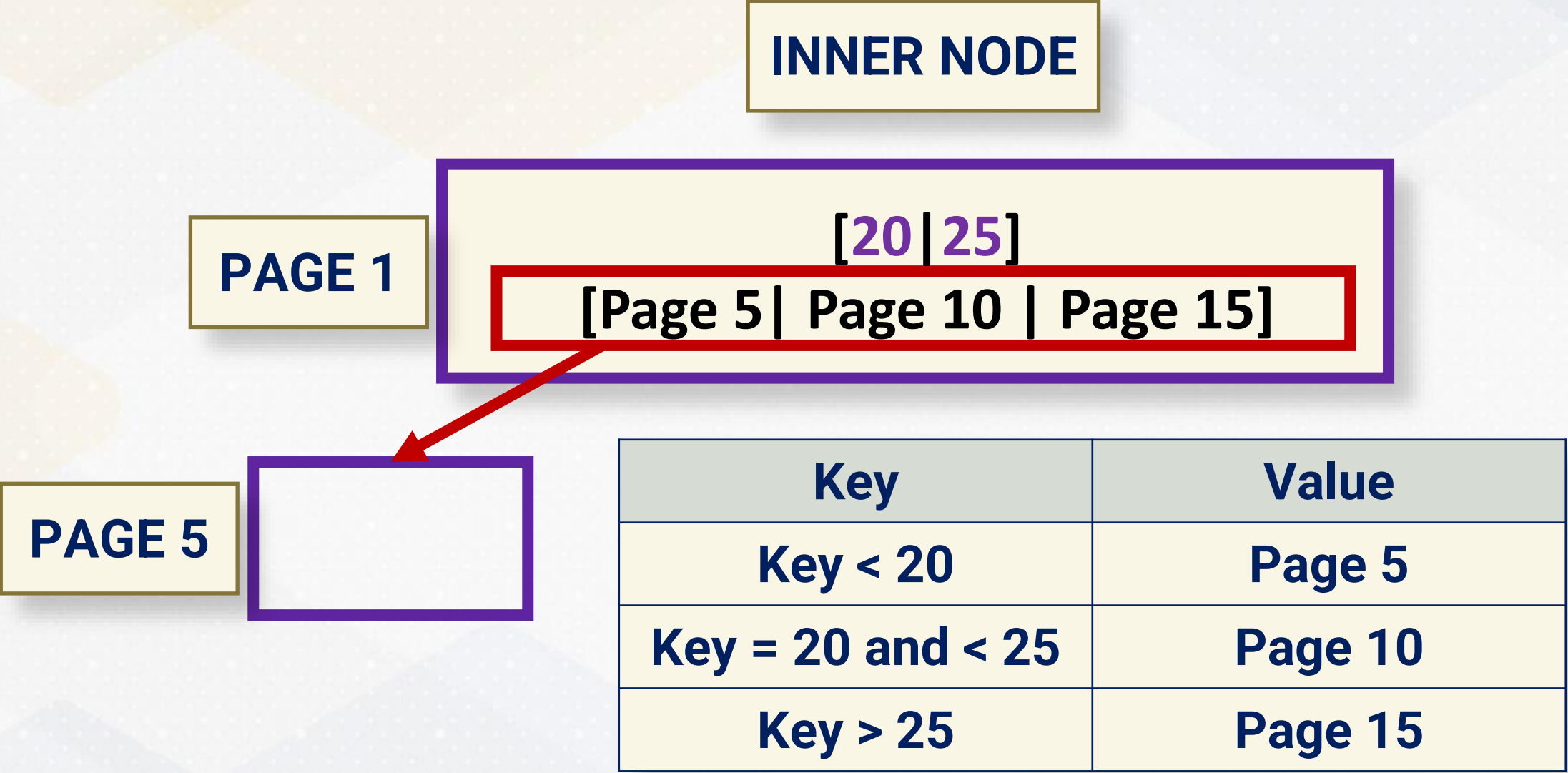
**Node
Structure**

Insertion

B+Tree Structure



Inner Node



Leaf Node

LEAF NODE

PAGE 8

[1 | 2 | 4]

[Tuple 310| Tuple 102 | Tuple 115]

Key	Value
Key = 1	Tuple 310
Key = 2	Tuple 102
Key = 4	Tuple 115

Key and Value Types

Entry	Key	Value
Entry 1	"max_file_size"	"10 MB"
Entry 2	"default_language"	"English"
Entry 3	"session_timeout"	"30 minutes"

LEAF NODE

PAGE 8

["max_file_size" | "default_language" | "session_timeout"]
["10 MB" | "English" | "30 minutes"]

Key and Value Types

ENTRIES

Entry	Key	Value
Entry 1	"max_file_size"	"10 MB"
Entry 2	"default_language"	"English"
Entry 3	"session_timeout"	"30 minutes"

LEAF NODE

PAGE 8

["max_file_size" | "default_language" | "session_timeout"]
["10 MB" | "English" | "30 minutes"]

Entry in C++

```
struct Entry {  
    Key key;  
    Value value; // Used only in leaf node entry  
    NodePtr next; // Used only in inner node entry  
  
    Entry(Key k, Value v) : key(k), value(v), next(nullptr) {}  
};
```

B+Tree Node in C++



```
struct Node {  
    bool isLeaf;  
    std::vector<Entry> entries;  
    std::unique_ptr<Node> next;  
  
    Node(bool leaf) : isLeaf(leaf), next(nullptr) {}  
};
```

B+Tree Node in C++



isLeaf?

```
struct Node {  
    bool isLeaf;  
    std::vector<Entry> entries;  
    std::unique_ptr<Node> next;  
  
    Node(bool leaf) : isLeaf(leaf), next(nullptr) {}  
};
```


B+Tree Node in C++



isLeaf?

Vector of Entries

```
struct Node {  
    bool isLeaf;  
    std::vector<Entry> entries;  
    std::unique_ptr<Node> next;  
  
    Node(bool leaf) : isLeaf(leaf), next(nullptr) {}  
};
```

B+Tree Node in C++



isLeaf?

Vector of Entries

Entry = {Key-Value}

```
struct Node {  
    bool isLeaf;  
    std::vector<Entry> entries;  
    std::unique_ptr<Node> next;  
  
    Node(bool leaf) : isLeaf(leaf), next(nullptr) {}  
};
```

B+Tree Node in C++



isLeaf?

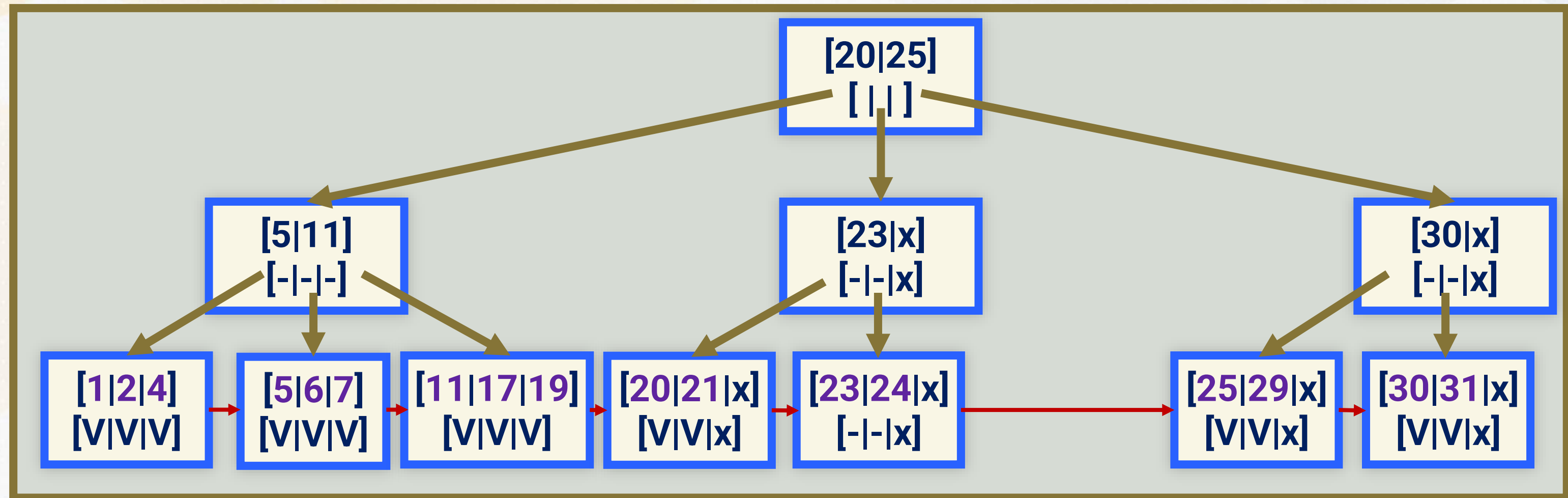
Vector of Entries

Entry = {Key-Value}

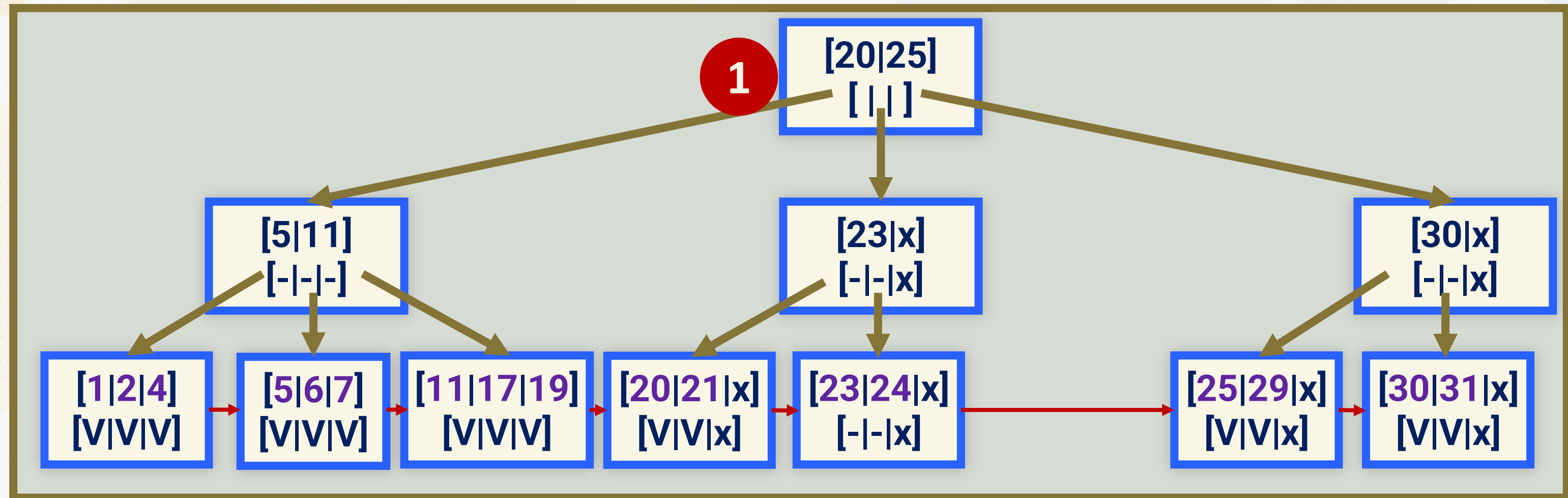
Next Pointer

```
struct Node {  
    bool isLeaf;  
    std::vector<Entry> entries;  
    std::unique_ptr<Node> next;  
  
    Node(bool leaf) : isLeaf(leaf), next(nullptr) {}  
};
```

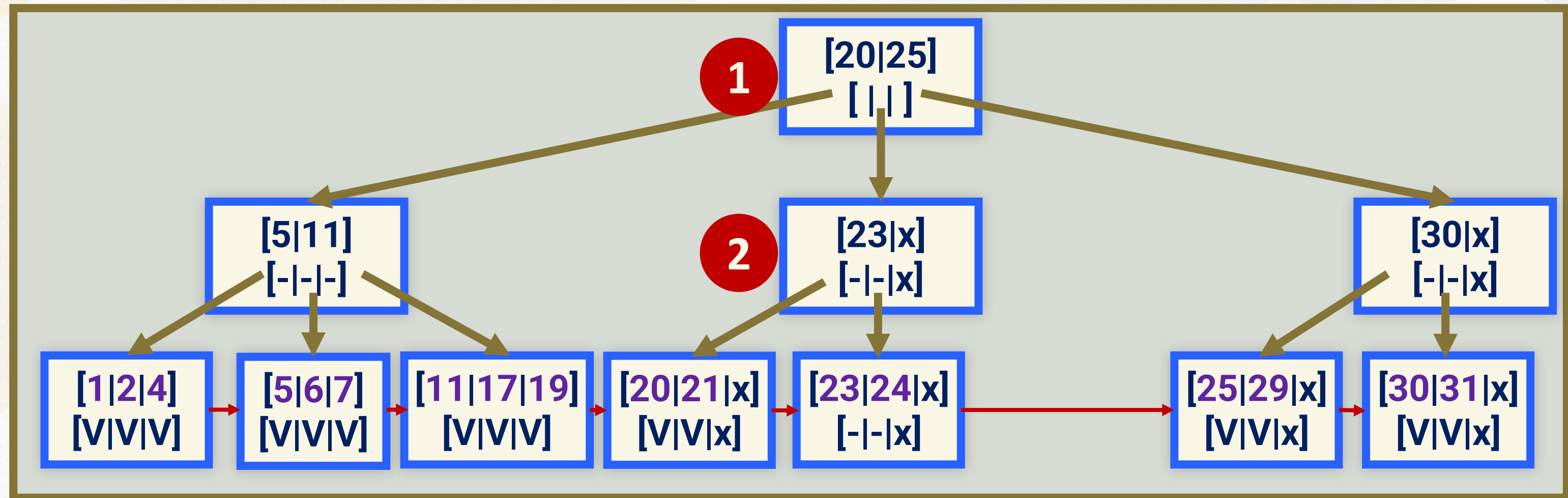

Inserting a Key-Value Pair



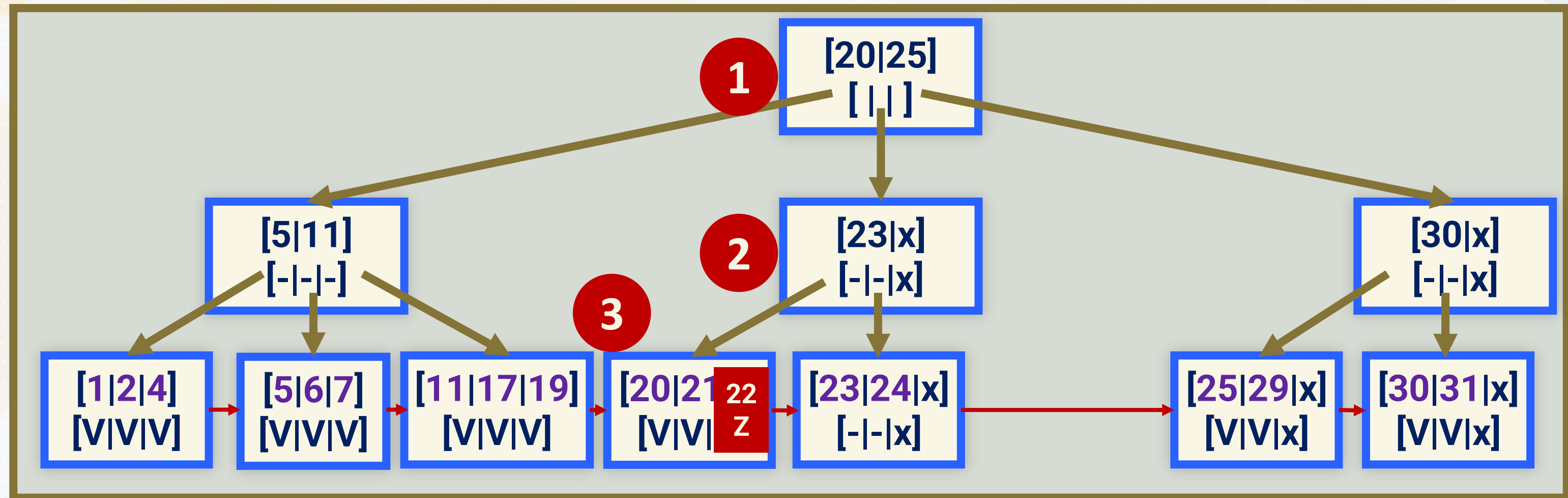
Inserting a Key-Value Pair



Inserting a Key-Value Pair



Inserting a Key-Value Pair



Inserting a Key-Value Pair

Insertion process is recursive

```
void insertOrUpdate(const Key &key, const Value &value) {  
    insertOrUpdateInternal(key, value, root);  
}
```

Recursive Descent in an Inner Node

Find the appropriate child node by comparing keys.

```
void insertOrUpdateInternal(const Key &key, const Value &value, NodePtr &node) {  
    ...  
    else {  
        bool found = false;  
        for (auto it = node->entries.begin(); it != node->entries.end(); ++it) {  
            if (key < it->key) {  
                insertOrUpdateInternal(key, value, it->next);  
                found = true;  
                break;  
            }  
        }  
    }  
}
```


Insertion in a Leaf Node

If key does not exist in leaf node, insert it at the correct position to maintain sorted order.

```
void insertOrUpdateInternal(const Key &key, const Value &value, NodePtr &node) {  
    if (node->isLeaf) {  
        // Search for the key in the leaf node  
        auto it = std::find_if(node->entries.begin(), node->entries.end(),  
                               [&](const Entry &entry) { return entry.key == key; });  
        // Key not found, proceed to insert in sorted order  
        auto comp = [](const Entry &entry, const Key &k) { return entry.key < k; };  
        it = std::lower_bound(node->entries.begin(), node->entries.end(), key, comp);  
        node->entries.insert(it, Entry(key, value));  
    }  
    ...  
}
```

Binary Search using a Lambda Function

λ Function

```
// Key not found, proceed to insert in sorted order
auto comp = [](const Entry &entry, const Key &k)
    { return entry.key < k; };
it = std::lower_bound(node->entries.begin(),
    node->entries.end(),
    key, comp);
node->entries.insert(it, Entry(key, value));
```

Binary Search using a Lambda Function

 **Function**

Anonymous

```
// Key not found, proceed to insert in sorted order
auto comp = [](const Entry &entry, const Key &k)
    { return entry.key < k; };
it = std::lower_bound(node->entries.begin(),
    node->entries.end(),
    key, comp);
node->entries.insert(it, Entry(key, value));
```


Binary Search using a Lambda Function

 **Function**

Anonymous

Short Functions

```
// Key not found, proceed to insert in sorted order
auto comp = [](const Entry &entry, const Key &k)
    { return entry.key < k; };
it = std::lower_bound(node->entries.begin(),
    node->entries.end(),
    key, comp);
node->entries.insert(it, Entry(key, value));
```

Binary Search using a Lambda Function

 **Function**

Anonymous

Short Functions

Comparator

```
// Key not found, proceed to insert in sorted order
auto comp = [](const Entry &entry, const Key &k)
    { return entry.key < k; };
it = std::lower_bound(node->entries.begin(),
    node->entries.end(),
    key, comp);
node->entries.insert(it, Entry(key, value));
```


Binary Search using a Lambda Function

λ Function

```
// Key not found, proceed to insert in sorted order
auto comp = [](const Entry &entry, const Key &k)
    { return entry.key < k; };
it = std::lower_bound(node->entries.begin(),
    node->entries.end(),
    key, comp);
node->entries.insert(it, Entry(key, value));
```


Binary Search using a Lambda Function

λ Function

lower_bound

```
// Key not found, proceed to insert in sorted order
auto comp = [](const Entry &entry, const Key &k)
    { return entry.key < k; };
it = std::lower_bound(node->entries.begin(),
    node->entries.end(),
    key, comp);
node->entries.insert(it, Entry(key, value));
```

Binary Search using a Lambda Function

λ Function

lower_bound

iterator

```
// Key not found, proceed to insert in sorted order
auto comp = [](const Entry &entry, const Key &k)
    { return entry.key < k; };
it = std::lower_bound(node->entries.begin(),
    node->entries.end(),
    key, comp);
node->entries.insert(it, Entry(key, value));
```


Binary Search using a Lambda Function

 **Function**

lower_bound

iterator

Sorted Order

```
// Key not found, proceed to insert in sorted order
auto comp = [](const Entry &entry, const Key &k)
    { return entry.key < k; };
it = std::lower_bound(node->entries.begin(),
    node->entries.end(),
    key, comp);
node->entries.insert(it, Entry(key, value));
```






Node Structure



**Node
Structure**

Insertion



Range Query Processing







Inefficient Range Query Processing

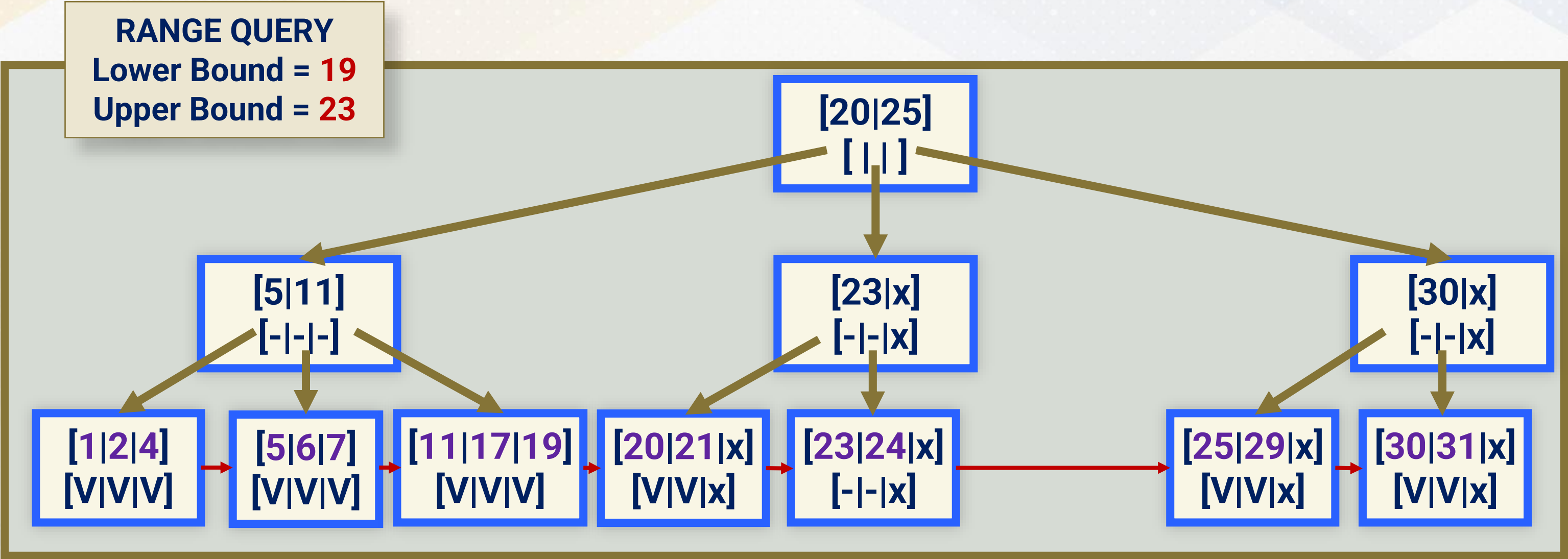


**Inefficient
Range Query
Processing**

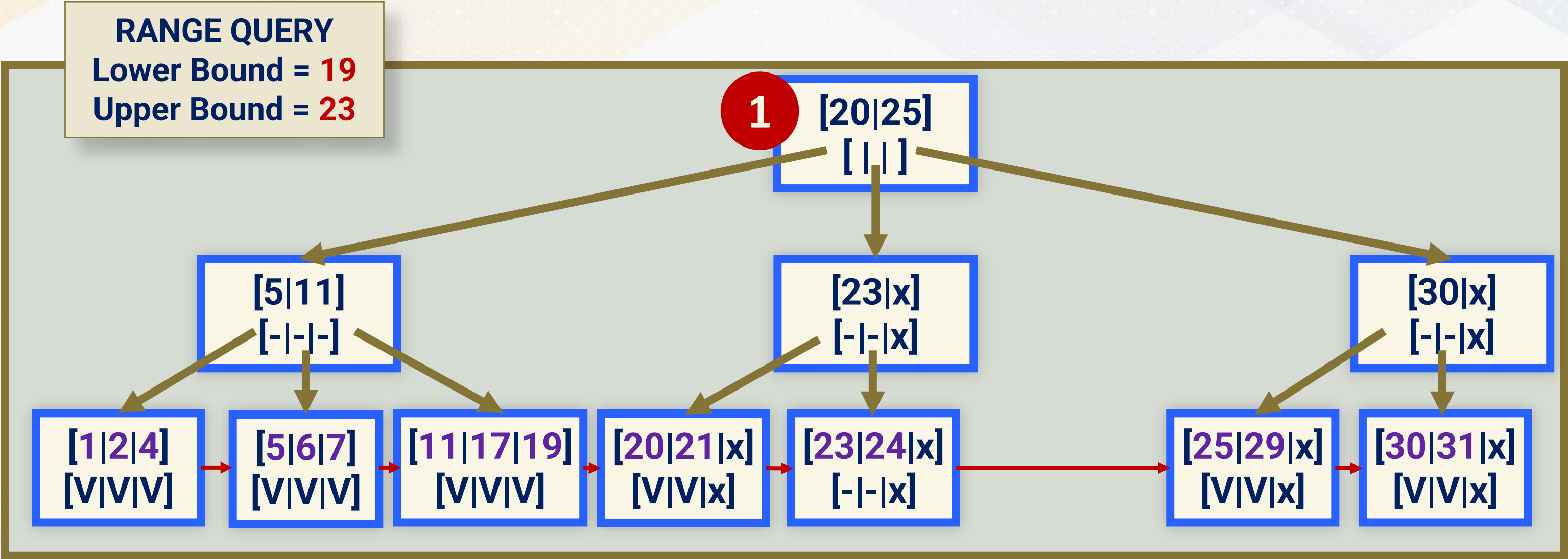


**Efficient
Range Query
Processing**

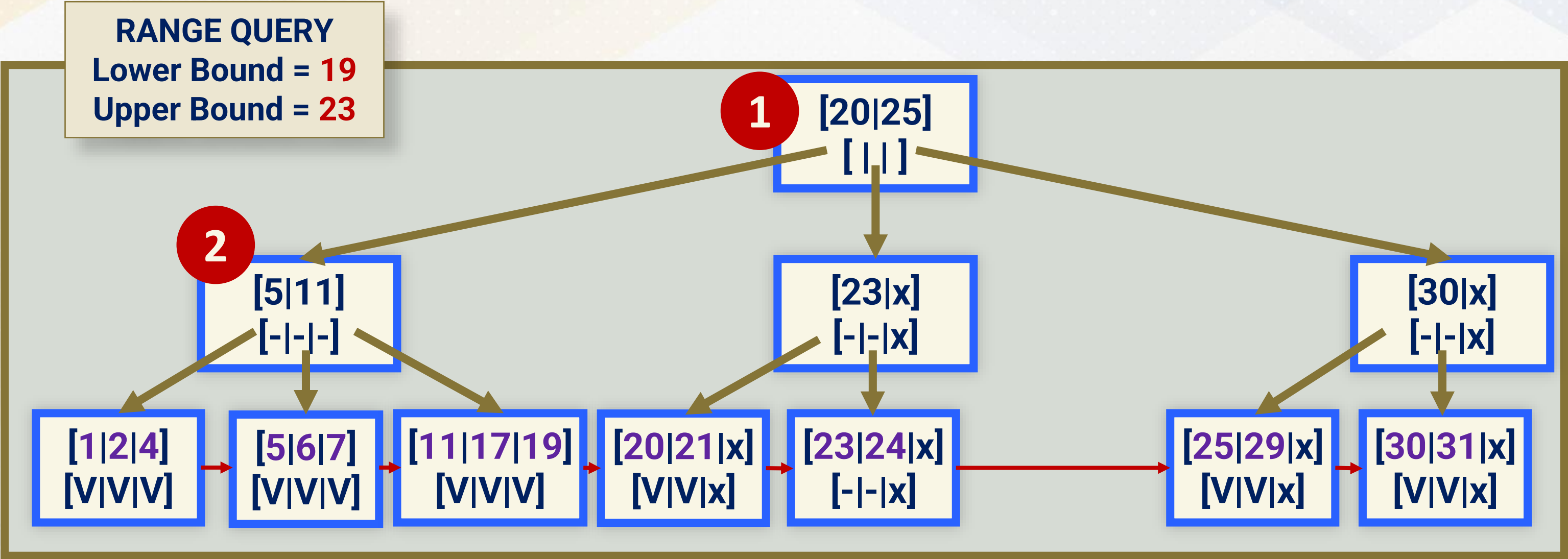
Inefficient Range Query Processing



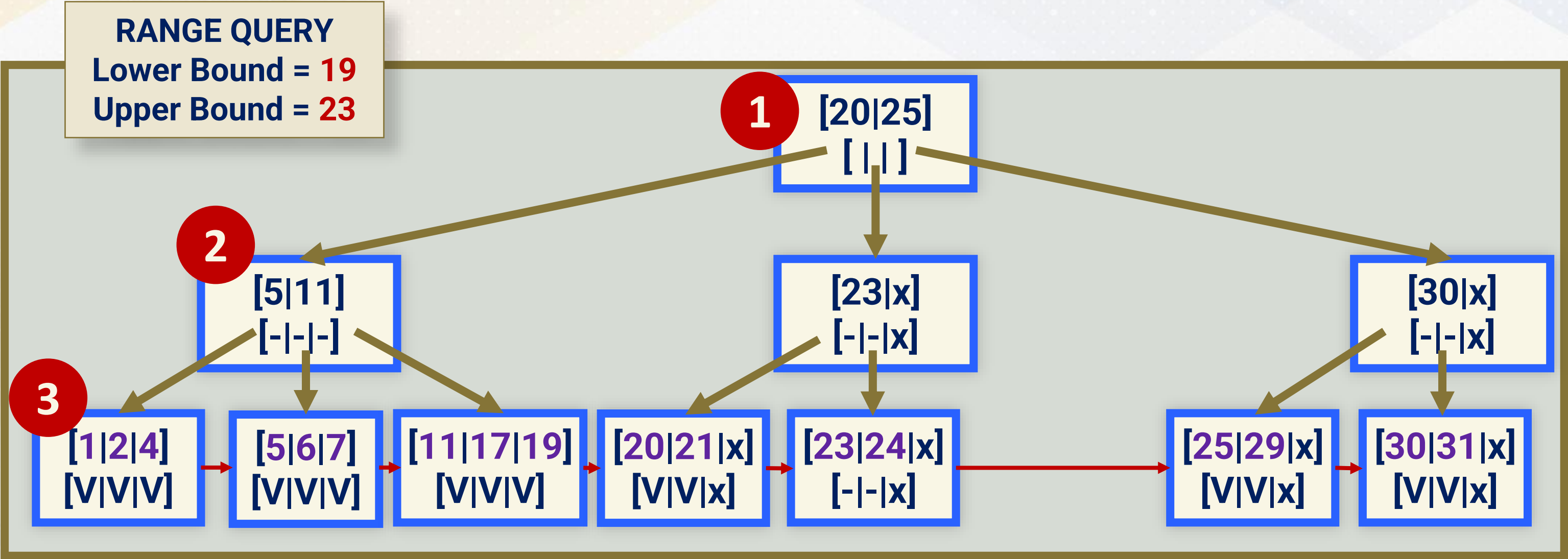
Inefficient Range Query Processing



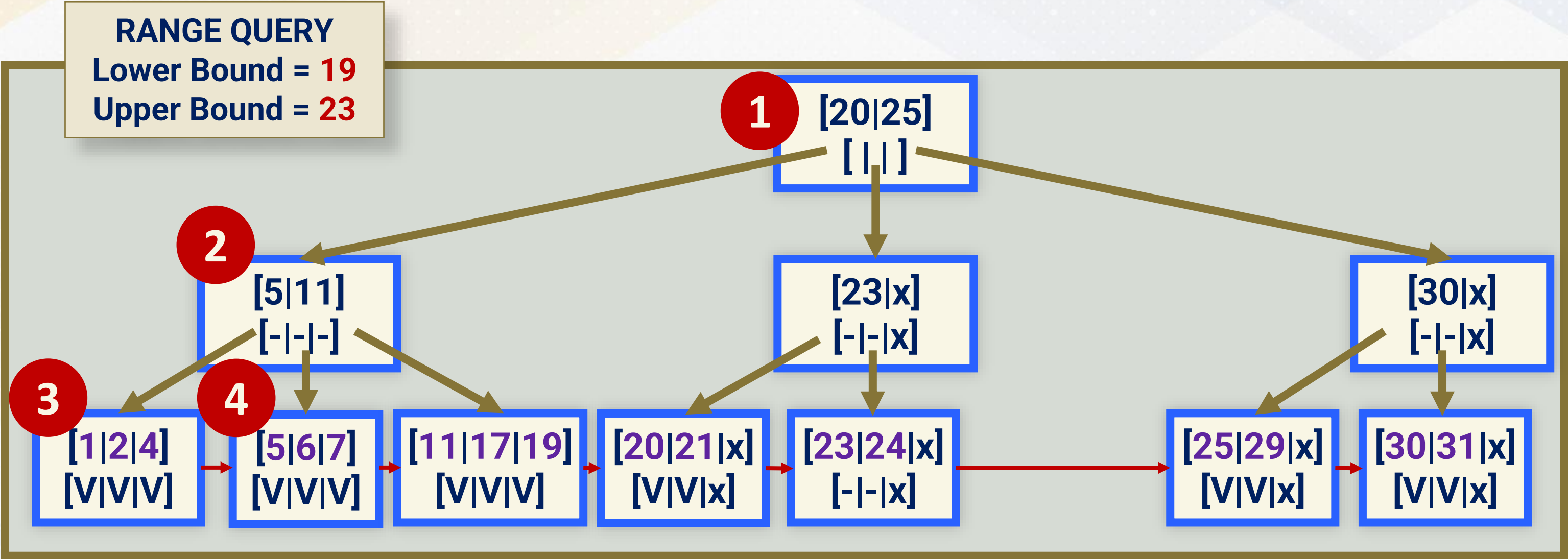
Inefficient Range Query Processing



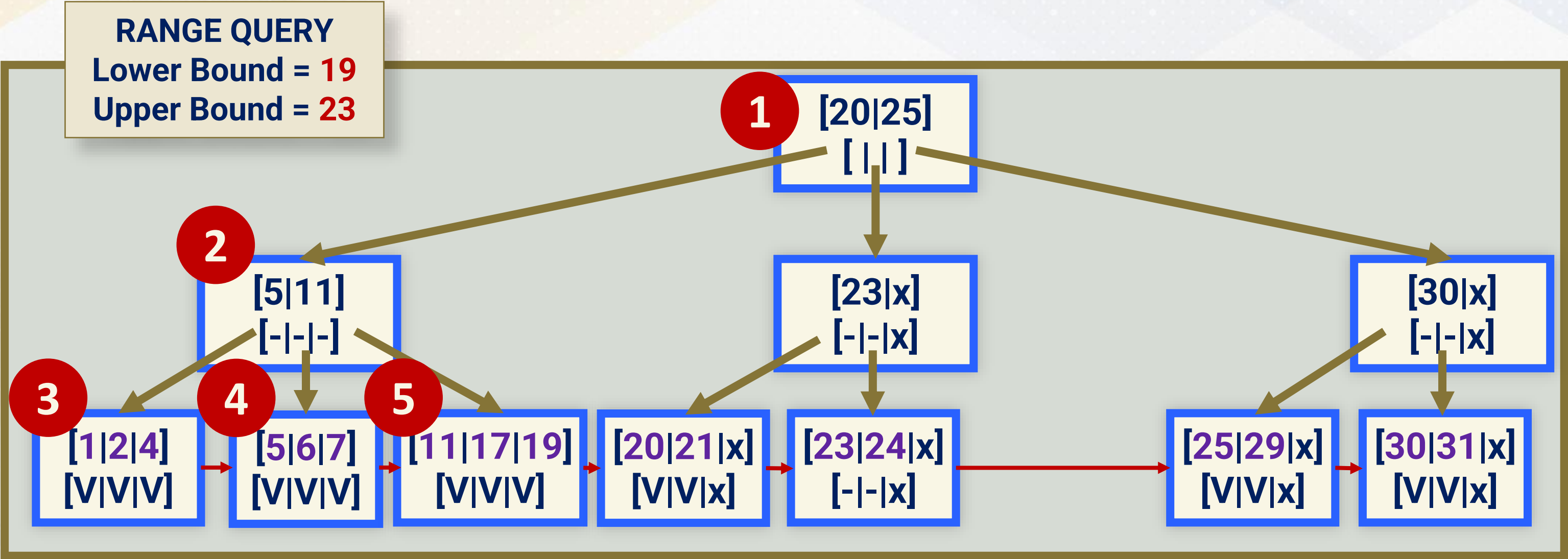
Inefficient Range Query Processing



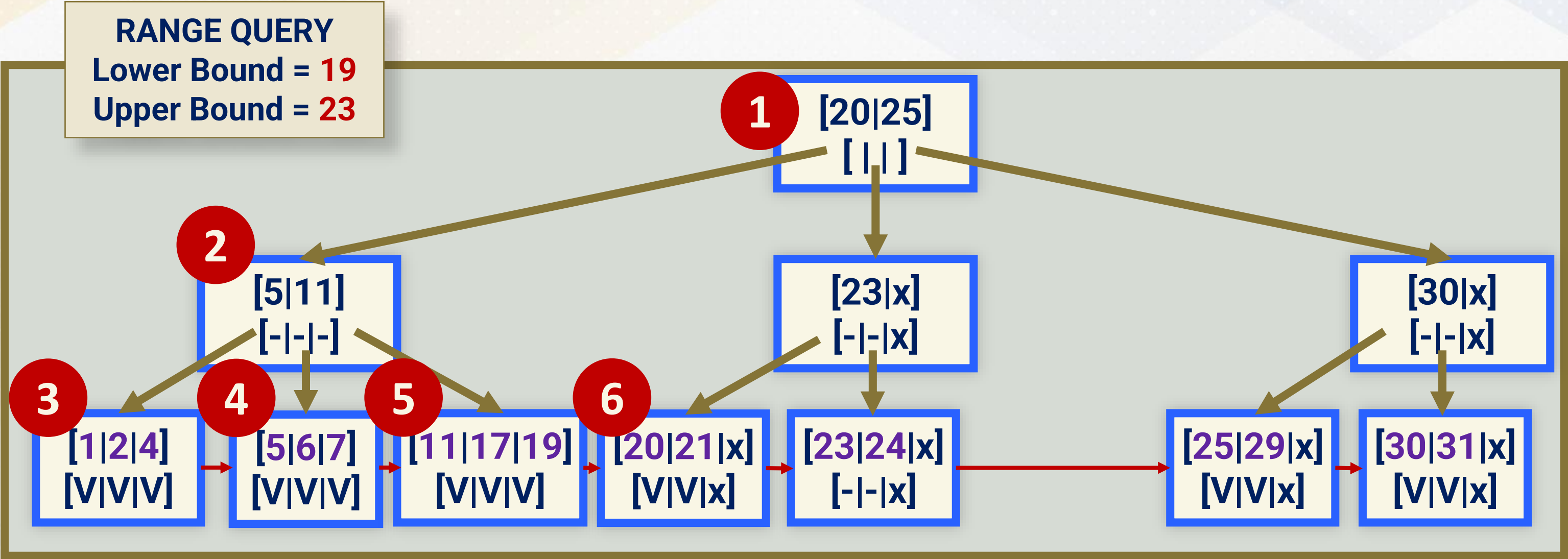
Inefficient Range Query Processing



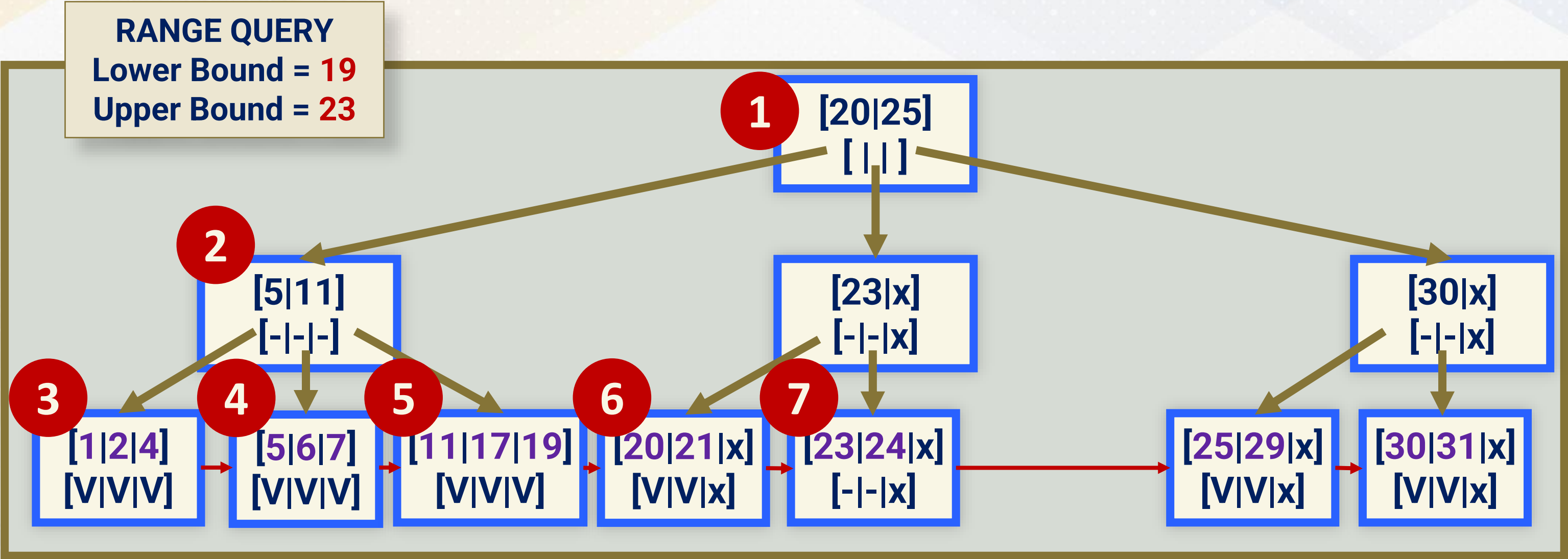
Inefficient Range Query Processing



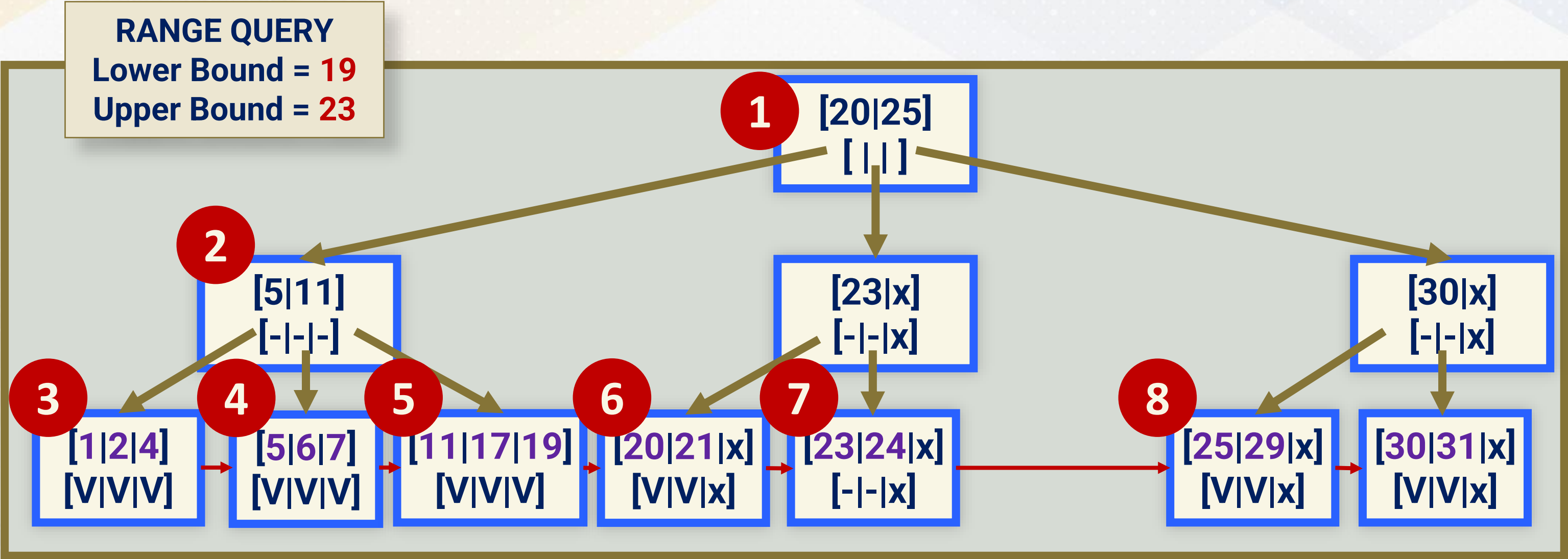
Inefficient Range Query Processing



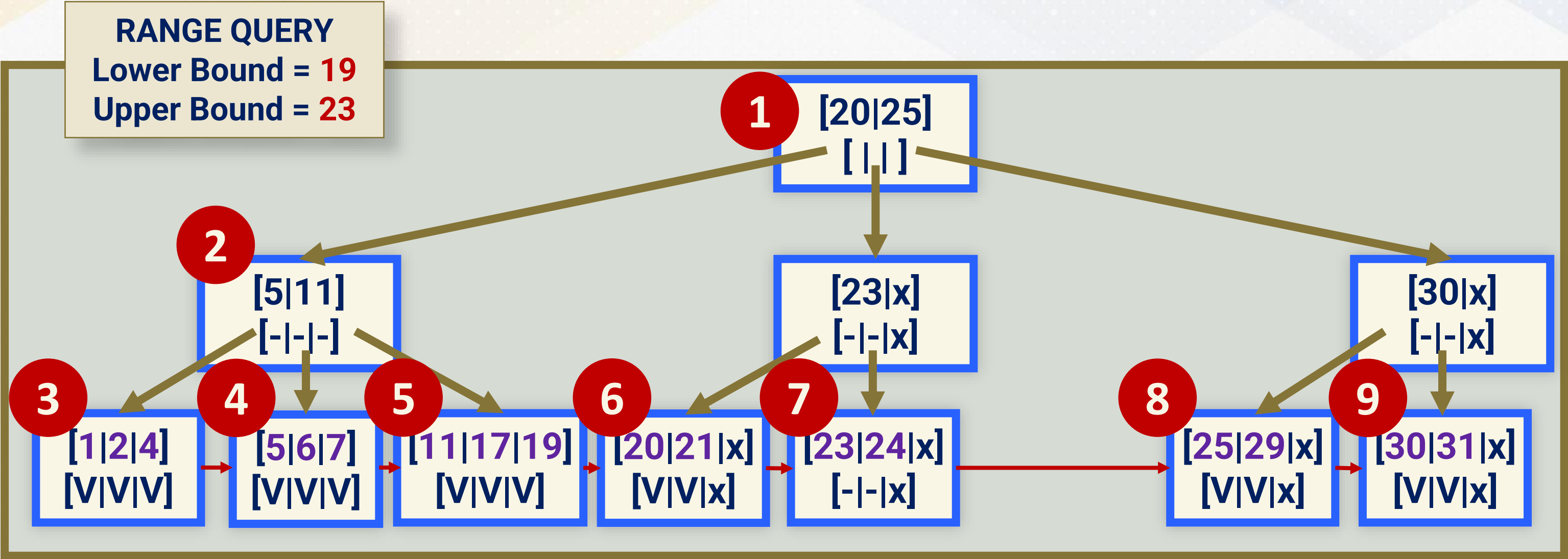
Inefficient Range Query Processing



Inefficient Range Query Processing



Inefficient Range Query Processing

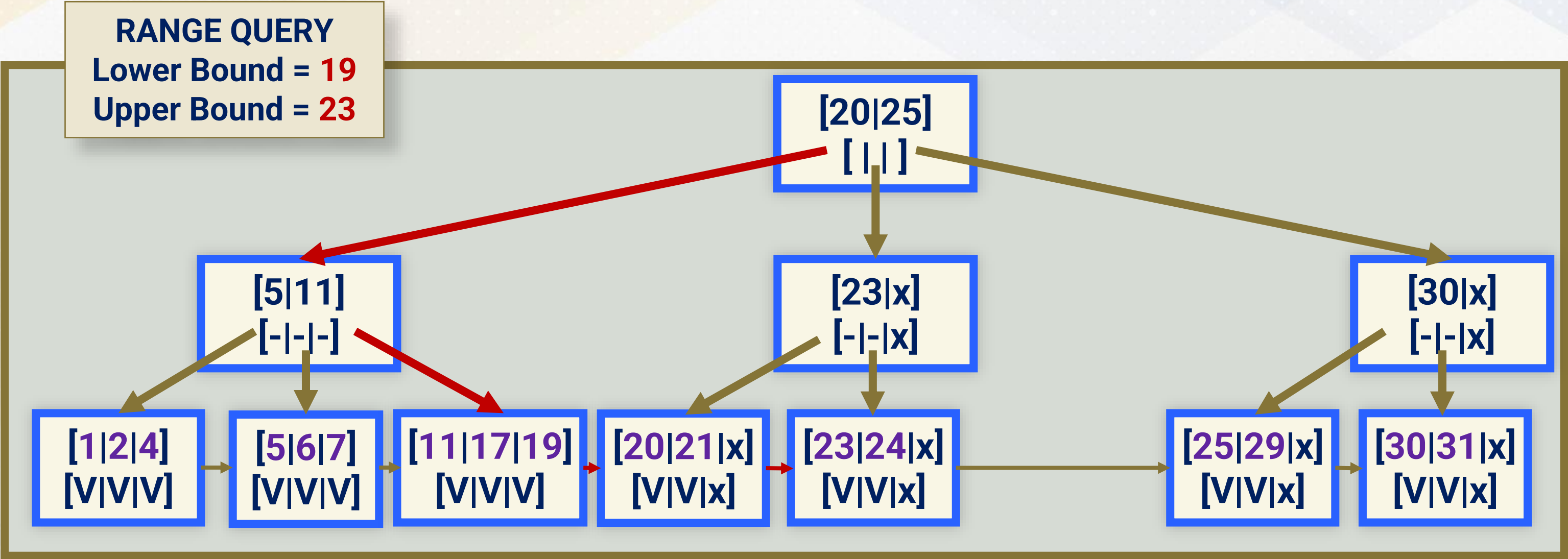


Inefficient Range Query Processing

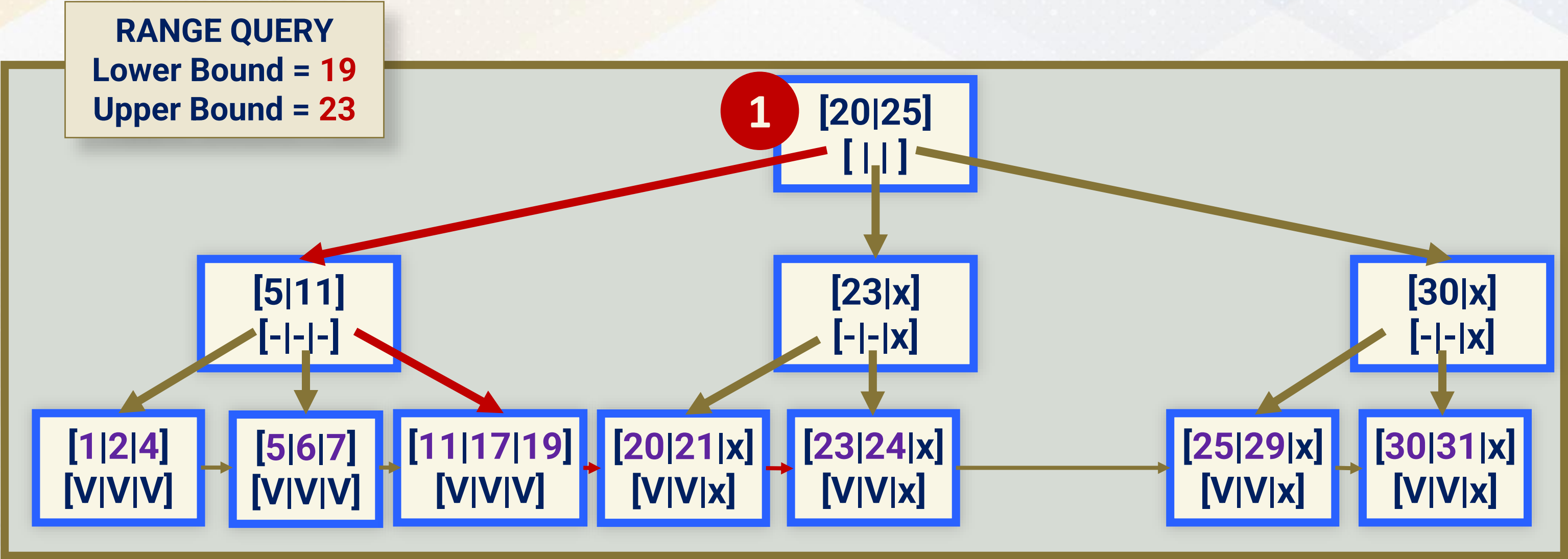
The rangeQuery method begins at the root and traverses down to the left-most leaf, then continues horizontally through the leaves collecting all qualifying values.

```
std::vector<Value> rangeQuery(const Key &lowerBound,
                             const Key &upperBound) const {
    std::vector<Value> result;
    // Traverse to leftmost leaf
    while (!currentNode->isLeaf) {
        currentNode = currentNode->entries.front().next.get();
    }
    // Traverse all the leaf nodes
    while (currentNode != nullptr) {
        // Add relevant entries within lower and upper bounds
        currentNode = currentNode->next.get();
    }
}
```

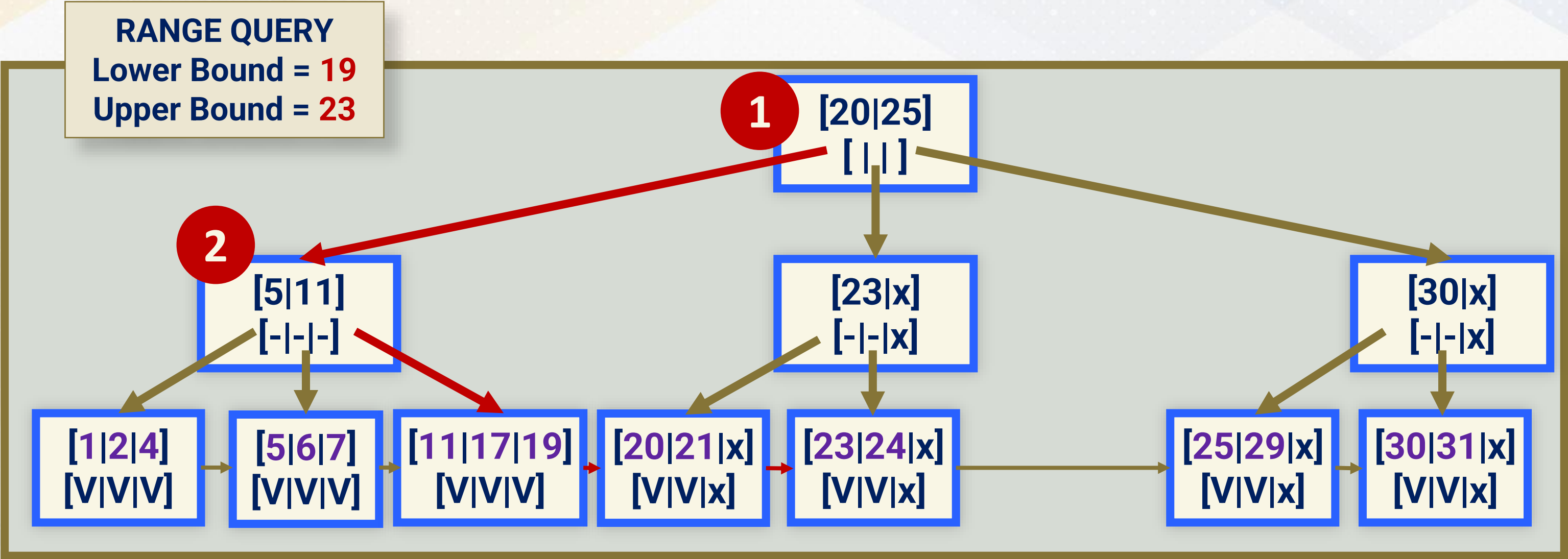

Efficient Range Query Processing



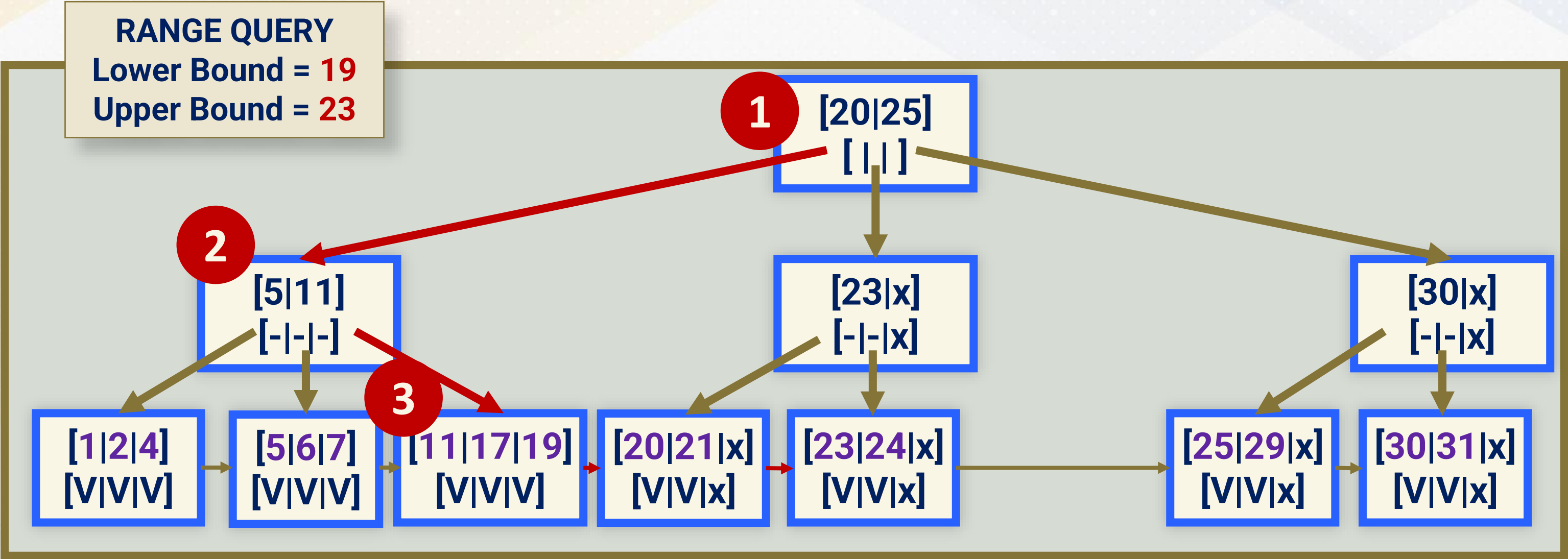
Efficient Range Query Processing



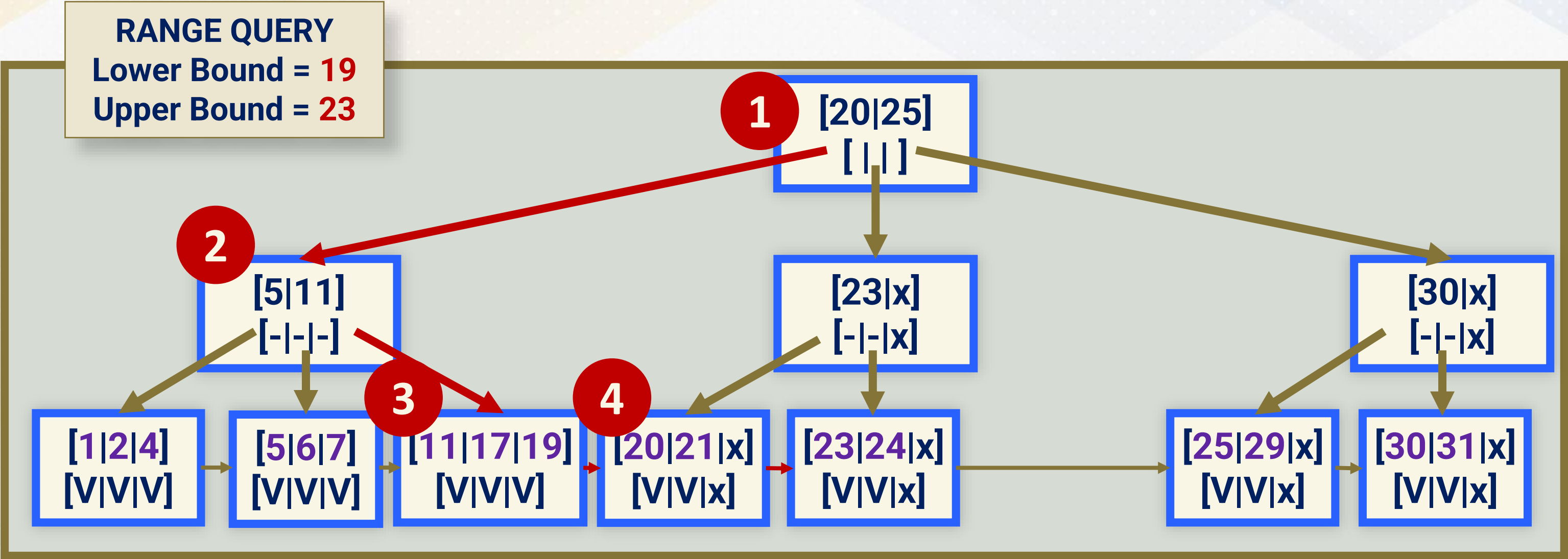
Efficient Range Query Processing



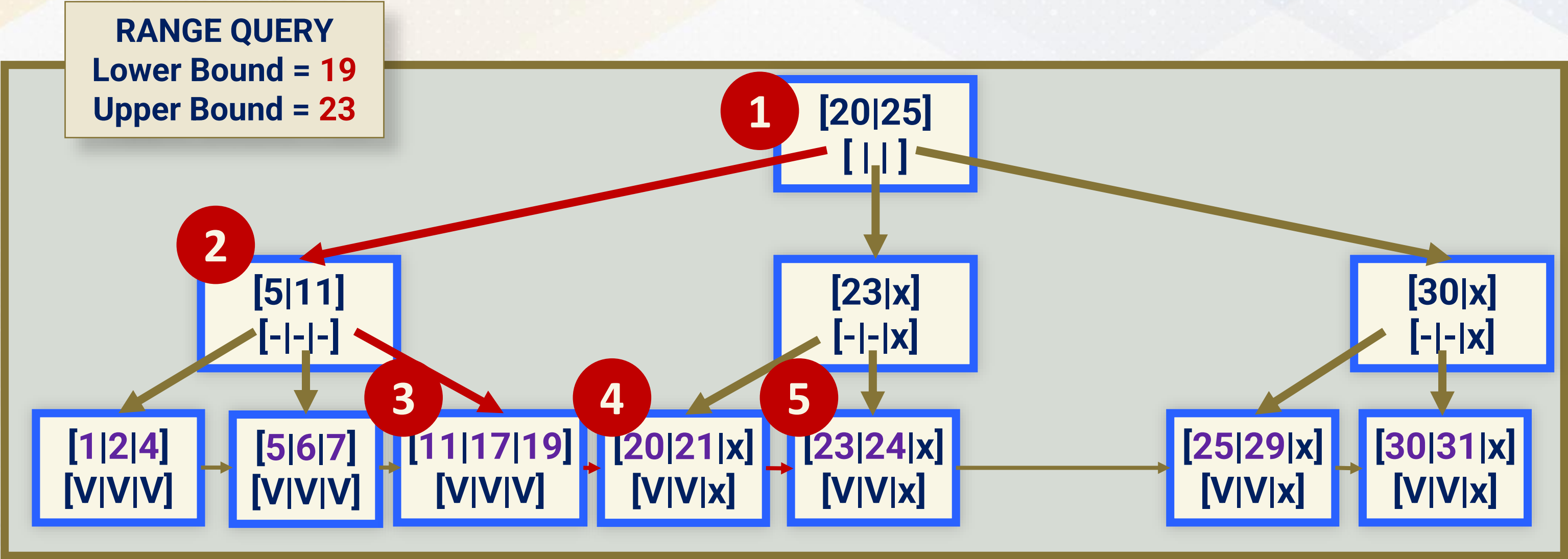
Efficient Range Query Processing



Efficient Range Query Processing



Efficient Range Query Processing

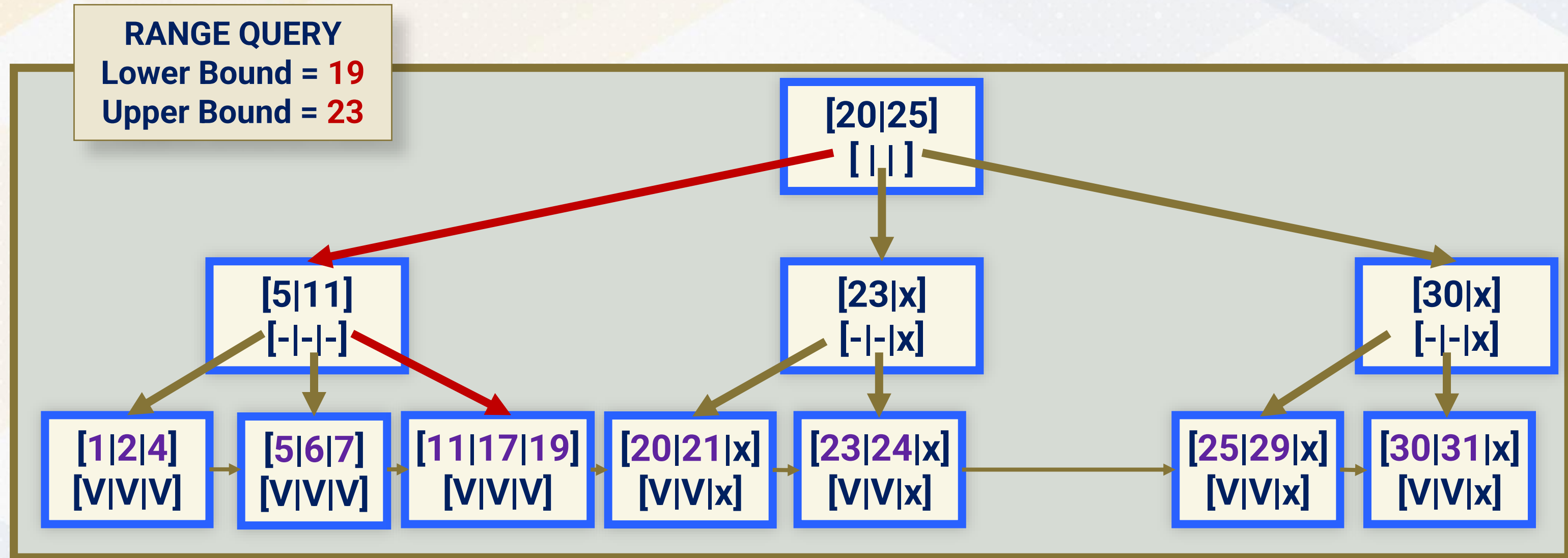


Vertical Traversal to Leaf Node

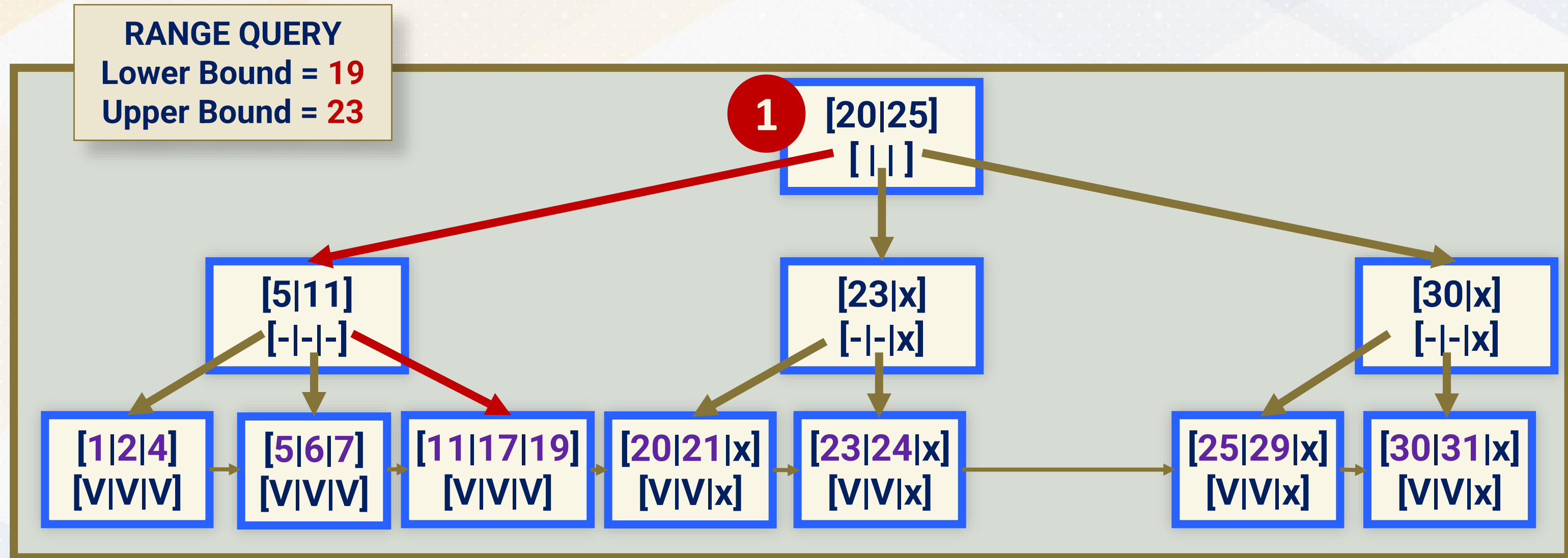
Move to appropriate child until a leaf node is reached by comparing lowerBound with node key, ensuring the starting point is as close as possible to lowerBound.

```
std::vector<Value> rangeQuery(const Key &lowerBound,
                             const Key &upperBound) const {
    auto node = root;
    while (node && !node->isLeaf) {
        size_t i = 0;
        while (i < node->keys.size() && lowerBound > node->keys[i]) {
            ++i;
        }
        node = node->children[i];
    }
    ...
}
```

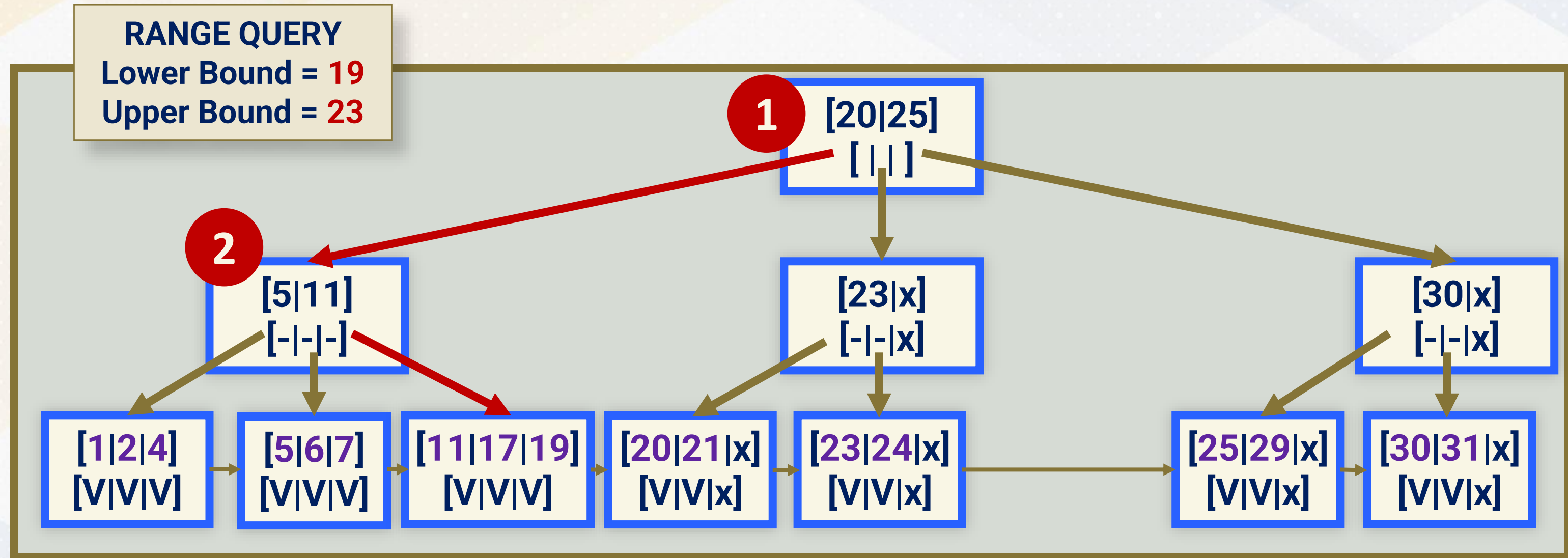
Vertical Traversal to Leaf Node



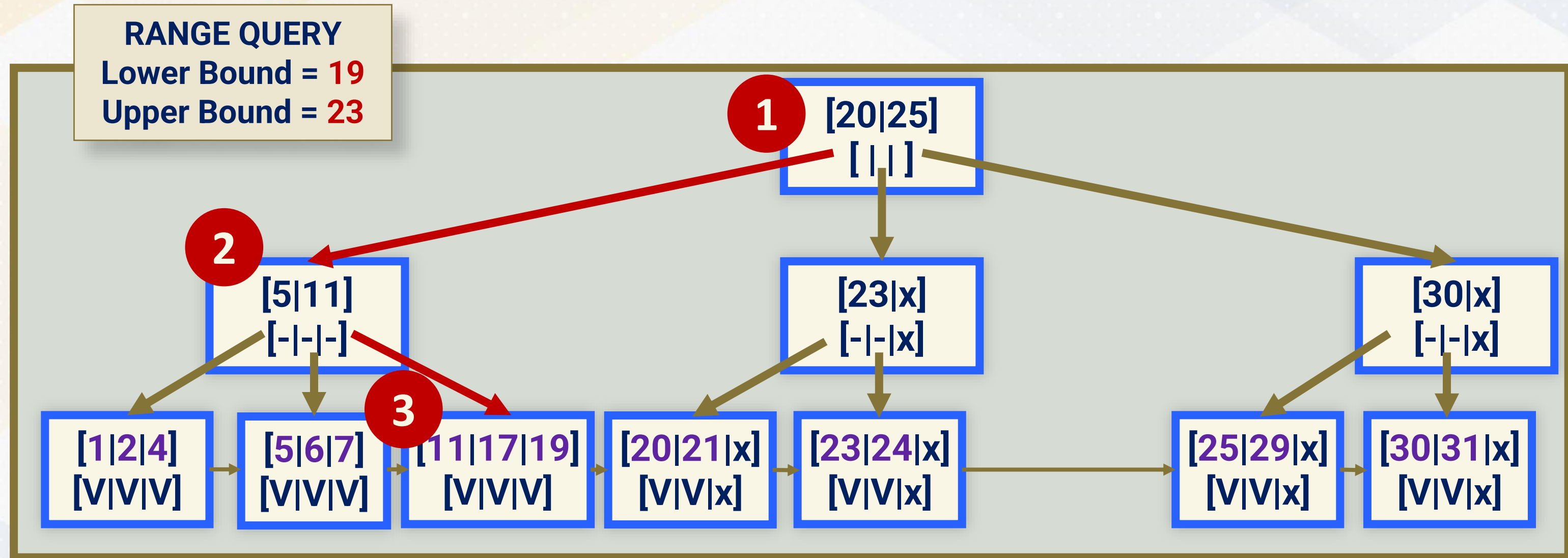
Vertical Traversal to Leaf Node



Vertical Traversal to Leaf Node



Vertical Traversal to Leaf Node

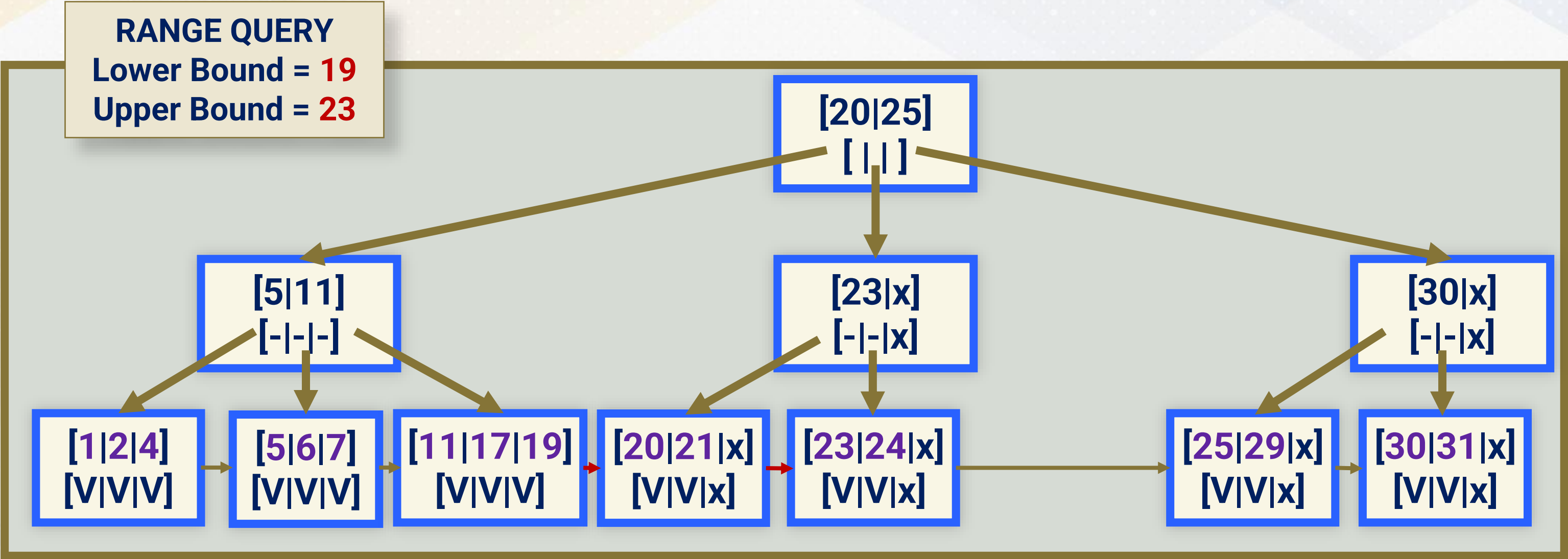


Horizontal Traversal Across Leaf Nodes

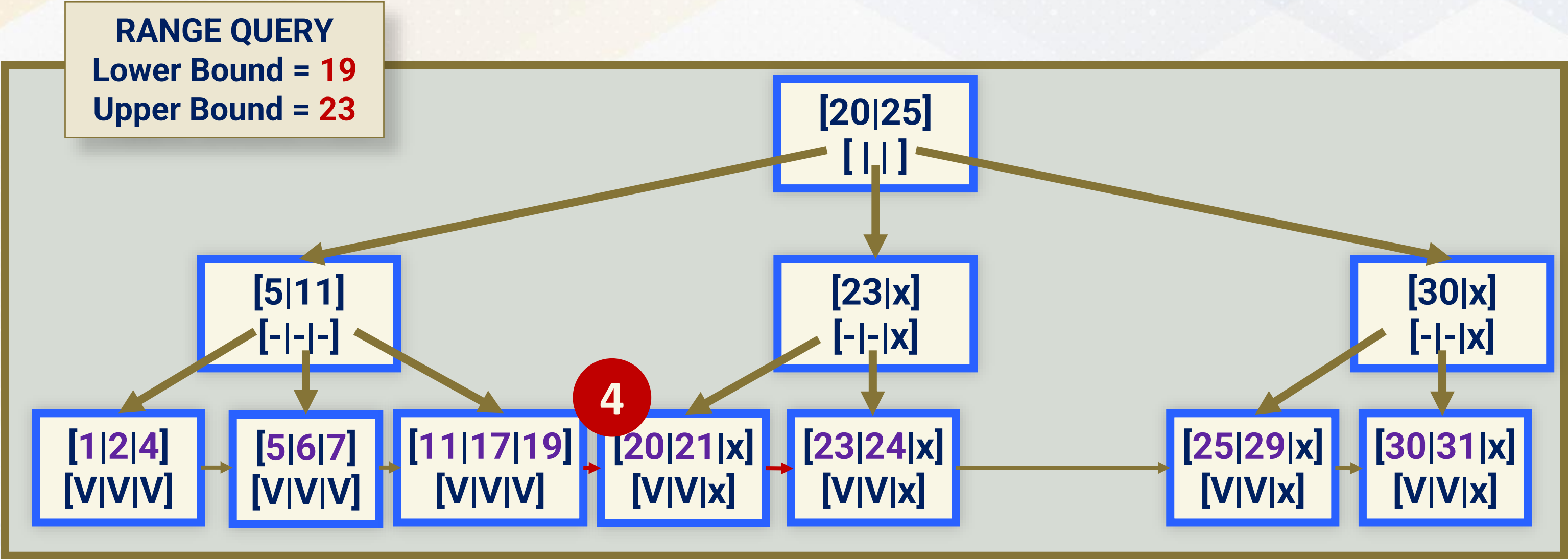
Once the appropriate leaf node is reached, the function iterates only through relevant nodes using the linked nature of leaf nodes in a B+Tree.

```
while (node) {  
    for (size_t i = 0; i < node->keys.size(); ++i) {  
        if (node->keys[i] > upperBound)  
            return result;  
        if (node->keys[i] >= lowerBound) {  
            result.push_back(node->values[i]);  
        }  
    }  
    node = node->next;  
}
```

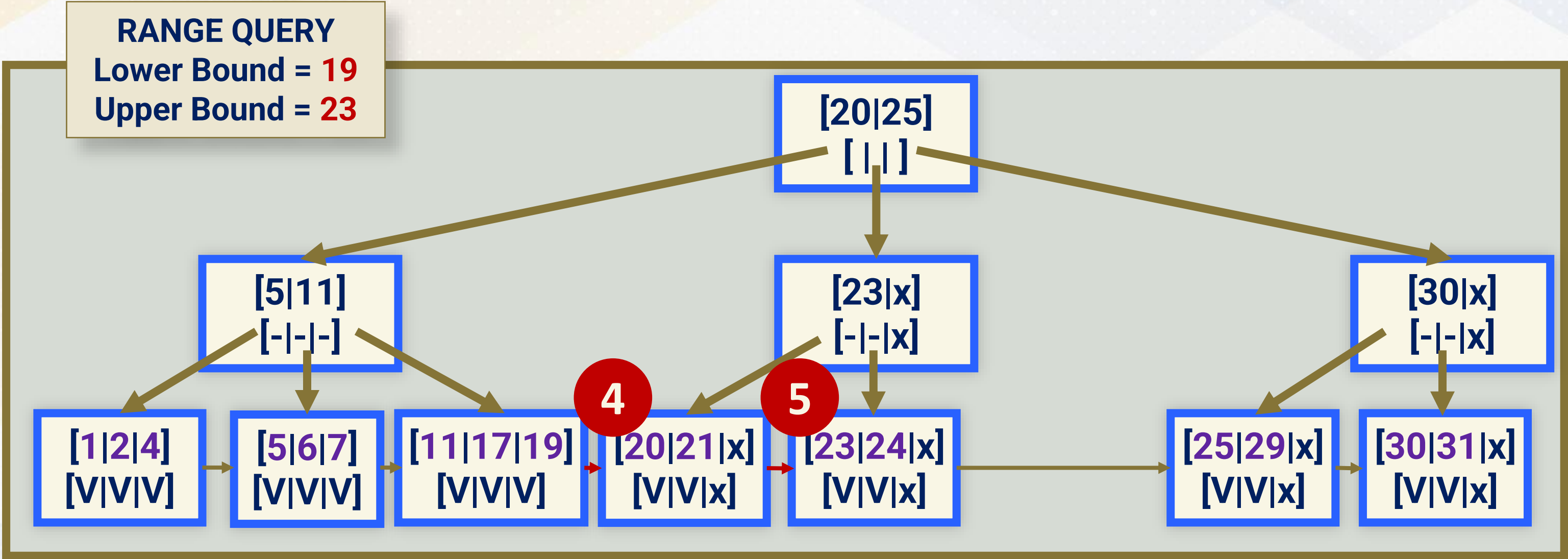

Horizontal Traversal Across Leaf Nodes



Horizontal Traversal Across Leaf Nodes



Horizontal Traversal Across Leaf Nodes







Inefficient Range Query Processing



**Inefficient
Range Query
Processing**



**Efficient
Range Query
Processing**

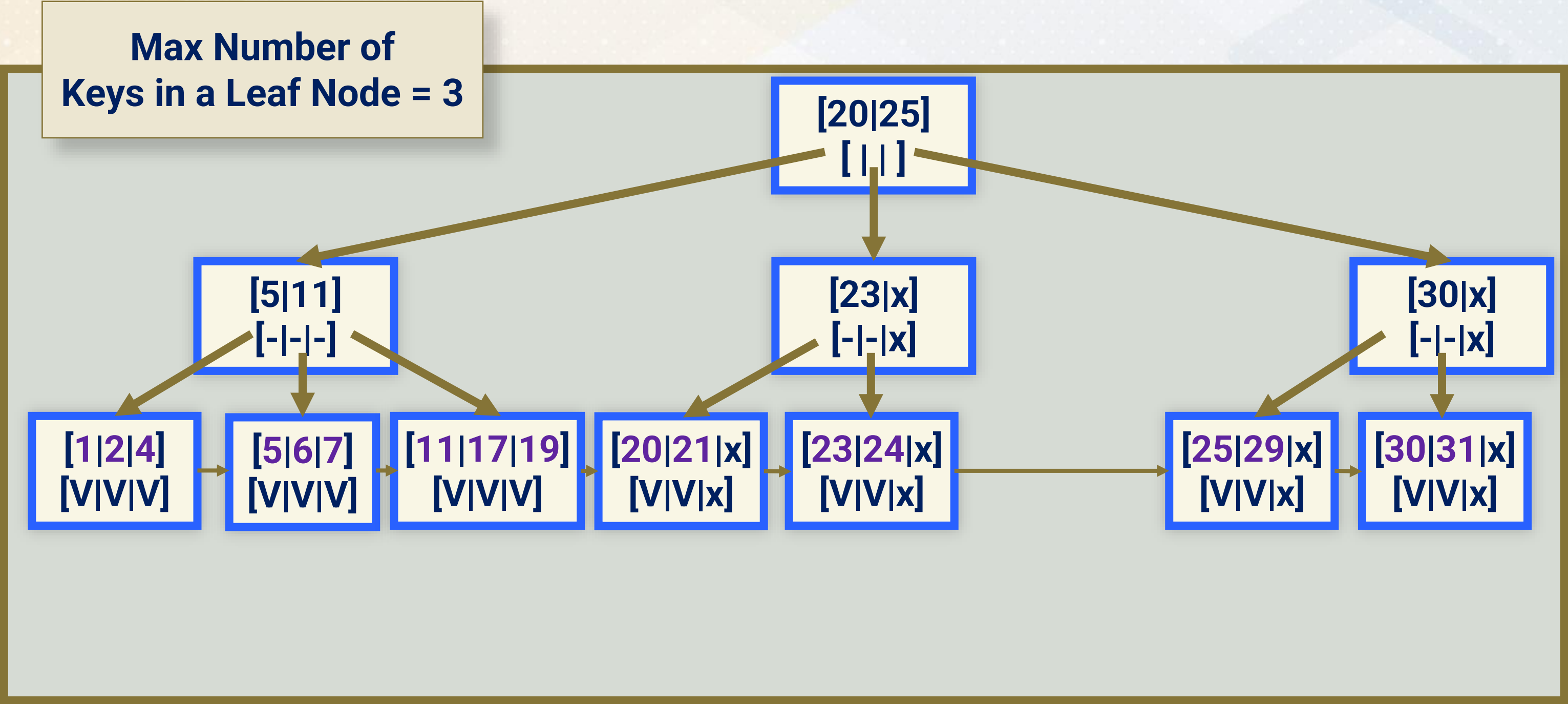
Node Split

Separate Vectors for Keys and Values

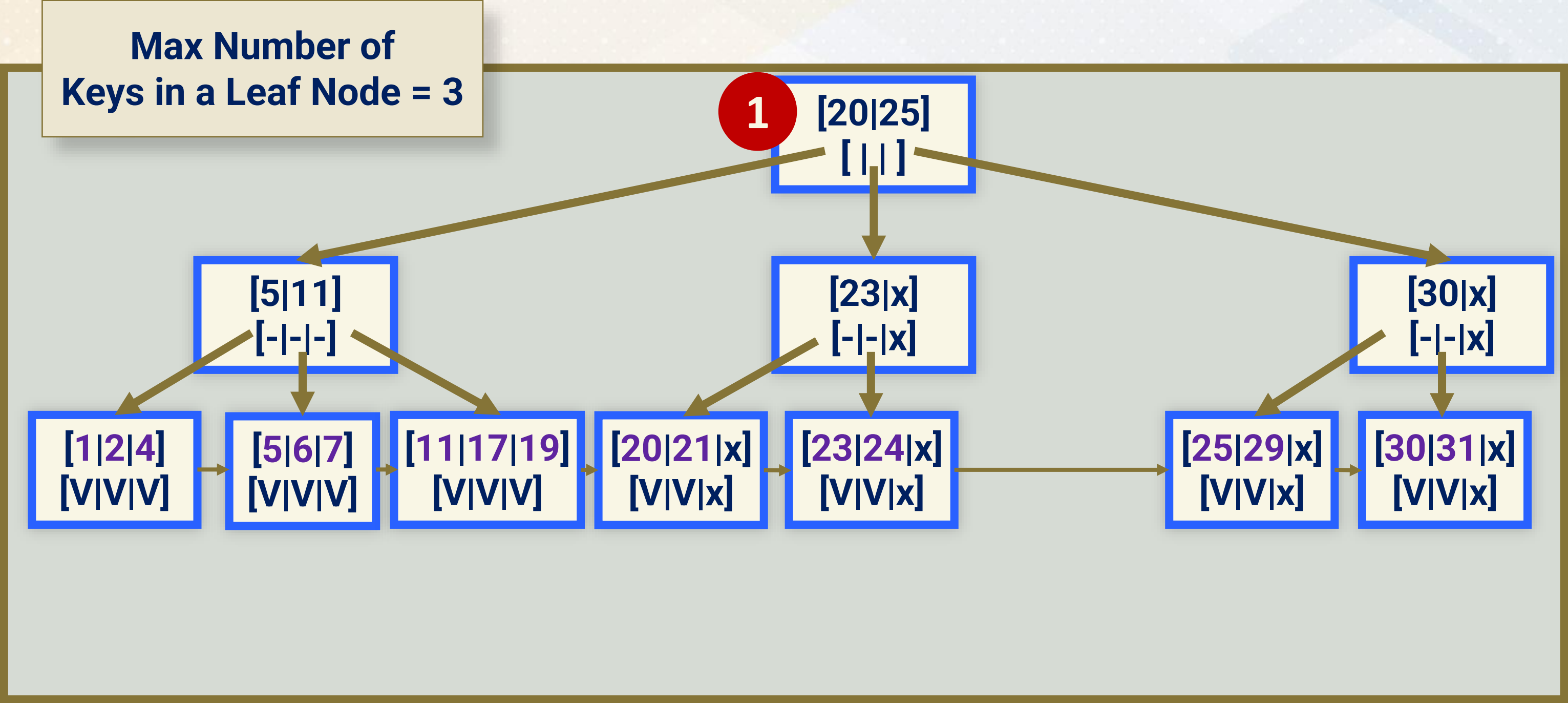
Node contains keys and values in separate vectors.

```
template <typename Key, typename Value> class BPlusTree {  
    struct Node {  
        std::vector<Key> keys;  
        std::vector<Value> values;           // Only used in leaf nodes  
        std::vector<std::shared_ptr<Node>> children; // Only used in internal nodes  
        std::shared_ptr<Node> next = nullptr;    // Next leaf node  
        bool isLeaf = false;  
        Node(bool leaf) : isLeaf(leaf) {}  
    };  
    size_t maxKeys; // Order of the tree  
    std::shared_ptr<Node> root;  
};
```

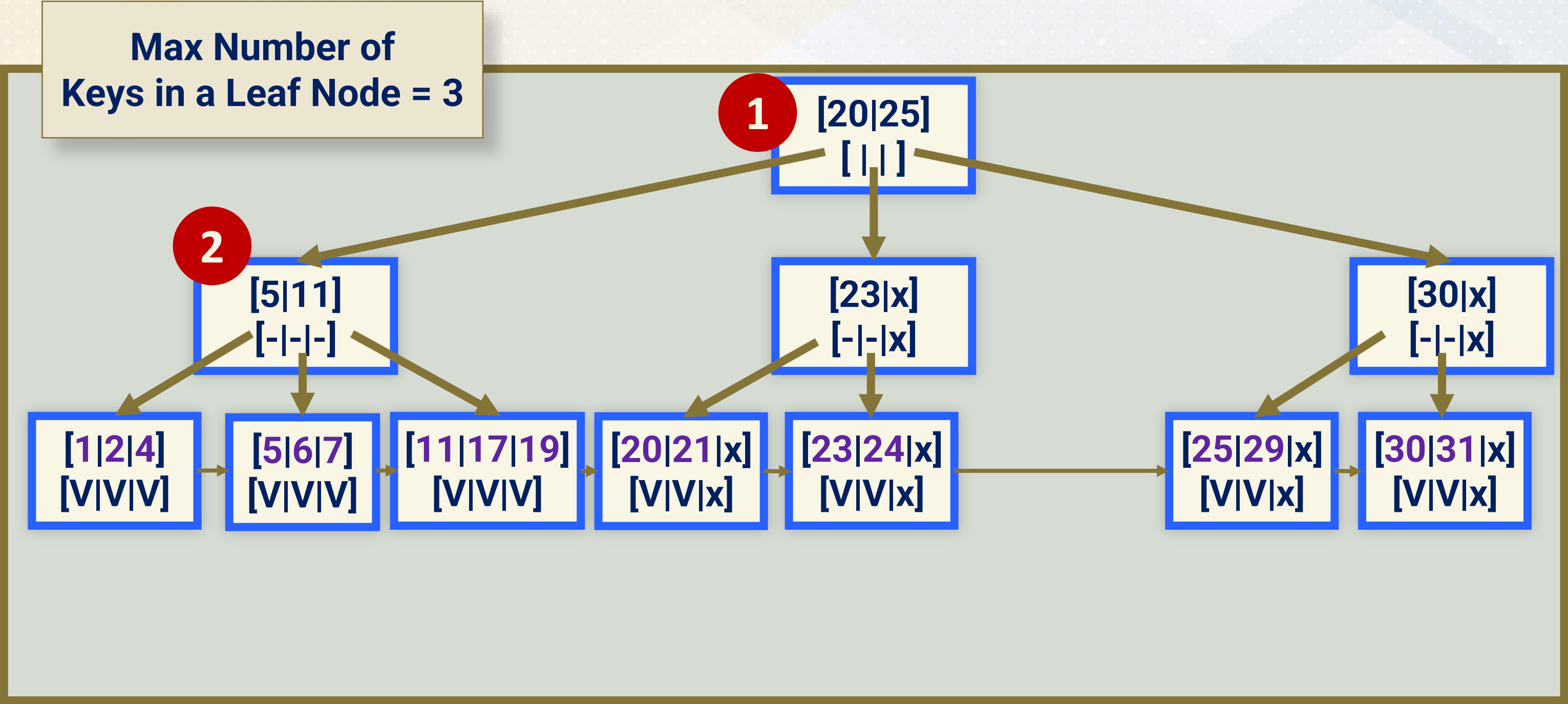
Node Split: Inserting [8, Z]



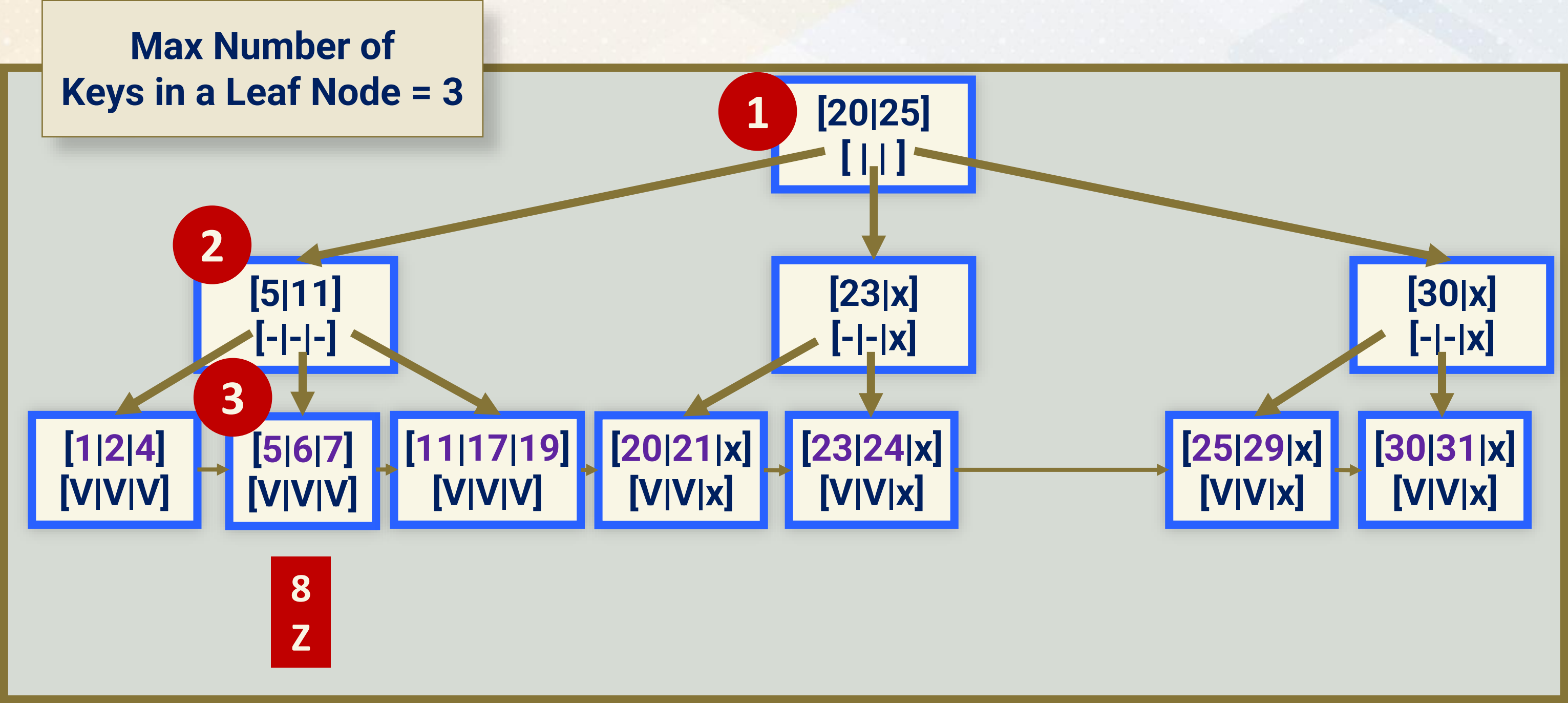
Node Split: Inserting [8, Z]



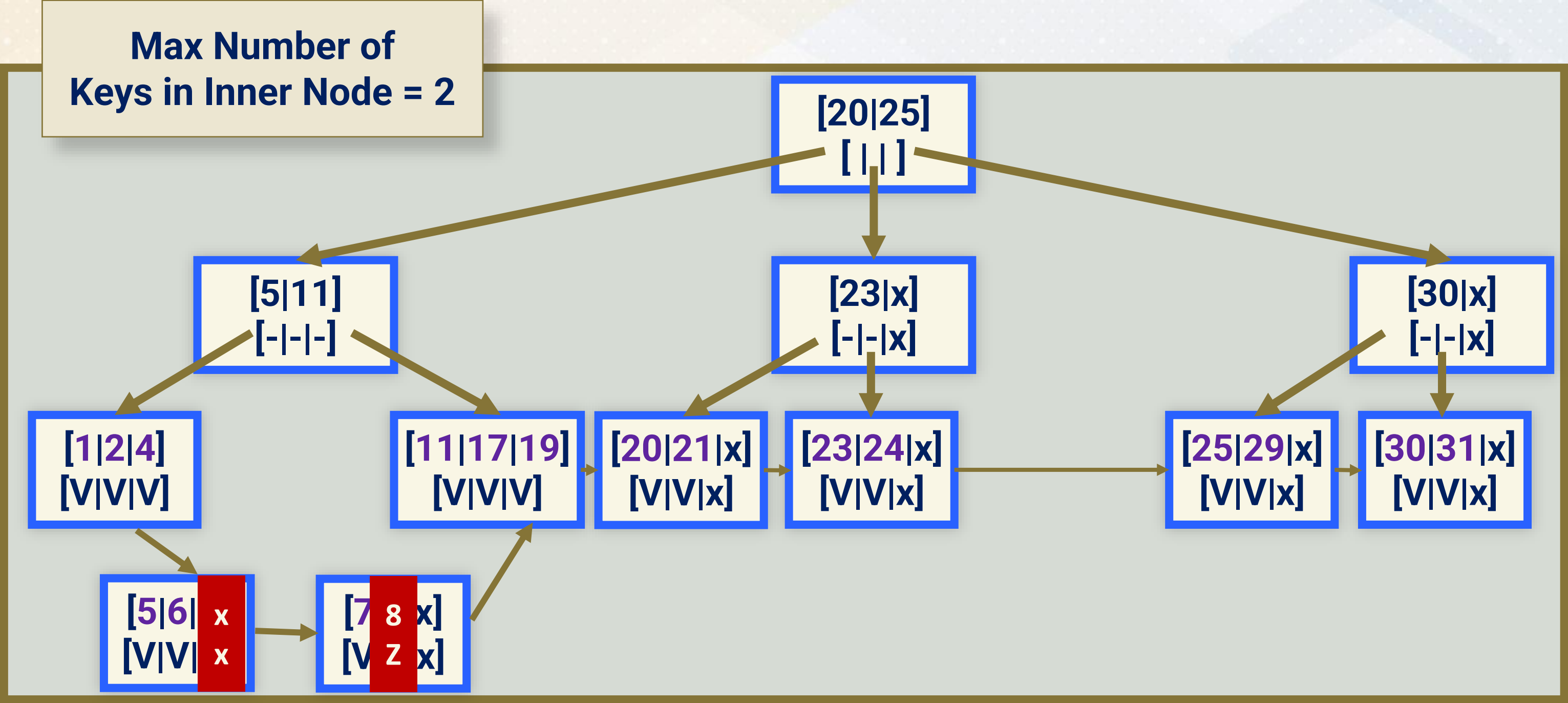
Node Split: Inserting [8, Z]



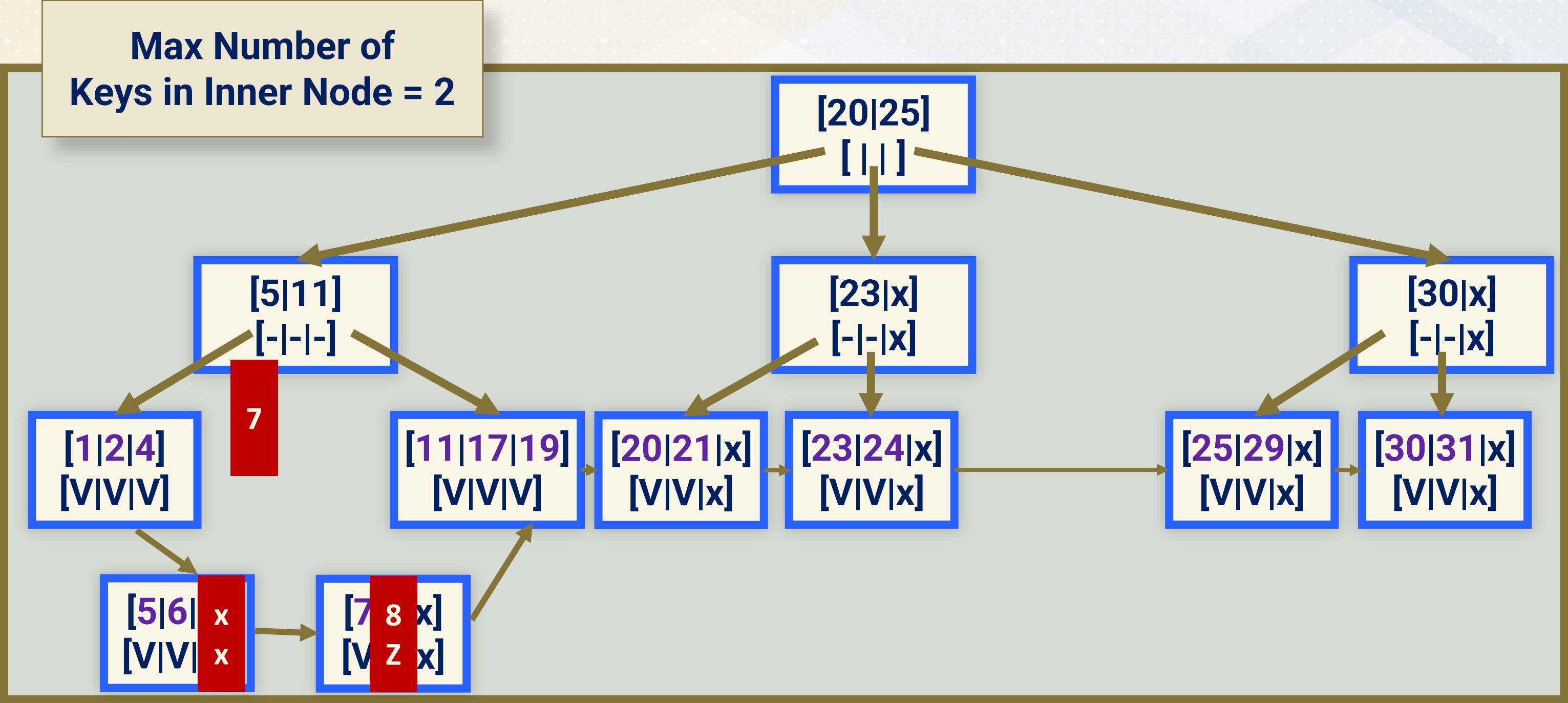
Node Split: Inserting [8, Z]



Node Split: Inserting [8, Z]

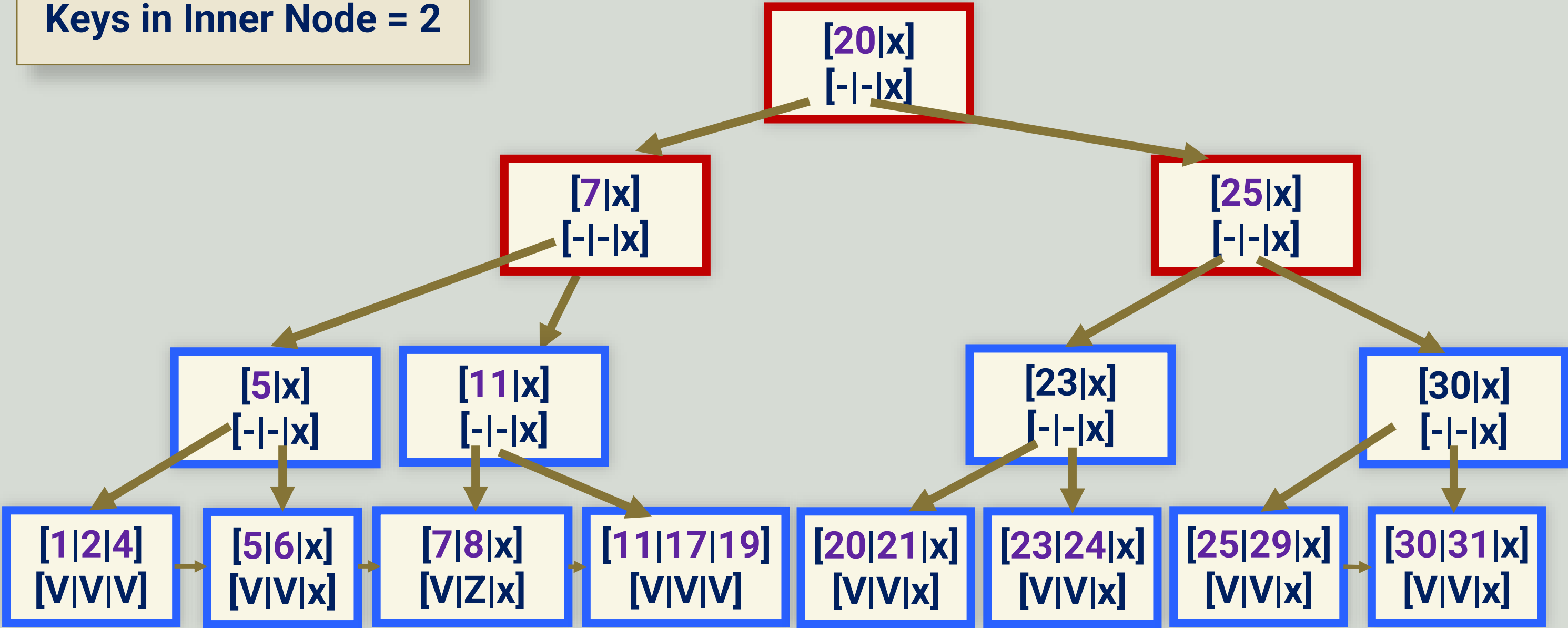


Node Split: Inserting [8, Z]



Node Split: Inserting [8, Z]

Max Number of
Keys in Inner Node = 2



Node Split in C++

buzzDB

```
if (node->keys.size() > maxKeys) {  
    splitNode(path, node);  
}
```

Node Split in C++

buzzDB

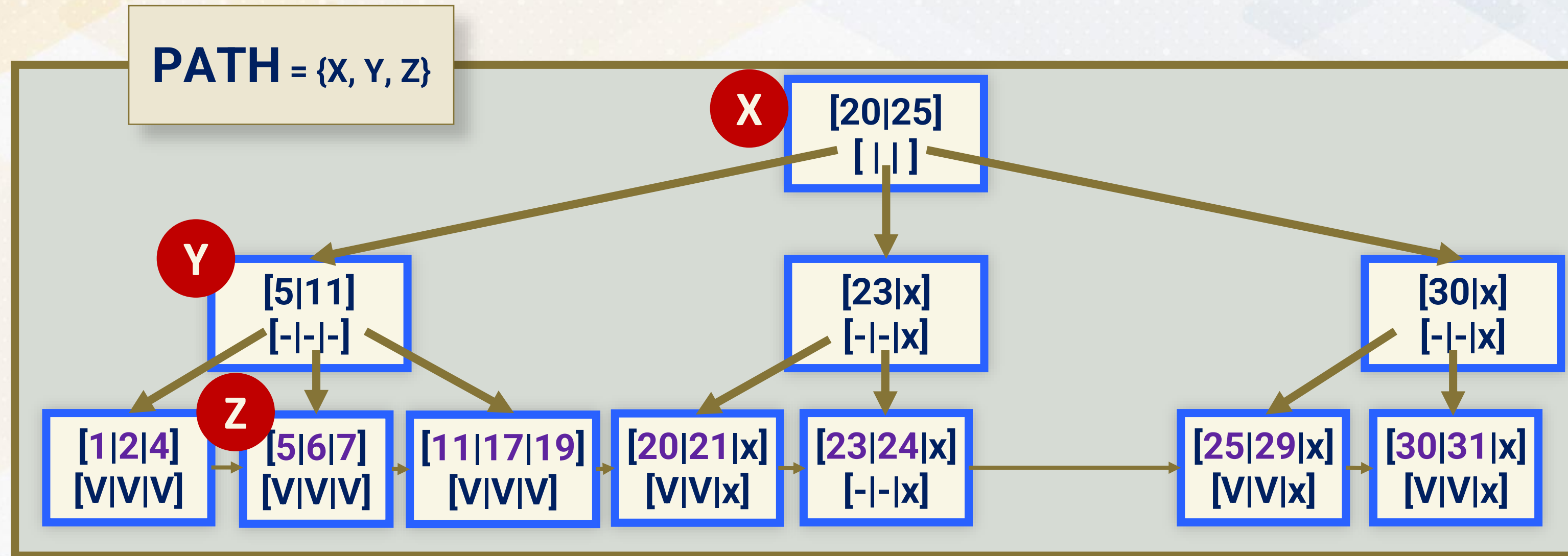
Node Exceeds Capacity → Split Node

```
if (node->keys.size() > maxKeys) {  
    splitNode(path, node);  
}
```


Insertion

```
auto node = root;
std::vector<std::shared_ptr<Node>> path; // Track the path for backtracking
while (!node->isLeaf) {
    path.push_back(node);
    auto it = std::upper_bound(node->keys.begin(), node->keys.end(), key);
    size_t index = it - node->keys.begin();
    node = node->children[index];
}
```


Path Tracking and Backtracking



Insertion in Leaf Node

```
auto it = std::lower_bound(node->keys.begin(), node->keys.end(), key);
if (it != node->keys.end() && *it == key) {
    size_t pos = std::distance(node->keys.begin(), it);
    node->values[pos] += value; // Update existing key's value
}
else {
    size_t pos = it - node->keys.begin();
    node->keys.insert(node->keys.begin() + pos, key);
    node->values.insert(node->values.begin() + pos, value);
}
```

Node Splitting in Leaf Node

```
if (node->isLeaf) {  
    auto newNode = std::make_shared<Node>(true); // Create a new leaf node  
    size_t mid = node->keys.size() / 2; // Calculate the middle index  
    // Move the second half of keys and values to the new node  
    std::move(node->keys.begin() + mid, node->keys.end(),  
              std::back_inserter(newNode->keys));  
    std::move(node->values.begin() + mid, node->values.end(),  
              std::back_inserter(newNode->values));  
    newNode->next = node->next;  
    node->next = newNode;  
}
```


Node Splitting in Inner Node

```
else {                                // Handling internal nodes
    auto newNode = std::make_shared<Node>(false); // Create a new internal node
    size_t mid = node->keys.size() / 2;        // Middle index
    Key midKey = node->keys[mid]; // Key that will move to the parent
    // Move keys and children to the new node
    std::move(node->keys.begin() + mid + 1, node->keys.end(),
               std::back_inserter(newNode->keys));
    std::move(node->children.begin() + mid + 1, node->children.end(),
               std::back_inserter(newNode->children));
}
```

Backtracking to Parent Node

```
else {  
    auto parent = path.back(); // Get the parent node  
    path.pop_back();          // Remove the last tracked node  
    size_t pos = std::distance(  
        parent->keys.begin(),  
        std::lower_bound(parent->keys.begin(), parent->keys.end(), midKey));  
    parent->keys.insert(parent->keys.begin() + pos, midKey); // Insert median key  
    parent->children.insert(parent->children.begin() + pos + 1, newNode); // Insert  
}
```

Backtracking to Parent Node

Path Vector

Determines
Parent Node

```
else {  
    auto parent = path.back(); // Get the parent node  
    path.pop_back();          // Remove the last tracked node  
    size_t pos = std::distance(  
        parent->keys.begin(),  
        std::lower_bound(parent->keys.begin(), parent->keys.end(), midKey));  
    parent->keys.insert(parent->keys.begin() + pos, midKey); // Insert median key  
    parent->children.insert(parent->children.begin() + pos + 1, newNode); // Insert  
}
```


Backtracking to Parent Node

Path Vector

Determines
Parent Node

Median Key
Insertion

Parent Node May
Exceed Capacity

```
else {  
    auto parent = path.back(); // Get the parent node  
    path.pop_back();          // Remove the last tracked node  
    size_t pos = std::distance(  
        parent->keys.begin(),  
        std::lower_bound(parent->keys.begin(), parent->keys.end(), midKey));  
    parent->keys.insert(parent->keys.begin() + pos, midKey); // Insert median key  
    parent->children.insert(parent->children.begin() + pos + 1, newNode); // Insert  
}
```

Backtracking to Parent Node

Path Vector

Determines
Parent Node

Median Key Insertion

Parent Node May
Exceed Capacity

Recursion

Can Travel Up to
Root Node

```
else {  
    auto parent = path.back(); // Get the parent node  
    path.pop_back();          // Remove the last tracked node  
    size_t pos = std::distance(  
        parent->keys.begin(),  
        std::lower_bound(parent->keys.begin(), parent->keys.end(), midKey));  
    parent->keys.insert(parent->keys.begin() + pos, midKey); // Insert median key  
    parent->children.insert(parent->children.begin() + pos + 1, newNode); // Insert  
}
```


Backtracking to Root Node

Backtracking

```
if (path.empty()) {  
    // New root for the tree  
    auto newRoot = std::make_shared<Node>(false);  
    newRoot->keys.push_back(midKey);    // Add median key  
    newRoot->children.push_back(node);  // Left child  
    newRoot->children.push_back(newNode); // Right child  
    // Update root pointer  
    root = newRoot;  
}
```


Backtracking to Root Node

Backtracking

Empty Path Vector

```
if (path.empty()) {  
    // New root for the tree  
    auto newRoot = std::make_shared<Node>(false);  
    newRoot->keys.push_back(midKey);    // Add median key  
    newRoot->children.push_back(node);  // Left child  
    newRoot->children.push_back(newNode); // Right child  
    // Update root pointer  
    root = newRoot;  
}
```

Backtracking to Root Node

Backtracking

Empty Path Vector

New Root Node

```
if (path.empty()) {  
    // New root for the tree  
    auto newRoot = std::make_shared<Node>(false);  
    newRoot->keys.push_back(midKey);    // Add median key  
    newRoot->children.push_back(node);  // Left child  
    newRoot->children.push_back(newNode); // Right child  
    // Update root pointer  
    root = newRoot;  
}
```


Backtracking to Root Node

Backtracking

Empty Path Vector

New Root Node

Two Children

```
if (path.empty()) {  
    // New root for the tree  
    auto newRoot = std::make_shared<Node>(false);  
    newRoot->keys.push_back(midKey);    // Add median key  
    newRoot->children.push_back(node);  // Left child  
    newRoot->children.push_back(newNode); // Right child  
    // Update root pointer  
    root = newRoot;  
}
```


Backtracking to Root Node

Backtracking

Empty Path Vector

New Root Node

Two Children

Tree Height += 1

```
if (path.empty()) {  
    // New root for the tree  
    auto newRoot = std::make_shared<Node>(false);  
    newRoot->keys.push_back(midKey);    // Add median key  
    newRoot->children.push_back(node);  // Left child  
    newRoot->children.push_back(newNode); // Right child  
    // Update root pointer  
    root = newRoot;  
}
```

Node Split

On-Disk B+Tree





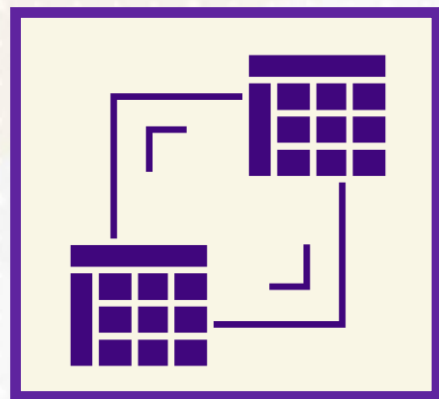
Adaptation for Disk



**Adaptation for
Disk**

**On-Disk
B+ Tree**

Need for On-Disk B+Tree



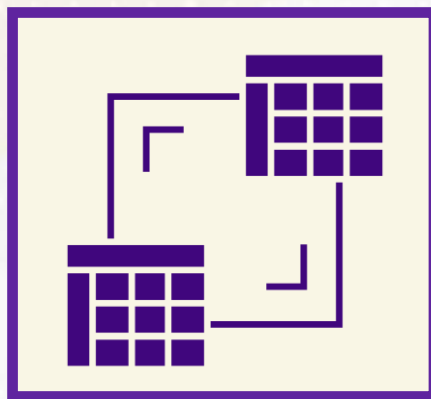
Need for On-Disk B+Tree



Scalability

Can Index Large
Tables

Due to Disk
Capacity



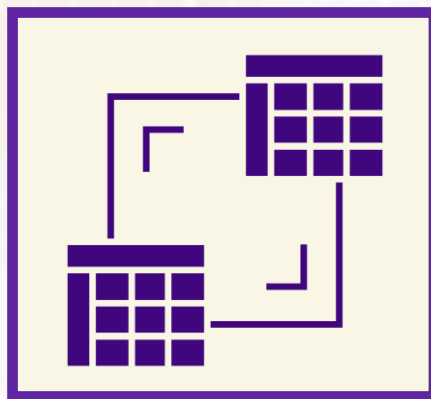
Need for On-Disk B+Tree



Scalability

Can Index Large
Tables

Due to Disk
Capacity



Durability

Index Rebuild
Unnecessary at
System Restart

Due to Disk
Durability

Adaptation for Disk



```
struct Node {  
    Key keys[MAX_KEYS];           // Array to store keys  
    uint32_t children[MAX_KEYS + 1]; // File offsets for child nodes  
    uint32_t next;                // File offset for the next leaf node  
    bool isLeaf;  
    int keyCount;  
};  
  
std::vector<uint8_t> serializeNodeToBytes(const DiskNode &node);  
void deserializeNodeFromBytes(const std::vector<uint8_t> &buffer,  
                               DiskNode &node);
```

Calculating Size of Node Struct

Size of Node struct with m keys =
 $m \times (\text{size of key} + \text{size of value}) + \text{size of an extra value} +$
 $\text{size of next pointer} + \text{size of isLeaf} + \text{size of keyCount} =$
 $m \times (4 + 4) + 4 + 4 + 1 + 4 = m \times 8 + 13$

Calculating Size of Node Struct



Size of Node struct with m keys =
 $m \times (\text{size of key} + \text{size of value}) + \text{size of an extra value} +$
 $\text{size of next pointer} + \text{size of isLeaf} + \text{size of keyCount} =$
 $m \times (4 + 4) + 4 + 4 + 1 + 4 = m \times 8 + 13$

Aligning Node Struct with Disk Page Size



Page



SSD

// 4 KB disk page size

$m \times 8 + 13 \leq 4096$ bytes

$m \approx 510$ keys

// 16 KB disk page size

$m \times 8 + 13 \leq 16384$ bytes

$m \approx 2046$ keys

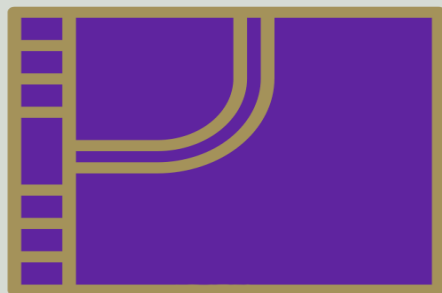
Aligning Node Struct with Disk Page Size



Page

4 kB Page

M = 510 keys



SSD

// 4 KB disk page size

$m \times 8 + 13 \leq 4096$ bytes

$m \approx 510$ keys

// 16 KB disk page size

$m \times 8 + 13 \leq 16384$ bytes

$m \approx 2046$ keys

Aligning Node Struct with Disk Page Size



Page

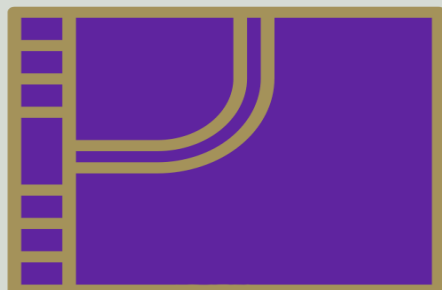
4 kB Page

$M = 510$ keys

// 4 KB disk page size

$m \times 8 + 13 \leq 4096$ bytes

$m \approx 510$ keys



SSD

16 kB Page

$M = 2046$ keys

// 16 KB disk page size

$m \times 8 + 13 \leq 16384$ bytes

$m \approx 2046$ keys

Capacity of a B+Tree

Capacity of a B+Tree

Really Wide

Super Short

High Fan Out

Page Size

Fan Out

Levels

4 kB

510

3

Leaf Pages

$510^{(LEVELS - 1)} = 510^{(3 - 1)} = 510^2 \text{ PAGES}$

Max Keys

$510^{(LEVELS)} = 510^3 = 132 \text{ Million KEYS}$

B+Tree File Size

$510^2 \text{ PAGES} * 4 \text{ KB / PAGE} = 1 \text{ GB}$

Capacity of a B+Tree (4 Levels)

Page Size

4 kB

Fan Out

510

Levels

3

Capacity of a B+Tree (4 Levels)

Page Size

4 kB

Fan Out

510

Levels

4

Leaf Pages

$510^{(LEVELS - 1)} = 510^{(4 - 1)} = 510^3 \text{ PAGES}$

Max Keys

$510^{(LEVELS)} = 510^4 = 67 \text{ Billion KEYS}$

B+Tree File Size

$510^3 \text{ PAGES} * 4 \text{ KB / PAGE} = 0.5 \text{ TB}$



Serialization & Deserialization



Serialization

Convert an in-memory DiskNode structure to a byte stream suitable for disk

```
std::vector<uint8_t> serializeNodeToBytes(const DiskNode &node) {  
    std::vector<uint8_t> buffer;  
    // Assuming each key and offset is of fixed size, e.g., 4 bytes for int keys  
    for (const auto &key : node.keys) {  
        auto bytes = reinterpret_cast<const uint8_t *>(&key);  
        buffer.insert(buffer.end(), bytes, bytes + sizeof(Key));  
    }  
    ...  
    return buffer;  
}
```


Deserialization

Convert a byte stream read from disk back to an in-memory Node

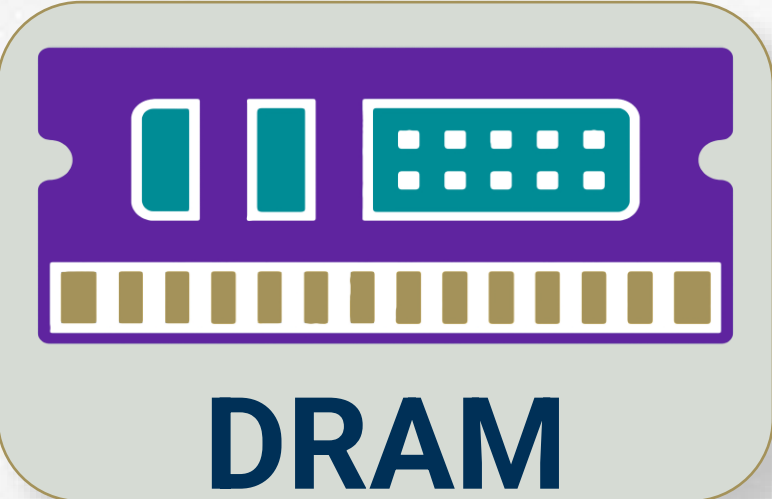
```
void deserializeNodeFromBytes(const std::vector<uint8_t> &buffer, DiskNode &node) {  
    size_t offset = 0;  
    // Assuming we know the number of keys and that Key is of a fixed size  
    for (int i = 0; i < node.keyCount; ++i) {  
        Key key;  
        std::memcpy(&key, buffer.data() + offset, sizeof(Key));  
        node.keys[i] = key;  
        offset += sizeof(Key);  
    }  
    ...  
}
```

Node Storage in On-Disk File



FILE OFFSET	
0	NODE 1
4 KB	NODE 2
12 KB	NODE 3
16 KB	NODE 4
20 KB	NODE 5
24 KB	NODE 6
30 KB	NODE 7
...	...

Minimizing Disk Reads



N 1	N 5	N 2
N 9	N 6	N 10

*Cached
Nodes*



N 1	N 2	N 3	N 4
N 5	N 6	N 7	N 8
N 9	N 10	N 11	

B+Tree

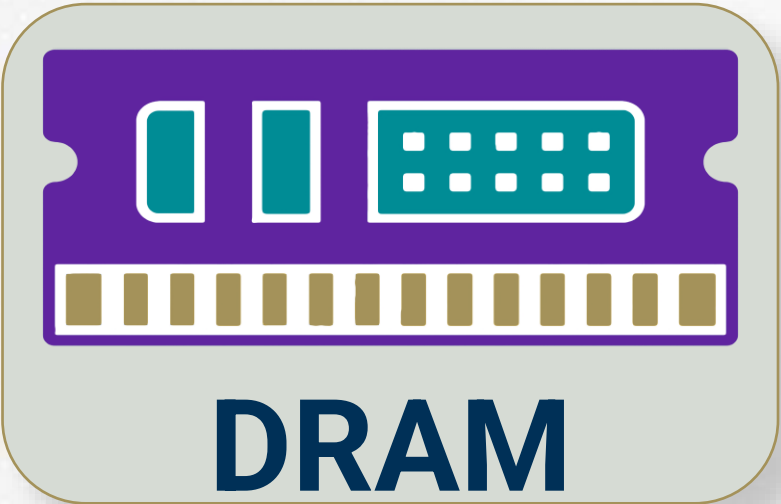


Minimizing Disk Reads

Reduce Disk
Operations

Cache B+Tree
Nodes

ROOT Node
Cached



N 1	N 5	N 2
N 9	N 6	N 10

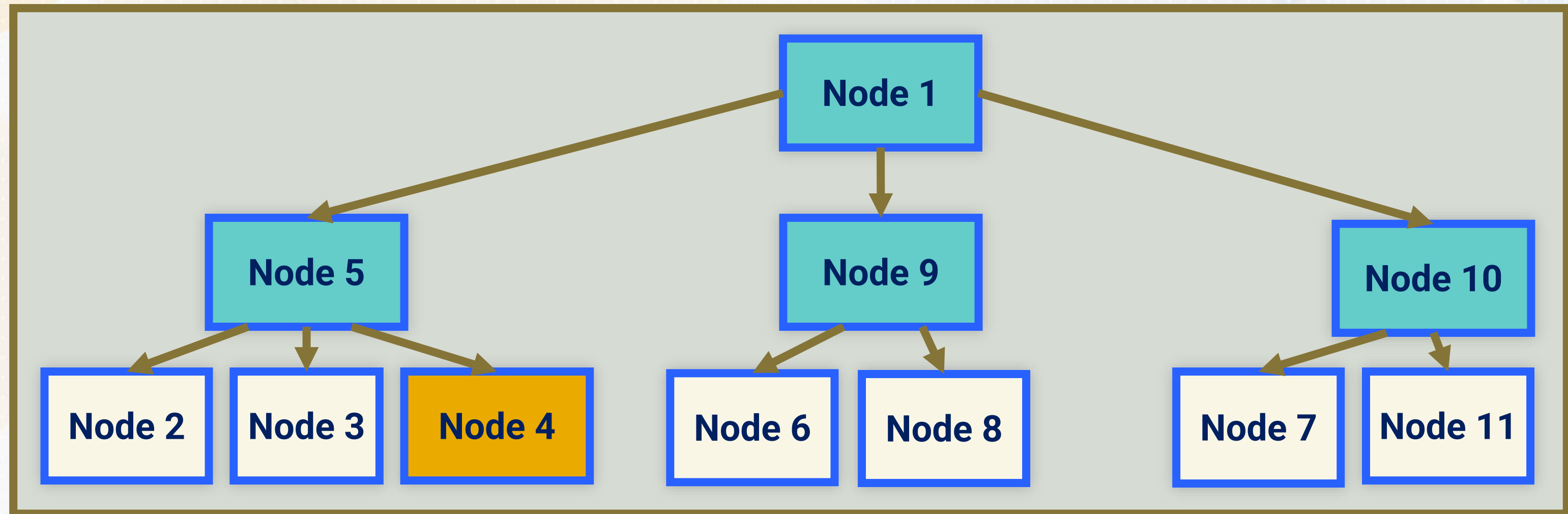
*Cached
Nodes*



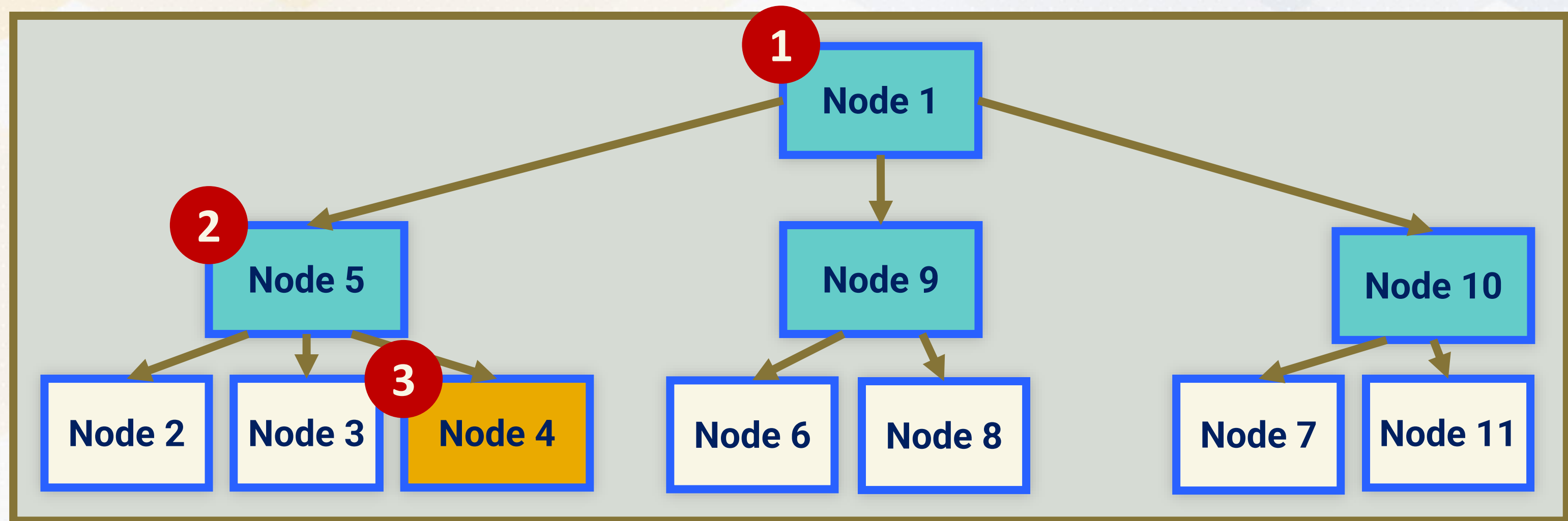
N 1	N 2	N 3	N 4
N 5	N 6	N 7	N 8
N 9	N 10	N 11	

B+Tree

Minimizing Disk Reads: Finding a Key in Node 4



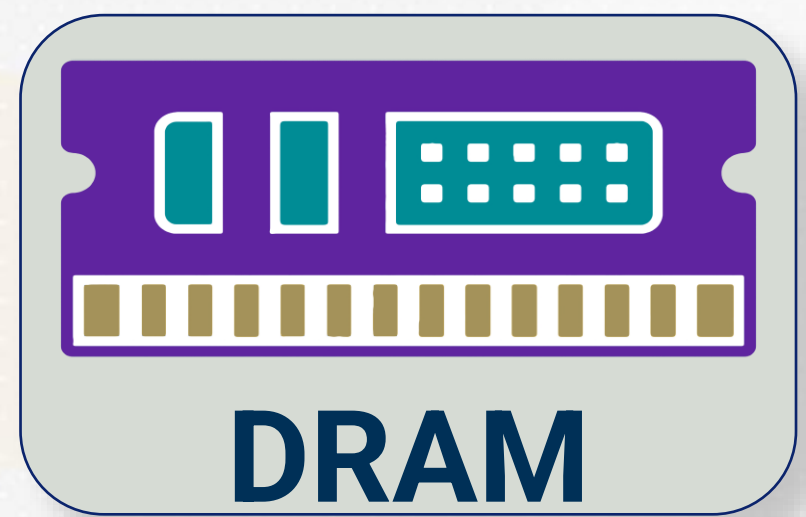
Minimizing Disk Reads: Finding a Key in Node 4



Minimizing Disk Writes

Node Eviction

Node evicted from buffer pool when it is full



N 1	N 5	N 2
N 9	N 6	N 10

*Cached
Nodes*



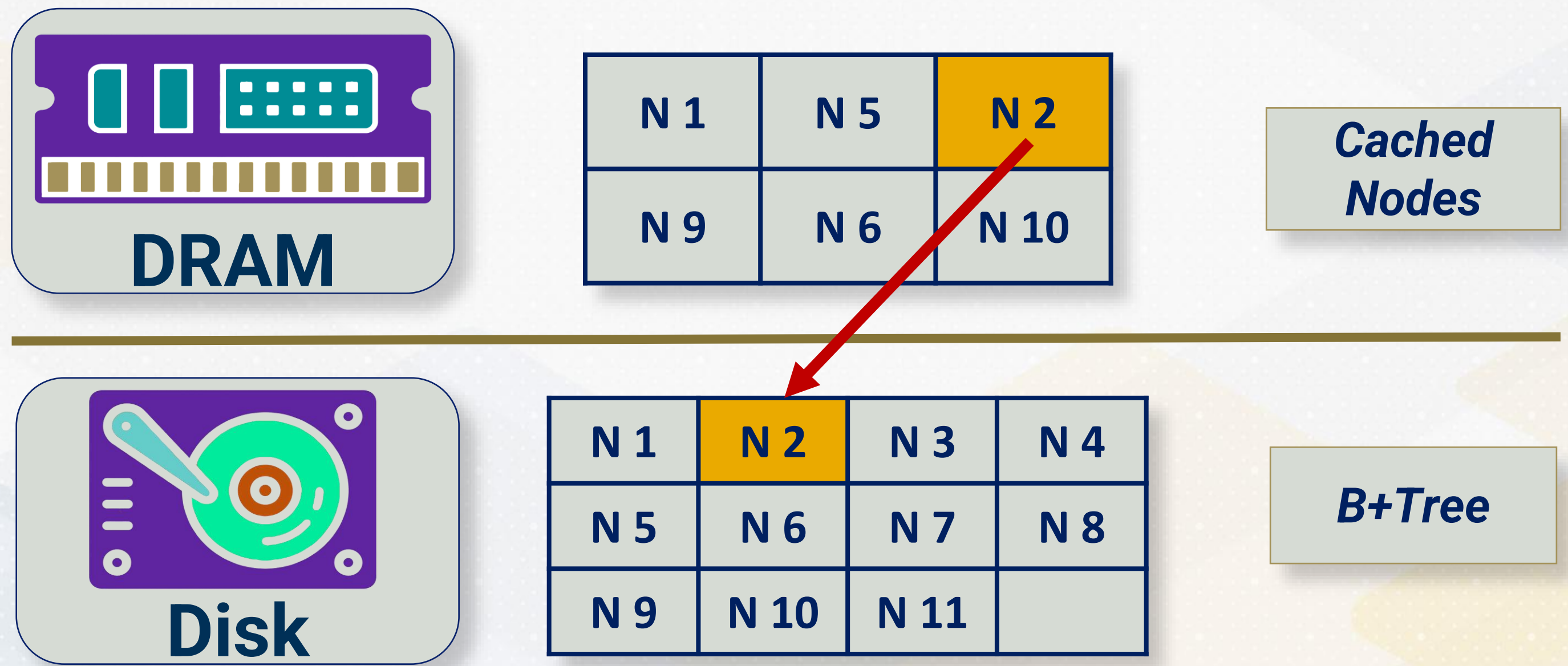
N 1	N 2	N 3	N 4
N 5	N 6	N 7	N 8
N 9	N 10	N 11	

B+Tree

Minimizing Disk Writes

Node Eviction

Node evicted from buffer pool when it is full

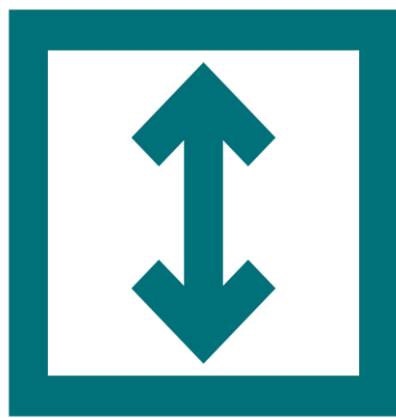


Advantages of B+Trees on Disk

B+Trees are designed for storage on disk

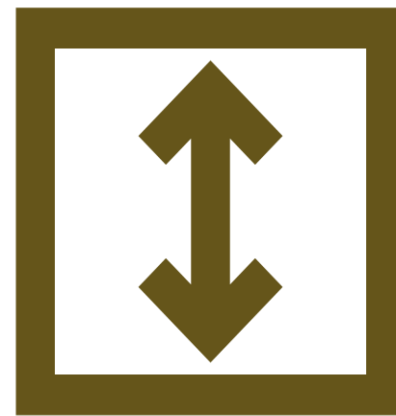
Advantages of B+Trees on Disk

B+Trees are designed for storage on disk



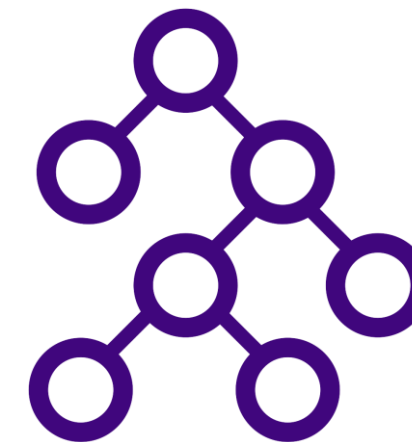
Uniform Height

All operations
need same
number of disk
reads/writes



Short Height

Higher-level inner
nodes are
buffered leading
to fewer page
misses



Linked Leaves

Range queries
can be processed
efficiently





Adaptation for Disk



**Adaptation for
Disk**

**On-Disk B+
Tree**